

EDISI GOOGLE COLAB | PYTORCH 2.11+

MASTERING PYTORCH HANDBOOK

Dari Tensor Dasar hingga Training Terdistribusi,
Generative AI, Deployment, dan MLOps

ROMI NUR ISMANTO

www.jekardah.com | hello@rominur.com

EDISI PROFESIONAL - 2026

EDISI PROFESIONAL 2026

Mastering PyTorch Handbook

Dari Tensor Dasar hingga Training Terdistribusi, Generative AI, Deployment, dan MLOps

MENTAL MODEL 01

Memulai PyTorch di Google Colab



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Romi Nur Ismantowww.jekardah.com | hello@rominur.com

TARGET LINGKUNGAN Google Colab dengan PyTorch 2.11+; bab compiler juga menjelaskan cara memeriksa kompatibilitas PyTorch 2.13.

Hak Cipta dan Catatan Teknis

Copyright (c) 2026 Romi Nur Ismanto. Seluruh hak cipta dilindungi.

Buku ini disusun untuk pendidikan dan rekayasa perangkat lunak. Contoh kode dapat diadaptasi, diuji, dan dikembangkan untuk kebutuhan belajar maupun prototipe internal. Nama produk dan merek dagang tetap menjadi milik pemegangnya masing-masing.

PERINGATAN VERSI Runtime Colab, GPU, batas penggunaan, dan paket bawaan dapat berubah. Selalu jalankan sel diagnostik versi sebelum mengeksekusi notebook.

Konvensi

- Kode diawali label COLAB - CODE CELL.
- Perintah shell memakai awalan ! pada notebook.
- Output yang diharapkan ditampilkan dalam kotak hijau.
- Bagian FAILURE MODE menjelaskan kesalahan yang sengaja harus dihindari.

Atribusi sumber

Rujukan utama berasal dari dokumentasi resmi PyTorch, tutorial PyTorch, dokumentasi Google Colab, ONNX, ExecuTorch, Captum, dan Python. Daftar URL tersedia di lampiran.

Prakata

PyTorch menjadi bahasa kerja bagi banyak peneliti dan engineer karena memberi kendali langsung atas tensor, autograd, model, compiler, dan sistem terdistribusi. Namun, menguasai sintaks saja tidak cukup. Engineer yang baik perlu memahami bentuk data, state, numerik, kinerja, reproducibility, dan risiko operasional.

Buku ini dirancang sebagai handbook praktik. Setiap bab bergerak dari mental model menuju sel Colab yang dapat dijalankan, kemudian mengajak pembaca membaca output, memodifikasi eksperimen, memeriksa bottleneck, dan menutup dengan proyek mini. Dengan pola tersebut, pembaca tidak hanya tahu apa yang harus diketik, tetapi juga mengapa kode bekerja dan kapan pendekatan tertentu sebaiknya dipilih.

Urutan materi dimulai dari tensor dan pipeline data, lalu berkembang ke computer vision, NLP, mixed precision, compiler, distributed training, generative model, deployment, interpretability, robustness, serta capstone end-to-end.

PRINSIP BUKU Ukur sebelum mengoptimalkan. Validasi bentuk sebelum melatih. Simpan state sebelum runtime putus. Uji kontrak sebelum model dirilis.

Untuk Siapa Buku Ini?

Profil	Jalur yang disarankan
Pemula Python/ML	Ikuti Bagian I-IV secara berurutan dan jalankan semua latihan.
Data scientist	Fokus Bagian III-VII untuk pipeline, model, dan eksperimen.
AI research engineer	Gunakan Bagian VIII-XII sebagai referensi compiler, scale-out, dan riset.
ML platform engineer	Prioritaskan Bagian IX-XI serta appendix quality gate.
Dosen/instruktur	Gunakan 48 proyek mini sebagai modul praktikum satu semester atau bootcamp.

Prasyarat minimum

- Python dasar: fungsi, class, list/dict, exception, dan context manager.
- Aljabar linear dasar: vektor, matriks, dot product, dan turunan.
- Akun Google untuk menyimpan notebook Colab.
- Kemauan membaca shape, dtype, device, loss, dan metric secara eksplisit.

Cara Menggunakan Setiap Bab

Tahap	Tujuan
1. Peta konsep	Bangun model mental sebelum menyentuh kode.
2. Setup Colab	Verifikasi versi, runtime, dan device.
3. Notebook inti	Jalankan kode apa adanya dan simpan output baseline.
4. Bedah kode	Telusuri kontrak shape, state, dan gradien.
5. Eksperimen	Ubah tepat satu variabel dan bandingkan metrik.
6. Profiling	Ukur waktu atau memori sebelum membuat klaim.
7. Failure mode	Reproduksi kesalahan secara terkendali.
8. Mini project	Integrasikan konsep dalam artefak kecil yang dapat dinilai.
9. Latihan	Jawab tanpa melihat ringkasan.
10. Cheat sheet	Gunakan sebagai referensi saat coding.
RITME BELAJAR Satu bab idealnya dikerjakan dalam 60-120 menit. Bab distributed dan deployment membutuhkan waktu lebih panjang.	

Legenda Google Colab

COLAB - SEL 0

```
1 # Sel diagnostik wajib
2 import torch
3 print(torch.__version__)
4 print(torch.cuda.is_available())
5 print(torch.cuda.get_device_name(0) if torch.cuda.is_available() else 'CPU')
```

OUTPUT 2.11.0 | True | NVIDIA GPU (tipe bergantung ketersediaan)

Elemen	Makna
!pip install ...	Memasang paket ke runtime aktif.
Runtime > Change runtime type	Memilih CPU, GPU, atau TPU yang tersedia.
/content	Penyimpanan sementara runtime.
/content/drive	Lokasi Drive setelah mount.
Run all	Menjalankan notebook dari atas; gunakan setelah dependensi dan path benar.
Restart session	Diperlukan setelah perubahan paket inti tertentu.
CATATAN Akses GPU tidak menjamin tensor atau model otomatis memakai GPU. Perpindahan device tetap harus eksplisit.	

Peta Belajar 12 Bagian

MENTAL MODEL 48

Capstone: Riset ke Produksi



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Bagian	Fokus
01	Orientasi Dan Fondasi Colab
02	Tensor Dan Autograd
03	Pipeline Data
04	Jaringan Saraf Dan Training
05	Computer Vision
06	Nlp Dan Transformer
07	Training Lanjutan
08	Kompilasi Dan Kinerja
09	Distributed Dan Scale-Out
10	Generative Dan Multimodal
11	Deployment Dan Mlops
12	Riset, Keamanan, Dan Capstone

Learning Path: Foundation ke Production

Level	Capaian
FOUNDATION	Bab 1-16: Colab, tensor, data, model, dan training loop.
APPLIED	Bab 17-24: computer vision, sequence, attention, transformer.
ACCELERATE	Bab 25-32: AMP, checkpoint, tuning, compiler, profiler, kompresi.
SCALE	Bab 33-36: DDP, FSDP, sharding, fault tolerance.
CREATE	Bab 37-40: VAE, GAN, diffusion, multimodal.
OPERATE	Bab 41-48: export, edge, serving, monitoring, interpretability, safety, capstone.

Aturan kelulusan mandiri

- Setiap proyek mini menghasilkan notebook yang dapat Run all.
- Semua eksperimen menyimpan seed, config, metric, dan versi paket.
- Semua deployment memiliki shape contract, smoke test, dan fallback.
- Semua klaim kinerja disertai warm-up, sinkronisasi, dan jumlah pengulangan.

Daftar Isi - Bagian I-VI

Bab	Judul	Hal.
01	Memulai PyTorch di Google Colab	13
02	Reproducibility dan Manajemen Lingkungan	25
03	Peta Ekosistem PyTorch	37
04	Klasifikasi Pertama End-to-End	49
05	Tensor: Bentuk, Dtype, dan Device	61
06	Indexing, Broadcasting, dan Einsum	73
07	Autograd dan Computational Graph	85
08	Custom Autograd dan Gradient Check	97
09	Dataset dan DataLoader	109
10	Transforms dan Augmentasi	121
11	Custom Dataset untuk Data Nyata	133
12	Optimasi Input Pipeline	145
13	Mendesain nn.Module	157
14	Loss Function dan Optimizer	169
15	Training Loop yang Benar	181
16	Regularisasi, Inisialisasi, dan Normalisasi	193
17	Convolutional Neural Network	205
18	Transfer Learning	217
19	Semantic Segmentation	229
20	Detection dan Vision Transformer	241
21	Representasi Teks dan Embedding	253
22	RNN, LSTM, dan GRU	265
23	Attention dan Multi-Head Attention	277
24	Transformer Encoder	289

Daftar Isi - Bagian VII-XII

Bab	Judul	Hal.
25	Automatic Mixed Precision	301
26	Scheduler, Gradient Accumulation, dan Clipping	313
27	Checkpoint, Resume, dan Early Stopping	325
28	Hyperparameter Tuning dan Experiment Tracking	337
29	torch.compile dan Graph Capture	349
30	Profiler dan Analisis Bottleneck	361
31	Efisiensi Memori	373
32	Quantization dan Pruning	385
33	DistributedDataParallel	397
34	Fully Sharded Data Parallel	409
35	Sharding Data dan Tensor	421
36	Multi-Node dan Fault Tolerance	433
37	Autoencoder dan Variational Autoencoder	445
38	Generative Adversarial Network	457
39	Diffusion Model	469
40	Multimodal dan Contrastive Learning	481
41	torch.export dan ONNX	493
42	Edge AI dengan ExecuTorch	505
43	Serving Model sebagai API	517
44	Monitoring, Testing, dan CI/CD	529
45	Interpretability dengan Captum	541
46	Robustness, Adversarial Testing, dan Fairness	553
47	Custom Operator dan Ekstensi	565
48	Capstone: Riset ke Produksi	577

Checklist Sebelum Mulai

- Buat salinan notebook sebelum mengubah eksperimen utama.
- Jalankan sel versi dan device.
- Simpan data besar di Drive atau object storage, bukan hanya /content.
- Tulis objective metric sebelum training.
- Pisahkan train, validation, dan test sejak awal.
- Mulai dari baseline kecil yang dapat selesai cepat.
- Aktifkan GPU hanya ketika workload memang menggunakannya.
- Simpan checkpoint dan config secara berkala.
- Jangan menyalin kode yang tidak dipahami ke runtime dengan kredensial aktif.
- Catat hasil eksperimen dalam tabel yang konsisten.

SIAP Jika seluruh item dapat dijelaskan dengan kata-kata Anda sendiri, lanjutkan ke Bab 1.

BAGIAN 1 - ORIENTASI DAN FONDASI COLAB

Bab 01. Memulai PyTorch di Google Colab

Membangun notebook pertama yang dapat diulang.

Hasil belajar

- Menjelaskan peran runtime dan hubungannya dengan GPU.
- Menulis notebook Colab untuk membangun notebook pertama yang dapat diulang.
- Mendiagnosis risiko: GPU dipilih tetapi tensor masih berada di CPU.
- Menghasilkan proyek mini: Notebook diagnostik perangkat.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: runtime, GPU, versi paket, device.

Parameter	Target
Level	Dasar
Waktu	60-120 menit
Artefak	Notebook diagnostik perangkat
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 01

Memulai PyTorch di Google Colab



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah membangun notebook pertama yang dapat diulang. Alur di atas memperlakukan runtime sebagai input keputusan, GPU sebagai transformasi utama, versi paket sebagai state atau mekanisme kontrol, dan device sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika GPU berubah, bagaimana dampaknya terhadap device? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk membangun notebook pertama yang dapat diulang. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch, platform
2 print('Python:', platform.python_version())
3 print('PyTorch:', torch.__version__)
4 print('CUDA available:', torch.cuda.is_available())
5 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
6 x = torch.randn(4, 4, device=device)
7 print(device, x.mean().item())
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi device dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
Python	Mewakili runtime; catat input, output, dtype, device, serta state yang berubah.
PyTorch	Mewakili GPU; catat input, output, dtype, device, serta state yang berubah.
torch.__version__	Mewakili versi paket; catat input, output, dtype, device, serta state yang berubah.
CUDA	Mewakili device; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Gpu dipilih tetapi tensor masih berada di cpu.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 01

Memulai PyTorch di Google Colab

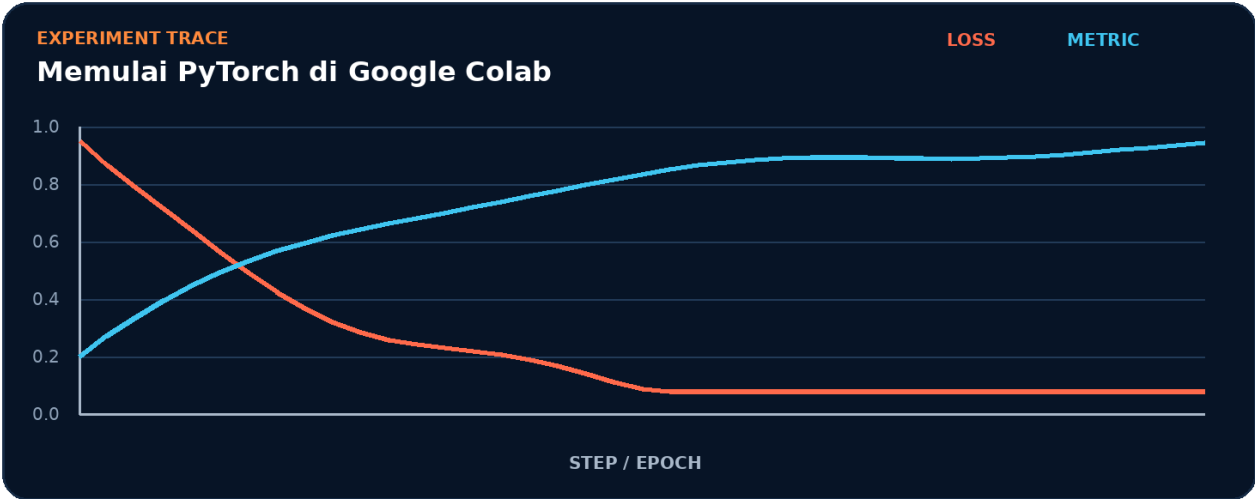


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk runtime.
2. Terapkan transformasi utama: GPU.
3. Kelola state atau kontrol melalui versi paket.
4. Ukur hasil akhir memakai device.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur runtime -> GPU -> versi paket -> device tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk device.
- 2. Ubah satu nilai yang memengaruhi GPU.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada GPU akan memengaruhi device.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	device
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah runtime menjadi bottleneck.
- Pisahkan biaya setup dari steady-state GPU.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Gpu dipilih tetapi tensor masih berada di cpu.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait runtime.
3. Isolasi	Nonaktifkan sementara GPU dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```

1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()

```

Mini Project: Notebook diagnostik perangkat

Bangun artefak kecil yang membuktikan Anda dapat membangun notebook pertama yang dapat diulang. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk device.
2. Implementasikan baseline memakai runtime dan GPU.
3. Tambahkan logging untuk versi paket.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan runtime tanpa menggunakan istilah GPU.
2. Apa shape, dtype, dan device yang diharapkan pada batas GPU?
3. Kapan versi paket berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas device?
5. Bagaimana Anda mereproduksi kegagalan: GPU dipilih tetapi tensor masih berada di CPU?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Notebook diagnostik perangkat?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk GPU, lalu buat tabel yang membandingkan device, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
runtime	Peran inti dalam membangun notebook pertama yang dapat diulang.
GPU	Peran inti dalam membangun notebook pertama yang dapat diulang.
versi paket	Peran inti dalam membangun notebook pertama yang dapat diulang.
device	Peran inti dalam membangun notebook pertama yang dapat diulang.

Checklist siap pakai

- Verifikasi runtime sebelum eksekusi utama.
- Catat perubahan pada GPU dan versi paket.
- Ukur device dengan baseline yang adil.
- Tambahkan test untuk mencegah: GPU dipilih tetapi tensor masih berada di CPU.
- Simpan artefak proyek: Notebook diagnostik perangkat.

API yang muncul

Python, PyTorch, torch.__version__, CUDA, torch.cuda.is_available, torch.device

SATU KALIMAT Bab 1 mengajarkan cara membangun notebook pertama yang dapat diulang dengan kontrak eksplisit dan bukti terukur.

BAGIAN 1 - ORIENTASI DAN FONDASI COLAB

Bab 02. Reproducibility dan Manajemen Lingkungan

Mengunci seed, versi, dan konfigurasi eksperimen.

Hasil belajar

- Menjelaskan peran seed dan hubungannya dengan determinisme.
- Menulis notebook Colab untuk mengunci seed, versi, dan konfigurasi eksperimen.
- Mendiagnosis risiko: hasil berubah karena sumber random belum dikunci.
- Menghasilkan proyek mini: Template eksperimen reproduktif.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: seed, determinisme, requirements, metadata.

Parameter	Target
Level	Dasar
Waktu	60-120 menit
Artefak	Template eksperimen reproduktif
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 02

Reproducibility dan Manajemen Lingkungan



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah mengunci seed, versi, dan konfigurasi eksperimen. Alur di atas memperlakukan seed sebagai input keputusan, determinisme sebagai transformasi utama, requirements sebagai state atau mekanisme kontrol, dan metadata sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika determinisme berubah, bagaimana dampaknya terhadap metadata? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk mengunci seed, versi, dan konfigurasi eksperimen. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import random, numpy as np, torch
2 SEED = 42
3 random.seed(SEED); np.random.seed(SEED)
4 torch.manual_seed(SEED)
5 if torch.cuda.is_available(): torch.cuda.manual_seed_all(SEED)
6 torch.backends.cudnn.benchmark = False
7 torch.use_deterministic_algorithms(True, warn_only=True)
8 print(torch.randn(3))
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi metadata dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
SEED	Mewakili seed; catat input, output, dtype, device, serta state yang berubah.
torch.manual_seed	Mewakili determinisme; catat input, output, dtype, device, serta state yang berubah.
torch.cuda.is_available	Mewakili requirements; catat input, output, dtype, device, serta state yang berubah.
torch.cuda.manual_seed_all	Mewakili metadata; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Hasil berubah karena sumber random belum dikunci.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 02

Reproducibility dan Manajemen Lingkungan

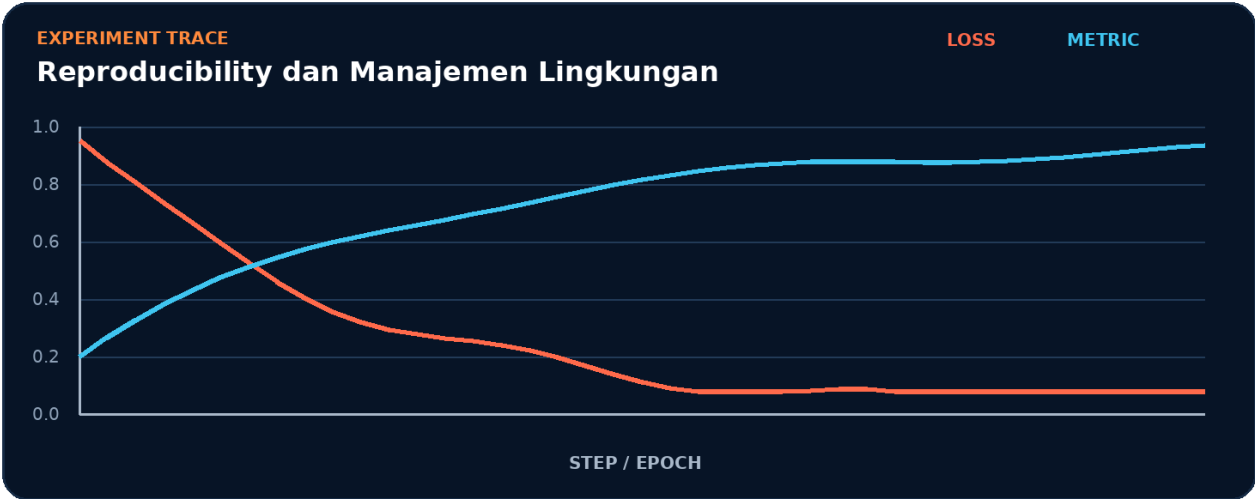


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk seed.
2. Terapkan transformasi utama: determinisme.
3. Kelola state atau kontrol melalui requirements.
4. Ukur hasil akhir memakai metadata.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur seed -> determinisme -> requirements -> metadata tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk metadata.
- 2. Ubah satu nilai yang memengaruhi determinisme.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada determinisme akan memengaruhi metadata.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	metadata
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah seed menjadi bottleneck.
- Pisahkan biaya setup dari steady-state determinisme.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Hasil berubah karena sumber random belum dikunci.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait seed.
3. Isolasi	Nonaktifkan sementara determinisme dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```

1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()

```

Mini Project: Template eksperimen reproduktif

Bangun artefak kecil yang membuktikan Anda dapat mengunci seed, versi, dan konfigurasi eksperimen. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk metadata.
2. Implementasikan baseline memakai seed dan determinisme.
3. Tambahkan logging untuk requirements.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan seed tanpa menggunakan istilah determinisme.
2. Apa shape, dtype, dan device yang diharapkan pada batas determinisme?
3. Kapan requirements berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas metadata?
5. Bagaimana Anda mereproduksi kegagalan: hasil berubah karena sumber random belum dikunci?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Template eksperimen reproduktif?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk determinisme, lalu buat tabel yang membandingkan metadata, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
seed	Peran inti dalam mengunci seed, versi, dan konfigurasi eksperimen.
determinisme	Peran inti dalam mengunci seed, versi, dan konfigurasi eksperimen.
requirements	Peran inti dalam mengunci seed, versi, dan konfigurasi eksperimen.
metadata	Peran inti dalam mengunci seed, versi, dan konfigurasi eksperimen.

Checklist siap pakai

- Verifikasi seed sebelum eksekusi utama.
- Catat perubahan pada determinisme dan requirements.
- Ukur metadata dengan baseline yang adil.
- Tambahkan test untuk mencegah: hasil berubah karena sumber random belum dikunci.
- Simpan artefak proyek: Template eksperimen reproduktif.

API yang muncul

SEED, torch.manual_seed, torch.cuda.is_available, torch.cuda.manual_seed_all, torch.backends.cudnn.benchmark, False

SATU KALIMAT Bab 2 mengajarkan cara mengunci seed, versi, dan konfigurasi eksperimen dengan kontrak eksplisit dan bukti terukur.

BAGIAN 1 - ORIENTASI DAN FONDASI COLAB

Bab 03. Peta Ekosistem PyTorch

Memahami hubungan torch, domain library, compiler, dan deployment.

Hasil belajar

- Menjelaskan peran torch dan hubungannya dengan torchvision.
- Menulis notebook Colab untuk memahami hubungan torch, domain library, compiler, dan deployment.
- Mendiagnosis risiko: memasang paket domain yang tidak kompatibel.
- Menghasilkan proyek mini: Peta dependensi proyek.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: torch, torchvision, torchaudio, torchdata.

Parameter	Target
Level	Dasar
Waktu	60-120 menit
Artefak	Peta dependensi proyek
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 03

Peta Ekosistem PyTorch



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah memahami hubungan torch, domain library, compiler, dan deployment. Alur di atas memperlakukan torch sebagai input keputusan, torchvision sebagai transformasi utama, torchaudio sebagai state atau mekanisme kontrol, dan torchdata sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika torchvision berubah, bagaimana dampaknya terhadap torchdata? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk memahami hubungan torch, domain library, compiler, dan deployment. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch, torchvision
2 print(torch.__version__, torchvision.__version__)
3 packages = {
4     'core': ['torch', 'torch.nn', 'torch.optim'],
5     'vision': ['torchvision.datasets', 'torchvision.models'],
6     'scale': ['torch.distributed', 'torch.compile'],
7 }
8 for group, names in packages.items():
9     print(group, '->', ', '.join(names))
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi torchdata dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.__version__	Mewakili torch; catat input, output, dtype, device, serta state yang berubah.
torch.nn	Mewakili torchvision; catat input, output, dtype, device, serta state yang berubah.
torch.optim	Mewakili torchaudio; catat input, output, dtype, device, serta state yang berubah.
torch.distributed	Mewakili torchdata; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Memasang paket domain yang tidak kompatibel.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 03

Peta Ekosistem PyTorch

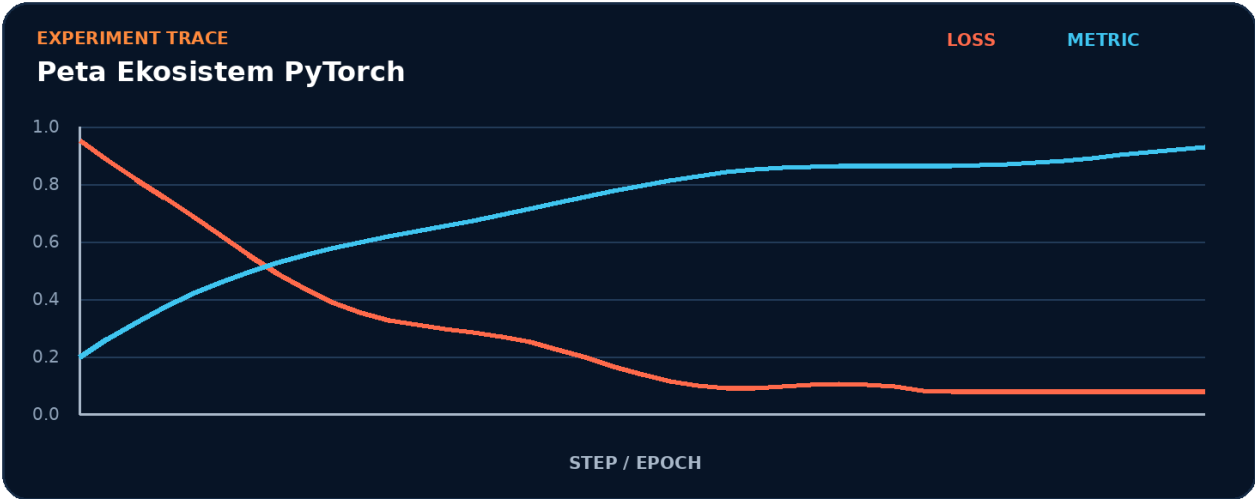


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk torch.
2. Terapkan transformasi utama: torchvision.
3. Kelola state atau kontrol melalui torchaudio.
4. Ukur hasil akhir memakai torchdata.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur torch -> torchvision -> torchaudio -> torchdata tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk torchdata.
- 2. Ubah satu nilai yang memengaruhi torchvision.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada torchvision akan memengaruhi torchdata.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	torchdata
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah torch menjadi bottleneck.
- Pisahkan biaya setup dari steady-state torchvision.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Memasang paket domain yang tidak kompatibel.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait torch.
3. Isolasi	Nonaktifkan sementara torchvision dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Peta dependensi proyek

Bangun artefak kecil yang membuktikan Anda dapat memahami hubungan torch, domain library, compiler, dan deployment. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk torchdata.
2. Implementasikan baseline memakai torch dan torchvision.
3. Tambahkan logging untuk torchaudio.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan torch tanpa menggunakan istilah torchvision.
2. Apa shape, dtype, dan device yang diharapkan pada batas torchvision?
3. Kapan torchaudio berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas torchdata?
5. Bagaimana Anda mereproduksi kegagalan: memasang paket domain yang tidak kompatibel?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Peta dependensi proyek?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk torchvision, lalu buat tabel yang membandingkan torchdata, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
torch	Peran inti dalam memahami hubungan torch, domain library, compiler, dan deployment.
torchvision	Peran inti dalam memahami hubungan torch, domain library, compiler, dan deployment.
torchaudio	Peran inti dalam memahami hubungan torch, domain library, compiler, dan deployment.
torchdata	Peran inti dalam memahami hubungan torch, domain library, compiler, dan deployment.

Checklist siap pakai

- Verifikasi torch sebelum eksekusi utama.
- Catat perubahan pada torchvision dan torchaudio.
- Ukur torchdata dengan baseline yang adil.
- Tambahkan test untuk mencegah: memasang paket domain yang tidak kompatibel.
- Simpan artefak proyek: Peta dependensi proyek.

API yang muncul

torch.__version__, torch.nn, torch.optim, torch.distributed, torch.compile

SATU KALIMAT Bab 3 mengajarkan cara memahami hubungan torch, domain library, compiler, dan deployment dengan kontrak eksplisit dan bukti terukur.

BAGIAN 1 - ORIENTASI DAN FONDASI COLAB

Bab 04. Klasifikasi Pertama End-to-End

Menyatukan data, model, loss, optimizer, dan evaluasi.

Hasil belajar

- Menjelaskan peran dataset dan hubungannya dengan model.
- Menulis notebook Colab untuk menyatukan data, model, loss, optimizer, dan evaluasi.
- Mendiagnosis risiko: lupa memanggil `model.eval` saat validasi.
- Menghasilkan proyek mini: Klasifier FashionMNIST.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: dataset, model, loss, optimizer.

Parameter	Target
Level	Dasar
Waktu	60-120 menit
Artefak	Klasifier FashionMNIST
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 04

Klasifikasi Pertama End-to-End



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah menyatukan data, model, loss, optimizer, dan evaluasi. Alur di atas memperlakukan dataset sebagai input keputusan, model sebagai transformasi utama, loss sebagai state atau mekanisme kontrol, dan optimizer sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika model berubah, bagaimana dampaknya terhadap optimizer? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk menyatukan data, model, loss, optimizer, dan evaluasi. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch import nn
3 from torchvision.datasets import FashionMNIST
4 from torchvision.transforms import ToTensor
5 from torch.utils.data import DataLoader
6 train = FashionMNIST('.', train=True, download=True, transform=ToTensor())
7 loader = DataLoader(train, batch_size=128, shuffle=True)
8 model = nn.Sequential(nn.Flatten(), nn.Linear(784, 128), nn.ReLU(), nn.Linear(128, 10))
9 opt = torch.optim.AdamW(model.parameters(), lr=3e-4)
10 x, y = next(iter(loader)); loss = nn.CrossEntropyLoss()(model(x), y)
11 opt.zero_grad(); loss.backward(); opt.step(); print(loss.item())
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi optimizer dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
FashionMNIST	Mewakili dataset; catat input, output, dtype, device, serta state yang berubah.
ToTensor	Mewakili model; catat input, output, dtype, device, serta state yang berubah.
torch.utils.data	Mewakili loss; catat input, output, dtype, device, serta state yang berubah.
DataLoader	Mewakili optimizer; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Lupa memanggil model.eval saat validasi.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 04

Klasifikasi Pertama End-to-End

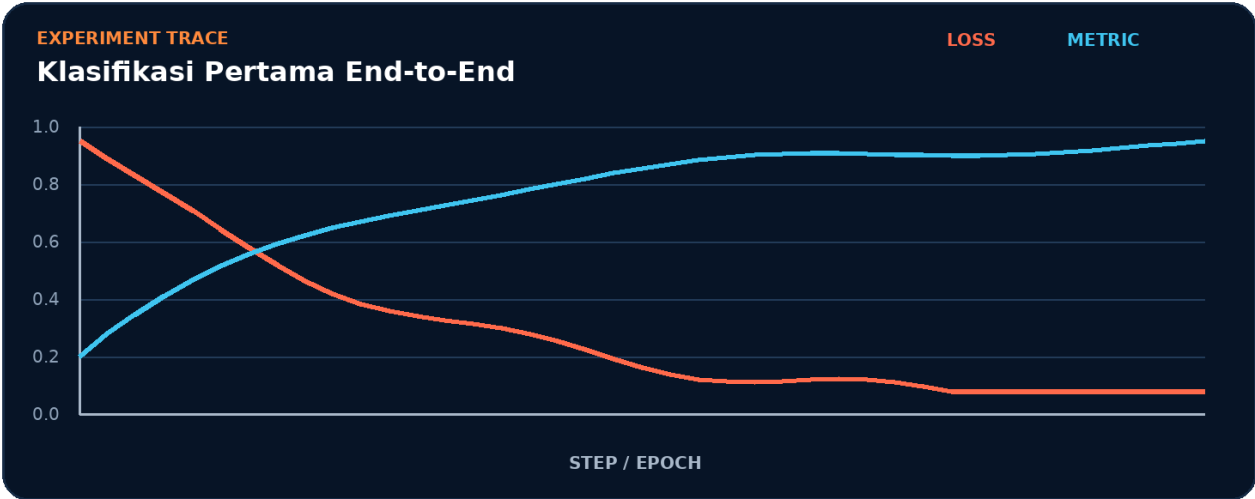


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk dataset.
2. Terapkan transformasi utama: model.
3. Kelola state atau kontrol melalui loss.
4. Ukur hasil akhir memakai optimizer.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur dataset -> model -> loss -> optimizer tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk optimizer.
- 2. Ubah satu nilai yang memengaruhi model.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada model akan memengaruhi optimizer.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	optimizer
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah dataset menjadi bottleneck.
- Pisahkan biaya setup dari steady-state model.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Lupa memanggil `model.eval` saat validasi.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait dataset.
3. Isolasi	Nonaktifkan sementara model dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Klasifier FashionMNIST

Bangun artefak kecil yang membuktikan Anda dapat menyatukan data, model, loss, optimizer, dan evaluasi. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk optimizer.
2. Implementasikan baseline memakai dataset dan model.
3. Tambahkan logging untuk loss.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan dataset tanpa menggunakan istilah model.
2. Apa shape, dtype, dan device yang diharapkan pada batas model?
3. Kapan loss berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas optimizer?
5. Bagaimana Anda mereproduksi kegagalan: lupa memanggil model.eval saat validasi?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Klasifier FashionMNIST?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk model, lalu buat tabel yang membandingkan optimizer, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
dataset	Peran inti dalam menyatukan data, model, loss, optimizer, dan evaluasi.
model	Peran inti dalam menyatukan data, model, loss, optimizer, dan evaluasi.
loss	Peran inti dalam menyatukan data, model, loss, optimizer, dan evaluasi.
optimizer	Peran inti dalam menyatukan data, model, loss, optimizer, dan evaluasi.

Checklist siap pakai

- Verifikasi dataset sebelum eksekusi utama.
- Catat perubahan pada model dan loss.
- Ukur optimizer dengan baseline yang adil.
- Tambahkan test untuk mencegah: lupa memanggil `model.eval` saat validasi.
- Simpan artefak proyek: Klasifier FashionMNIST.

API yang muncul

FashionMNIST, ToTensor, torch.utils.data, DataLoader, True, nn.Sequential

SATU KALIMAT Bab 4 mengajarkan cara menyatukan data, model, loss, optimizer, dan evaluasi dengan kontrak eksplisit dan bukti terukur.

BAGIAN 2 - TENSOR DAN AUTOGRAD

Bab 05. Tensor: Bentuk, Dtype, dan Device

Menguasai objek data inti pytorch.

Hasil belajar

- Menjelaskan peran shape dan hubungannya dengan dtype.
- Menulis notebook Colab untuk menguasai objek data inti PyTorch.
- Mendiagnosis risiko: operasi gagal karena dtype atau device berbeda.
- Menghasilkan proyek mini: Inspektor tensor interaktif.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: shape, dtype, device, stride.

Parameter	Target
Level	Dasar
Waktu	60-120 menit
Artefak	Inspektor tensor interaktif
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 05

Tensor: Bentuk, Dtype, dan Device



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah menguasai objek data inti PyTorch. Alur di atas memperlakukan shape sebagai input keputusan, dtype sebagai transformasi utama, device sebagai state atau mekanisme kontrol, dan stride sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika dtype berubah, bagaimana dampaknya terhadap stride? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk menguasai objek data inti PyTorch. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 x = torch.arange(12, dtype=torch.float32).reshape(3, 4)
3 print('shape:', x.shape, 'stride:', x.stride())
4 print('dtype:', x.dtype, 'device:', x.device)
5 y = x.to(dtype=torch.float16)
6 z = x.contiguous().view(2, 6)
7 print(y.dtype, z.shape)
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi stride dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.arange	Mewakili shape; catat input, output, dtype, device, serta state yang berubah.
torch.float32	Mewakili dtype; catat input, output, dtype, device, serta state yang berubah.
torch.float16	Mewakili device; catat input, output, dtype, device, serta state yang berubah.
torch.arange	Mewakili stride; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Operasi gagal karena dtype atau device berbeda.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 05

Tensor: Bentuk, Dtype, dan Device

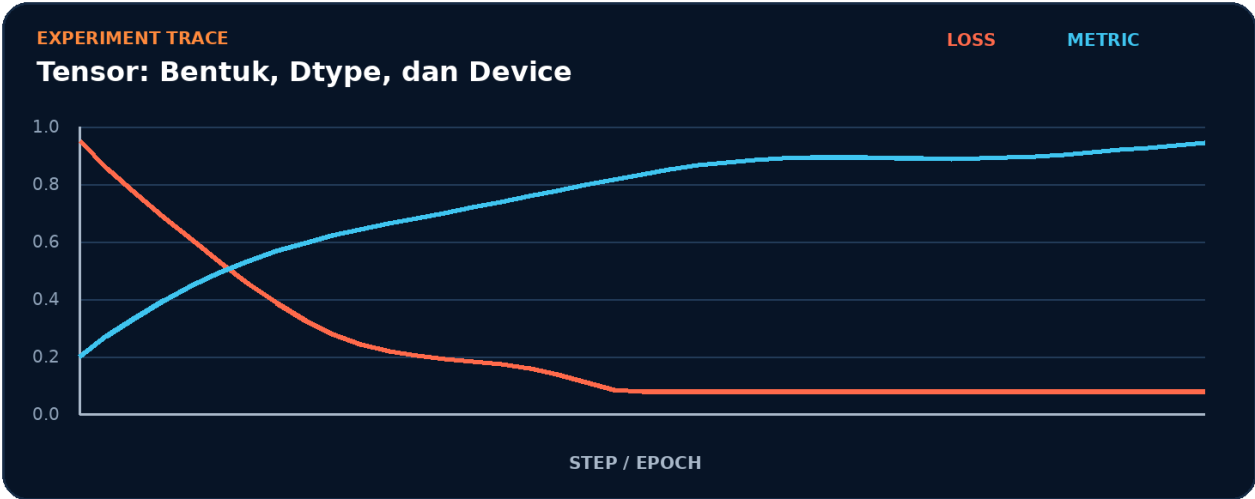


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk shape.
2. Terapkan transformasi utama: dtype.
3. Kelola state atau kontrol melalui device.
4. Ukur hasil akhir memakai stride.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur shape -> dtype -> device -> stride tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk stride.
- 2. Ubah satu nilai yang memengaruhi dtype.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada dtype akan memengaruhi stride.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	stride
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah shape menjadi bottleneck.
- Pisahkan biaya setup dari steady-state dtype.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Operasi gagal karena dtype atau device berbeda.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait shape.
3. Isolasi	Nonaktifkan sementara dtype dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Inspektor tensor interaktif

Bangun artefak kecil yang membuktikan Anda dapat menguasai objek data inti PyTorch. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk stride.
2. Implementasikan baseline memakai shape dan dtype.
3. Tambahkan logging untuk device.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan shape tanpa menggunakan istilah dtype.
2. Apa shape, dtype, dan device yang diharapkan pada batas dtype?
3. Kapan device berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas stride?
5. Bagaimana Anda mereproduksi kegagalan: operasi gagal karena dtype atau device berbeda?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Inspektur tensor interaktif?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk dtype, lalu buat tabel yang membandingkan stride, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
shape	Peran inti dalam menguasai objek data inti PyTorch.
dtype	Peran inti dalam menguasai objek data inti PyTorch.
device	Peran inti dalam menguasai objek data inti PyTorch.
stride	Peran inti dalam menguasai objek data inti PyTorch.

Checklist siap pakai

- Verifikasi shape sebelum eksekusi utama.
- Catat perubahan pada dtype dan device.
- Ukur stride dengan baseline yang adil.
- Tambahkan test untuk mencegah: operasi gagal karena dtype atau device berbeda.
- Simpan artefak proyek: Inspektor tensor interaktif.

API yang muncul

torch.arange, torch.float32, torch.float16

SATU KALIMAT Bab 5 mengajarkan cara menguasai objek data inti PyTorch dengan kontrak eksplisit dan bukti terukur.

BAGIAN 2 - TENSOR DAN AUTOGRAD

Bab 06. Indexing, Broadcasting, dan Einsum

Menulis operasi tensor ringkas tanpa loop python.

Hasil belajar

- Menjelaskan peran indexing dan hubungannya dengan broadcasting.
- Menulis notebook Colab untuk menulis operasi tensor ringkas tanpa loop Python.
- Mendiagnosis risiko: broadcasting menghasilkan bentuk benar tetapi makna salah.
- Menghasilkan proyek mini: Mesin similarity matriks.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: indexing, broadcasting, einsum, vectorization.

Parameter	Target
Level	Dasar
Waktu	60-120 menit
Artefak	Mesin similarity matriks
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 06

Indexing, Broadcasting, dan Einsum



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah menulis operasi tensor ringkas tanpa loop Python. Alur di atas memperlakukan indexing sebagai input keputusan, broadcasting sebagai transformasi utama, einsum sebagai state atau mekanisme kontrol, dan vectorization sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika broadcasting berubah, bagaimana dampaknya terhadap vectorization? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk menulis operasi tensor ringkas tanpa loop Python. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 a = torch.randn(32, 128)
3 b = torch.randn(64, 128)
4 sim = torch.einsum('id,jd->ij', a, b)
5 a_norm = a / a.norm(dim=1, keepdim=True)
6 b_norm = b / b.norm(dim=1, keepdim=True)
7 cosine = a_norm @ b_norm.T
8 print(sim.shape, cosine.min().item(), cosine.max().item())
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi vectorization dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.randn	Mewakili indexing; catat input, output, dtype, device, serta state yang berubah.
torch.einsum	Mewakili broadcasting; catat input, output, dtype, device, serta state yang berubah.
True	Mewakili einsum; catat input, output, dtype, device, serta state yang berubah.
torch.randn	Mewakili vectorization; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Broadcasting menghasilkan bentuk benar tetapi makna salah.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 06

Indexing, Broadcasting, dan Einsum

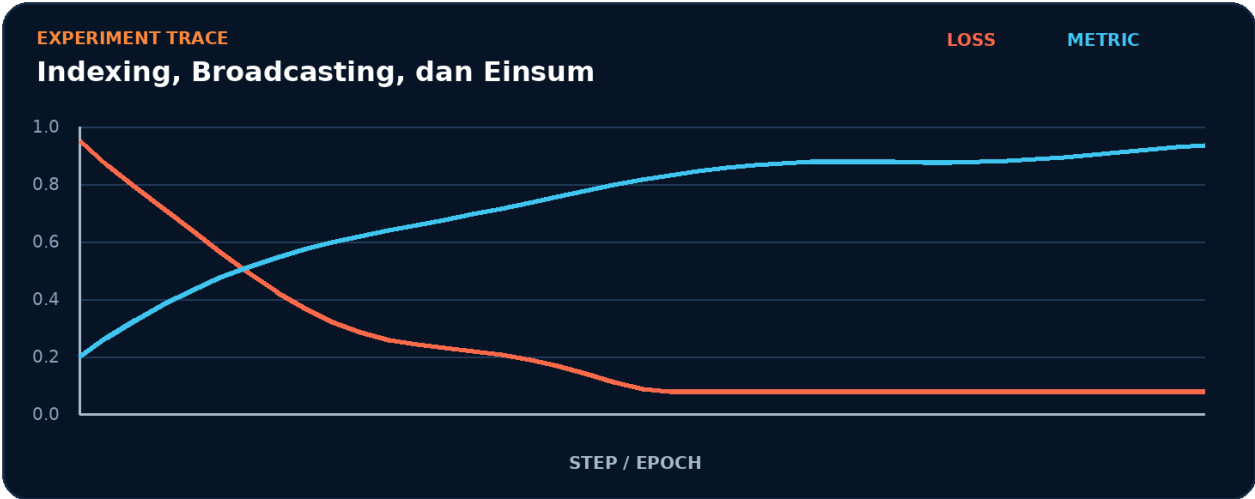


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk indexing.
2. Terapkan transformasi utama: broadcasting.
3. Kelola state atau kontrol melalui einsum.
4. Ukur hasil akhir memakai vectorization.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur indexing -> broadcasting -> einsum -> vectorization tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk vectorization.
- 2. Ubah satu nilai yang memengaruhi broadcasting.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada broadcasting akan memengaruhi vectorization.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	vectorization
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah indexing menjadi bottleneck.
- Pisahkan biaya setup dari steady-state broadcasting.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Broadcasting menghasilkan bentuk benar tetapi makna salah.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait indexing.
3. Isolasi	Nonaktifkan sementara broadcasting dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```

1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()

```

Mini Project: Mesin similarity matriks

Bangun artefak kecil yang membuktikan Anda dapat menulis operasi tensor ringkas tanpa loop Python. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk vectorization.
2. Implementasikan baseline memakai indexing dan broadcasting.
3. Tambahkan logging untuk einsum.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan indexing tanpa menggunakan istilah broadcasting.
2. Apa shape, dtype, dan device yang diharapkan pada batas broadcasting?
3. Kapan einsum berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas vectorization?
5. Bagaimana Anda mereproduksi kegagalan: broadcasting menghasilkan bentuk benar tetapi makna salah?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Mesin similarity matriks?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk broadcasting, lalu buat tabel yang membandingkan vectorization, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
indexing	Peran inti dalam menulis operasi tensor ringkas tanpa loop Python.
broadcasting	Peran inti dalam menulis operasi tensor ringkas tanpa loop Python.
einsum	Peran inti dalam menulis operasi tensor ringkas tanpa loop Python.
vectorization	Peran inti dalam menulis operasi tensor ringkas tanpa loop Python.

Checklist siap pakai

- Verifikasi indexing sebelum eksekusi utama.
- Catat perubahan pada broadcasting dan einsum.
- Ukur vectorization dengan baseline yang adil.
- Tambahkan test untuk mencegah: broadcasting menghasilkan bentuk benar tetapi makna salah.
- Simpan artefak proyek: Mesin similarity matriks.

API yang muncul

torch.randn, torch.einsum, True

SATU KALIMAT Bab 6 mengajarkan cara menulis operasi tensor ringkas tanpa loop Python dengan kontrak eksplisit dan bukti terukur.

BAGIAN 2 - TENSOR DAN AUTOGRAD

Bab 07. Autograd dan Computational Graph

Memahami gradien otomatis dari forward ke backward.

Hasil belajar

- Menjelaskan peran requires_grad dan hubungannya dengan backward.
- Menulis notebook Colab untuk memahami gradien otomatis dari forward ke backward.
- Mendiagnosis risiko: graf komputasi diputus oleh detach atau konversi NumPy.
- Menghasilkan proyek mini: Visualisasi aliran gradien.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: requires_grad, backward, grad_fn, detach.

Parameter	Target
Level	Dasar
Waktu	60-120 menit
Artefak	Visualisasi aliran gradien
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 07

Autograd dan Computational Graph



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah memahami gradien otomatis dari forward ke backward. Alur di atas memperlakukan `requires_grad` sebagai input keputusan, `backward` sebagai transformasi utama, `grad_fn` sebagai state atau mekanisme kontrol, dan `detach` sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika `backward` berubah, bagaimana dampaknya terhadap `detach`? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk memahami gradien otomatis dari forward ke backward. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 x = torch.tensor(2.0, requires_grad=True)
3 y = x**3 + 2*x**2 - x
4 y.backward()
5 print('y =', y.item(), 'dy/dx =', x.grad.item())
6 x.grad.zero_()
7 with torch.no_grad():
8     probe = x * 10
9 print(probe.requires_grad)
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi detach dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.tensor	Mewakili requires_grad; catat input, output, dtype, device, serta state yang berubah.
True	Mewakili backward; catat input, output, dtype, device, serta state yang berubah.
torch.no_grad	Mewakili grad_fn; catat input, output, dtype, device, serta state yang berubah.
torch.tensor	Mewakili detach; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Graf komputasi diputus oleh detach atau konversi numpy.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 07

Autograd dan Computational Graph

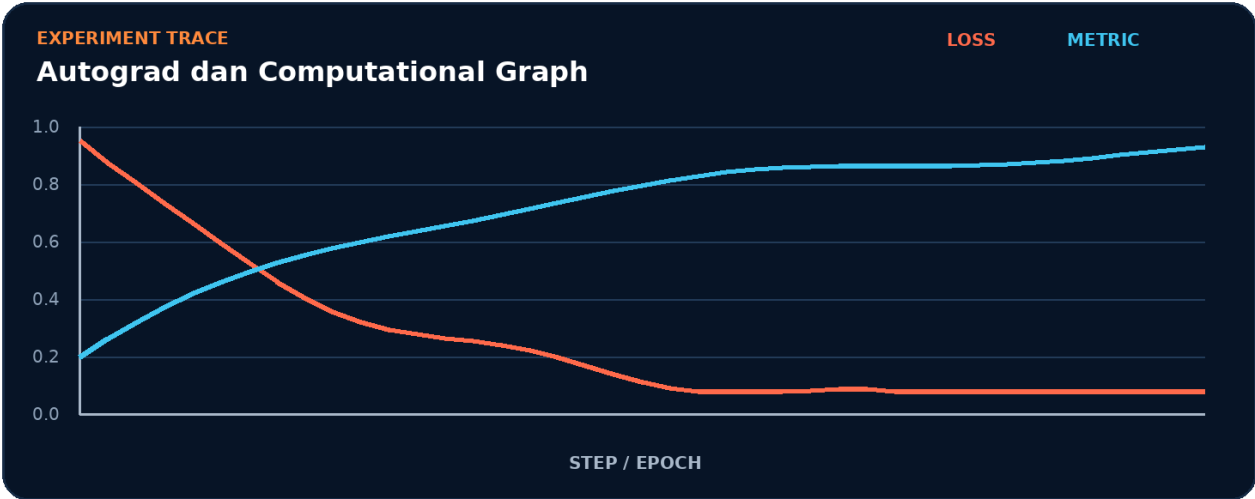


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk `requires_grad`.
2. Terapkan transformasi utama: `backward`.
3. Kelola state atau kontrol melalui `grad_fn`.
4. Ukur hasil akhir memakai `detach`.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur `requires_grad` -> `backward` -> `grad_fn` -> `detach` tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk detach.
- 2. Ubah satu nilai yang memengaruhi backward.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada backward akan memengaruhi detach.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	detach
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah `requires_grad` menjadi bottleneck.
- Pisahkan biaya setup dari steady-state backward.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Graf komputasi diputus oleh detach atau konversi numpy.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait requires_grad.
3. Isolasi	Nonaktifkan sementara backward dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```

1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()

```

Mini Project: Visualisasi aliran gradien

Bangun artefak kecil yang membuktikan Anda dapat memahami gradien otomatis dari forward ke backward. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk detach.
2. Implementasikan baseline memakai `requires_grad` dan `backward`.
3. Tambahkan logging untuk `grad_fn`.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan `requires_grad` tanpa menggunakan istilah `backward`.
2. Apa `shape`, `dtype`, dan `device` yang diharapkan pada batas `backward`?
3. Kapan `grad_fn` berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas `detach`?
5. Bagaimana Anda mereproduksi kegagalan: graf komputasi diputus oleh `detach` atau konversi NumPy?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Visualisasi aliran gradien?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk `backward`, lalu buat tabel yang membandingkan `detach`, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
<code>requires_grad</code>	Peran inti dalam memahami gradien otomatis dari forward ke backward.
<code>backward</code>	Peran inti dalam memahami gradien otomatis dari forward ke backward.
<code>grad_fn</code>	Peran inti dalam memahami gradien otomatis dari forward ke backward.
<code>detach</code>	Peran inti dalam memahami gradien otomatis dari forward ke backward.

Checklist siap pakai

- Verifikasi `requires_grad` sebelum eksekusi utama.
- Catat perubahan pada `backward` dan `grad_fn`.
- Ukur `detach` dengan baseline yang adil.
- Tambahkan test untuk mencegah: graf komputasi diputus oleh `detach` atau konversi NumPy.
- Simpan artefak proyek: Visualisasi aliran gradien.

API yang muncul

`torch.tensor`, `True`, `torch.no_grad`

SATU KALIMAT Bab 7 mengajarkan cara memahami gradien otomatis dari forward ke backward dengan kontrak eksplisit dan bukti terukur.

BAGIAN 2 - TENSOR DAN AUTOGRAD

Bab 08. Custom Autograd dan Gradient Check

Membuat operasi diferensiabel khusus secara aman.

Hasil belajar

- Menjelaskan peran Function dan hubungannya dengan forward.
- Menulis notebook Colab untuk membuat operasi diferensiabel khusus secara aman.
- Mendiagnosis risiko: rumus backward tidak konsisten dengan forward.
- Menghasilkan proyek mini: Aktivasi kustom tervalidasi.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: Function, forward, backward, gradcheck.

Parameter	Target
Level	Dasar
Waktu	60-120 menit
Artefak	Aktivasi kustom tervalidasi
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 08

Custom Autograd dan Gradient Check



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah membuat operasi diferensiabel khusus secara aman. Alur di atas memperlakukan Function sebagai input keputusan, forward sebagai transformasi utama, backward sebagai state atau mekanisme kontrol, dan gradcheck sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika forward berubah, bagaimana dampaknya terhadap gradcheck? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk membuat operasi diferensiabel khusus secara aman. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 class Cube(torch.autograd.Function):
3     @staticmethod
4     def forward(ctx, x):
5         ctx.save_for_backward(x); return x**3
6     @staticmethod
7     def backward(ctx, grad_out):
8         (x,) = ctx.saved_tensors
9         return grad_out * 3 * x**2
10 x = torch.randn(4, dtype=torch.double, requires_grad=True)
11 print(torch.autograd.gradcheck(Cube.apply, (x,)))
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi gradcheck dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
Cube	Mewakili Function; catat input, output, dtype, device, serta state yang berubah.
torch.autograd.Function	Mewakili forward; catat input, output, dtype, device, serta state yang berubah.
torch.randn	Mewakili backward; catat input, output, dtype, device, serta state yang berubah.
torch.double	Mewakili gradcheck; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Rumus backward tidak konsisten dengan forward.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 08

Custom Autograd dan Gradient Check

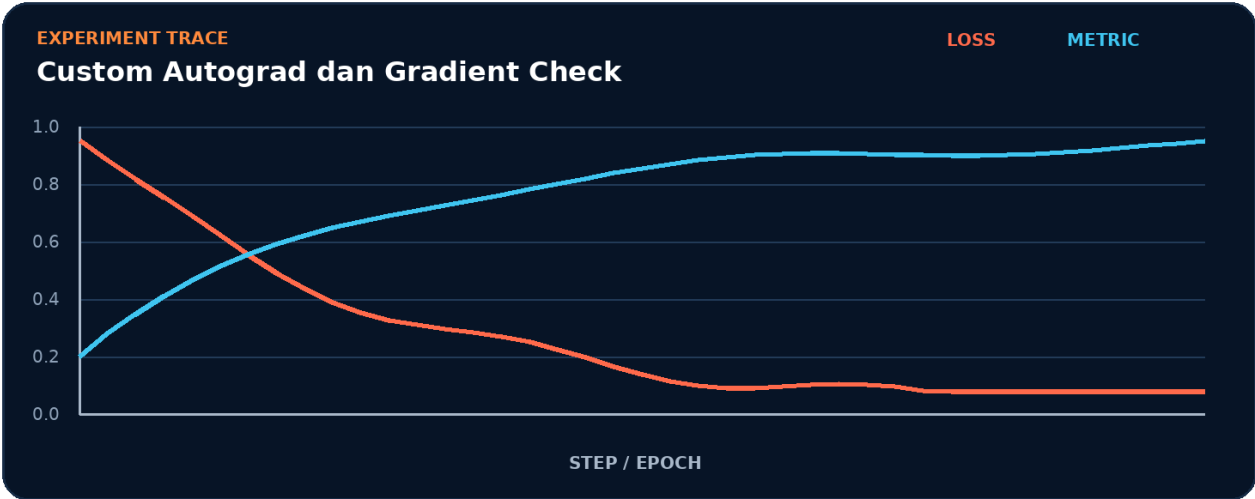


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk Function.
2. Terapkan transformasi utama: forward.
3. Kelola state atau kontrol melalui backward.
4. Ukur hasil akhir memakai gradcheck.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur Function -> forward -> backward -> gradcheck tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk gradcheck.
- 2. Ubah satu nilai yang memengaruhi forward.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada forward akan memengaruhi gradcheck.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	gradcheck
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah Function menjadi bottleneck.
- Pisahkan biaya setup dari steady-state forward.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Rumus backward tidak konsisten dengan forward.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait Function.
3. Isolasi	Nonaktifkan sementara forward dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Aktivasi kustom tervalidasi

Bangun artefak kecil yang membuktikan Anda dapat membuat operasi diferensiabel khusus secara aman. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk gradcheck.
2. Implementasikan baseline memakai Function dan forward.
3. Tambahkan logging untuk backward.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan Function tanpa menggunakan istilah forward.
2. Apa shape, dtype, dan device yang diharapkan pada batas forward?
3. Kapan backward berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas gradcheck?
5. Bagaimana Anda mereproduksi kegagalan: rumus backward tidak konsisten dengan forward?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Aktivasi kustom tervalidasi?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk forward, lalu buat tabel yang membandingkan gradcheck, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
Function	Peran inti dalam membuat operasi diferensiabel khusus secara aman.
forward	Peran inti dalam membuat operasi diferensiabel khusus secara aman.
backward	Peran inti dalam membuat operasi diferensiabel khusus secara aman.
gradcheck	Peran inti dalam membuat operasi diferensiabel khusus secara aman.

Checklist siap pakai

- Verifikasi Function sebelum eksekusi utama.
- Catat perubahan pada forward dan backward.
- Ukur gradcheck dengan baseline yang adil.
- Tambahkan test untuk mencegah: rumus backward tidak konsisten dengan forward.
- Simpan artefak proyek: Aktivasi kustom tervalidasi.

API yang muncul

Cube, torch.autograd.Function, torch.randn, torch.double, True, torch.autograd.gradcheck

SATU KALIMAT Bab 8 mengajarkan cara membuat operasi diferensiabel khusus secara aman dengan kontrak eksplisit dan bukti terukur.

BAGIAN 3 - PIPELINE DATA

Bab 09. Dataset dan DataLoader

Membangun iterasi data yang bersih dan modular.

Hasil belajar

- Menjelaskan peran Dataset dan hubungannya dengan DataLoader.
- Menulis notebook Colab untuk membangun iterasi data yang bersih dan modular.
- Mendiagnosis risiko: urutan target dan fitur tidak sinkron.
- Menghasilkan proyek mini: Loader data tabular.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: Dataset, DataLoader, batch, shuffle.

Parameter	Target
Level	Dasar
Waktu	60-120 menit
Artefak	Loader data tabular
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 09

Dataset dan DataLoader



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah membangun iterasi data yang bersih dan modular. Alur di atas memperlakukan Dataset sebagai input keputusan, DataLoader sebagai transformasi utama, batch sebagai state atau mekanisme kontrol, dan shuffle sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika DataLoader berubah, bagaimana dampaknya terhadap shuffle? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk membangun iterasi data yang bersih dan modular. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch.utils.data import Dataset, DataLoader
3 class Points(Dataset):
4     def __init__(self, n=1000):
5         self.x = torch.randn(n, 2); self.y = (self.x.sum(1) > 0).long()
6     def __len__(self): return len(self.x)
7     def __getitem__(self, i): return self.x[i], self.y[i]
8 loader = DataLoader(Points(), batch_size=64, shuffle=True)
9 xb, yb = next(iter(loader)); print(xb.shape, yb.shape)
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi shuffle dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.utils.data	Mewakili Dataset; catat input, output, dtype, device, serta state yang berubah.
Dataset	Mewakili DataLoader; catat input, output, dtype, device, serta state yang berubah.
DataLoader	Mewakili batch; catat input, output, dtype, device, serta state yang berubah.
Points	Mewakili shuffle; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Urutan target dan fitur tidak sinkron.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 09

Dataset dan DataLoader

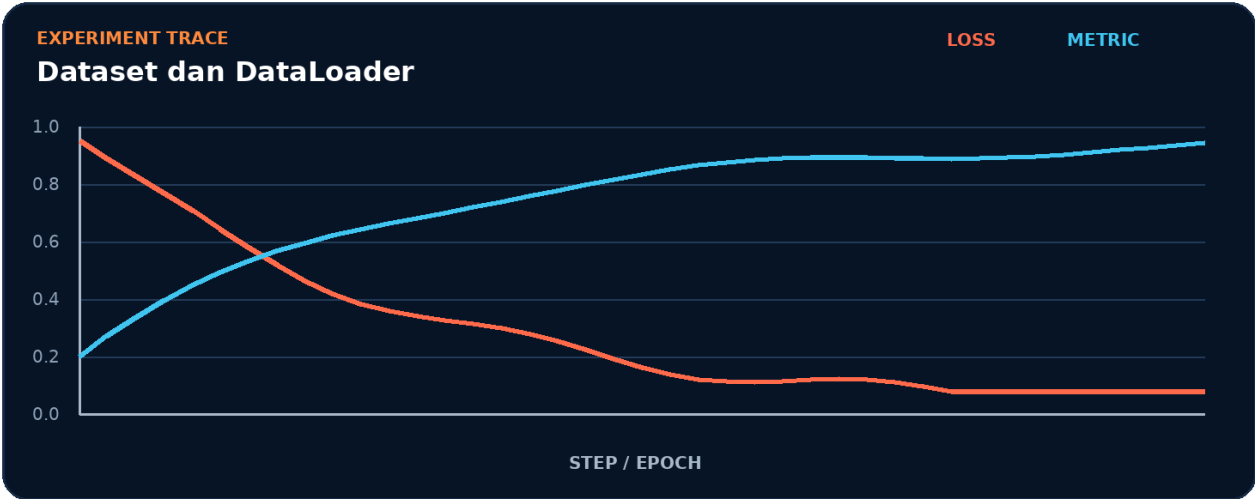


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk Dataset.
2. Terapkan transformasi utama: DataLoader.
3. Kelola state atau kontrol melalui batch.
4. Ukur hasil akhir memakai shuffle.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur Dataset -> DataLoader -> batch -> shuffle tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk shuffle.
- 2. Ubah satu nilai yang memengaruhi DataLoader.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada DataLoader akan memengaruhi shuffle.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	shuffle
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah Dataset menjadi bottleneck.
- Pisahkan biaya setup dari steady-state DataLoader.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Urutan target dan fitur tidak sinkron.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait Dataset.
3. Isolasi	Nonaktifkan sementara DataLoader dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Loader data tabular

Bangun artefak kecil yang membuktikan Anda dapat membangun iterasi data yang bersih dan modular. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk shuffle.
2. Implementasikan baseline memakai Dataset dan DataLoader.
3. Tambahkan logging untuk batch.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan Dataset tanpa menggunakan istilah DataLoader.
2. Apa shape, dtype, dan device yang diharapkan pada batas DataLoader?
3. Kapan batch berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas shuffle?
5. Bagaimana Anda mereproduksi kegagalan: urutan target dan fitur tidak sinkron?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Loader data tabular?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk DataLoader, lalu buat tabel yang membandingkan shuffle, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
Dataset	Peran inti dalam membangun iterasi data yang bersih dan modular.
DataLoader	Peran inti dalam membangun iterasi data yang bersih dan modular.
batch	Peran inti dalam membangun iterasi data yang bersih dan modular.
shuffle	Peran inti dalam membangun iterasi data yang bersih dan modular.

Checklist siap pakai

- Verifikasi Dataset sebelum eksekusi utama.
- Catat perubahan pada DataLoader dan batch.
- Ukur shuffle dengan baseline yang adil.
- Tambahkan test untuk mencegah: urutan target dan fitur tidak sinkron.
- Simpan artefak proyek: Loader data tabular.

API yang muncul

torch.utils.data, Dataset, DataLoader, Points, torch.randn, True

SATU KALIMAT Bab 9 mengajarkan cara membangun iterasi data yang bersih dan modular dengan kontrak eksplisit dan bukti terukur.

BAGIAN 3 - PIPELINE DATA

Bab 10. Transforms dan Augmentasi

Membuat preprocessing yang konsisten antara train dan inference.

Hasil belajar

- Menjelaskan peran transform dan hubungannya dengan normalize.
- Menulis notebook Colab untuk membuat preprocessing yang konsisten antara train dan inference.
- Mendiagnosis risiko: augmentasi acak ikut diterapkan saat validasi.
- Menghasilkan proyek mini: Galeri augmentasi gambar.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: transform, normalize, augmentation, pipeline.

Parameter	Target
Level	Dasar
Waktu	60-120 menit
Artefak	Galeri augmentasi gambar
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 10

Transforms dan Augmentasi



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah membuat preprocessing yang konsisten antara train dan inference. Alur di atas memperlakukan transform sebagai input keputusan, normalize sebagai transformasi utama, augmentation sebagai state atau mekanisme kontrol, dan pipeline sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika normalize berubah, bagaimana dampaknya terhadap pipeline? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk membuat preprocessing yang konsisten antara train dan inference. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 from torchvision.transforms import v2
2 train_tf = v2.Compose([
3     v2.RandomResizedCrop((224, 224), antialias=True),
4     v2.RandomHorizontalFlip(),
5     v2.ToDtype(torch.float32, scale=True),
6     v2.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
7 ])
8 valid_tf = v2.Compose([
9     v2.Resize((224, 224), antialias=True),
10    v2.ToDtype(torch.float32, scale=True),
11    v2.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
12 ])
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi pipeline dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
Compose	Mewakili transform; catat input, output, dtype, device, serta state yang berubah.
RandomResizedCrop	Mewakili normalize; catat input, output, dtype, device, serta state yang berubah.
True	Mewakili augmentation; catat input, output, dtype, device, serta state yang berubah.
RandomHorizontalFlip	Mewakili pipeline; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Augmentasi acak ikut diterapkan saat validasi.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 10

Transforms dan Augmentasi

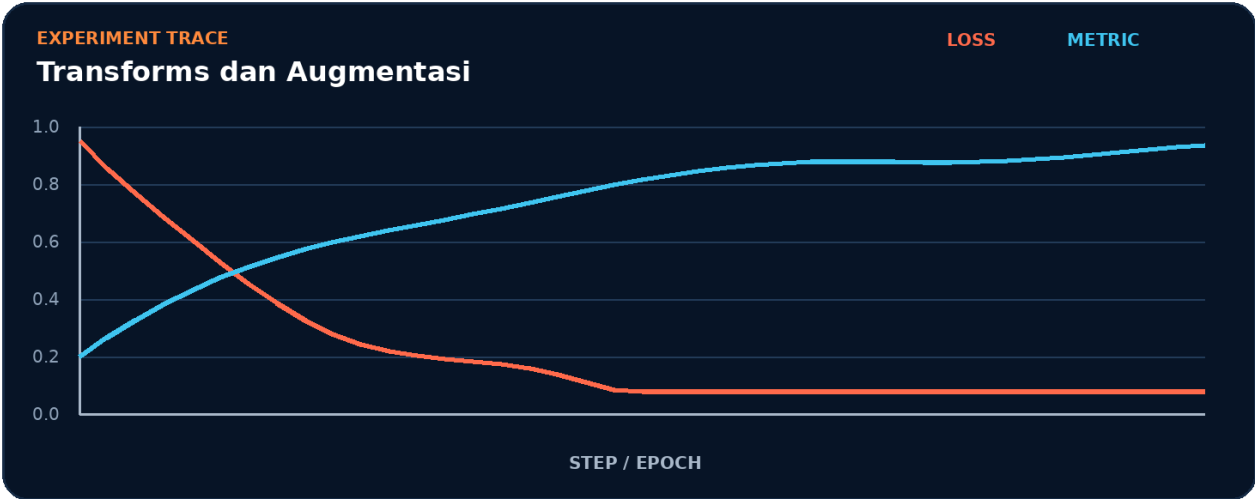


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk transform.
2. Terapkan transformasi utama: normalize.
3. Kelola state atau kontrol melalui augmentation.
4. Ukur hasil akhir memakai pipeline.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur transform -> normalize -> augmentation -> pipeline tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk pipeline.
- 2. Ubah satu nilai yang memengaruhi normalize.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada normalize akan memengaruhi pipeline.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	pipeline
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah transform menjadi bottleneck.
- Pisahkan biaya setup dari steady-state normalize.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Augmentasi acak ikut diterapkan saat validasi.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait transform.
3. Isolasi	Nonaktifkan sementara normalize dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Galeri augmentasi gambar

Bangun artefak kecil yang membuktikan Anda dapat membuat preprocessing yang konsisten antara train dan inference. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk pipeline.
2. Implementasikan baseline memakai transform dan normalize.
3. Tambahkan logging untuk augmentation.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan transform tanpa menggunakan istilah normalize.
2. Apa shape, dtype, dan device yang diharapkan pada batas normalize?
3. Kapan augmentation berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas pipeline?
5. Bagaimana Anda mereproduksi kegagalan: augmentasi acak ikut diterapkan saat validasi?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Galeri augmentasi gambar?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk normalize, lalu buat tabel yang membandingkan pipeline, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
transform	Peran inti dalam membuat preprocessing yang konsisten antara train dan inference.
normalize	Peran inti dalam membuat preprocessing yang konsisten antara train dan inference.
augmentation	Peran inti dalam membuat preprocessing yang konsisten antara train dan inference.
pipeline	Peran inti dalam membuat preprocessing yang konsisten antara train dan inference.

Checklist siap pakai

- Verifikasi transform sebelum eksekusi utama.
- Catat perubahan pada normalize dan augmentation.
- Ukur pipeline dengan baseline yang adil.
- Tambahkan test untuk mencegah: augmentasi acak ikut diterapkan saat validasi.
- Simpan artefak proyek: Galeri augmentasi gambar.

API yang muncul

Compose, RandomResizedCrop, True, RandomHorizontalFlip, ToDtype, torch.float32

SATU KALIMAT Bab 10 mengajarkan cara membuat preprocessing yang konsisten antara train dan inference dengan kontrak eksplisit dan bukti terukur.

BAGIAN 3 - PIPELINE DATA

Bab 11. Custom Dataset untuk Data Nyata

Membaca citra, teks, dan tabel dengan kontrak yang jelas.

Hasil belajar

- Menjelaskan peran `__len__` dan hubungannya dengan `__getitem__`.
- Menulis notebook Colab untuk membaca citra, teks, dan tabel dengan kontrak yang jelas.
- Mendiagnosis risiko: sampel memiliki panjang atau tipe tidak konsisten.
- Menghasilkan proyek mini: Dataset multi-input.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: `__len__`, `__getitem__`, `schema`, `collate`.

Parameter	Target
Level	Dasar
Waktu	60-120 menit
Artefak	Dataset multi-input
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 11

Custom Dataset untuk Data Nyata



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah membaca citra, teks, dan tabel dengan kontrak yang jelas. Alur di atas memperlakukan `__len__` sebagai input keputusan, `__getitem__` sebagai transformasi utama, `schema` sebagai state atau mekanisme kontrol, dan `collate` sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika `__getitem__` berubah, bagaimana dampaknya terhadap `collate`? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk membaca citra, teks, dan tabel dengan kontrak yang jelas. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch.utils.data import Dataset
3 class MultiInput(Dataset):
4     def __init__(self, rows): self.rows = rows
5     def __len__(self): return len(self.rows)
6     def __getitem__(self, i):
7         row = self.rows[i]
8         numeric = torch.tensor(row['x'], dtype=torch.float32)
9         token_ids = torch.tensor(row['tokens'], dtype=torch.long)
10        target = torch.tensor(row['y'], dtype=torch.long)
11        return {'x': numeric, 'tokens': token_ids, 'target': target}
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi collate dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.utils.data	Mewakili <code>__len__</code> ; catat input, output, dtype, device, serta state yang berubah.
Dataset	Mewakili <code>__getitem__</code> ; catat input, output, dtype, device, serta state yang berubah.
MultilInput	Mewakili schema; catat input, output, dtype, device, serta state yang berubah.
torch.tensor	Mewakili <code>collate</code> ; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Sampel memiliki panjang atau tipe tidak konsisten.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 11

Custom Dataset untuk Data Nyata

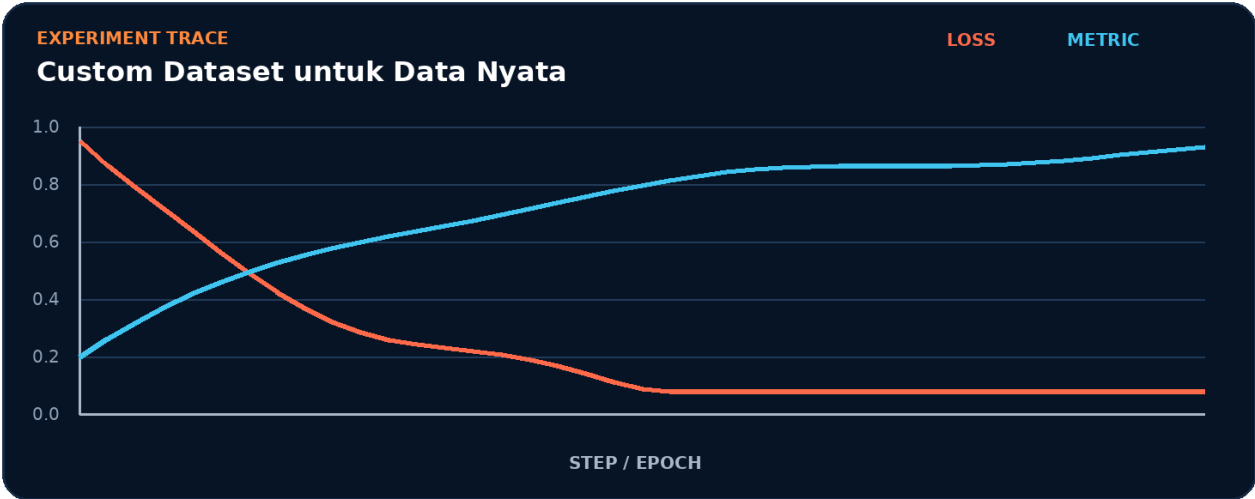


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk `__len__`.
2. Terapkan transformasi utama: `__getitem__`.
3. Kelola state atau kontrol melalui schema.
4. Ukur hasil akhir memakai `collate`.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur `__len__` -> `__getitem__` -> `schema` -> `collate` tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk collate.
- 2. Ubah satu nilai yang memengaruhi `__getitem__`.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada <code>__getitem__</code> akan memengaruhi collate.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	collate
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah `__len__` menjadi bottleneck.
- Pisahkan biaya setup dari steady-state `__getitem__`.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Sampel memiliki panjang atau tipe tidak konsisten.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait <code>__len__</code> .
3. Isolasi	Nonaktifkan sementara <code>__getitem__</code> dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Dataset multi-input

Bangun artefak kecil yang membuktikan Anda dapat membaca citra, teks, dan tabel dengan kontrak yang jelas. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk collate.
2. Implementasikan baseline memakai `__len__` dan `__getitem__`.
3. Tambahkan logging untuk schema.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan `__len__` tanpa menggunakan istilah `__getitem__`.
2. Apa shape, dtype, dan device yang diharapkan pada batas `__getitem__`?
3. Kapan schema berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas collate?
5. Bagaimana Anda mereproduksi kegagalan: sampel memiliki panjang atau tipe tidak konsisten?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Dataset multi-input?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk `__getitem__`, lalu buat tabel yang membandingkan collate, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
<code>__len__</code>	Peran inti dalam membaca citra, teks, dan tabel dengan kontrak yang jelas.
<code>__getitem__</code>	Peran inti dalam membaca citra, teks, dan tabel dengan kontrak yang jelas.
<code>schema</code>	Peran inti dalam membaca citra, teks, dan tabel dengan kontrak yang jelas.
<code>collate</code>	Peran inti dalam membaca citra, teks, dan tabel dengan kontrak yang jelas.

Checklist siap pakai

- Verifikasi `__len__` sebelum eksekusi utama.
- Catat perubahan pada `__getitem__` dan `schema`.
- Ukur `collate` dengan baseline yang adil.
- Tambahkan test untuk mencegah: sampel memiliki panjang atau tipe tidak konsisten.
- Simpan artefak proyek: Dataset multi-input.

API yang muncul

`torch.utils.data`, `Dataset`, `MultilInput`, `torch.tensor`, `torch.float32`, `torch.long`

SATU KALIMAT Bab 11 mengajarkan cara membaca citra, teks, dan tabel dengan kontrak yang jelas dengan kontrak eksplisit dan bukti terukur.

BAGIAN 3 - PIPELINE DATA

Bab 12. Optimasi Input Pipeline

Mengurangi waktu tunggu gpu akibat data loading.

Hasil belajar

- Menjelaskan peran num_workers dan hubungannya dengan pin_memory.
- Menulis notebook Colab untuk mengurangi waktu tunggu GPU akibat data loading.
- Mendiagnosis risiko: menambah worker tanpa mengukur bottleneck I/O.
- Menghasilkan proyek mini: Benchmark throughput loader.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: num_workers, pin_memory, prefetch, persistent_workers.

Parameter	Target
Level	Dasar
Waktu	60-120 menit
Artefak	Benchmark throughput loader
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 12

Optimasi Input Pipeline



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah mengurangi waktu tunggu GPU akibat data loading. Alur di atas memperlakukan `num_workers` sebagai input keputusan, `pin_memory` sebagai transformasi utama, `prefetch` sebagai state atau mekanisme kontrol, dan `persistent_workers` sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika `pin_memory` berubah, bagaimana dampaknya terhadap `persistent_workers`? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk mengurangi waktu tunggu GPU akibat data loading. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import time, torch
2 from torch.utils.data import DataLoader
3 def benchmark(dataset, workers):
4     loader = DataLoader(dataset, batch_size=256, num_workers=workers,
5                         pin_memory=torch.cuda.is_available(),
6                         persistent_workers=workers > 0)
7     start = time.perf_counter(); n = 0
8     for x, _ in loader:
9         n += len(x)
10        if n >= 4096: break
11    return n / (time.perf_counter() - start)
12 for w in [0, 2, 4]: print(w, benchmark(dataset, w))
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi persistent_workers dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.utils.data	Mewakili num_workers; catat input, output, dtype, device, serta state yang berubah.
DataLoader	Mewakili pin_memory; catat input, output, dtype, device, serta state yang berubah.
torch.cuda.is_available	Mewakili prefetch; catat input, output, dtype, device, serta state yang berubah.
torch.utils.data	Mewakili persistent_workers; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Menambah worker tanpa mengukur bottleneck i/o.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 12

Optimasi Input Pipeline

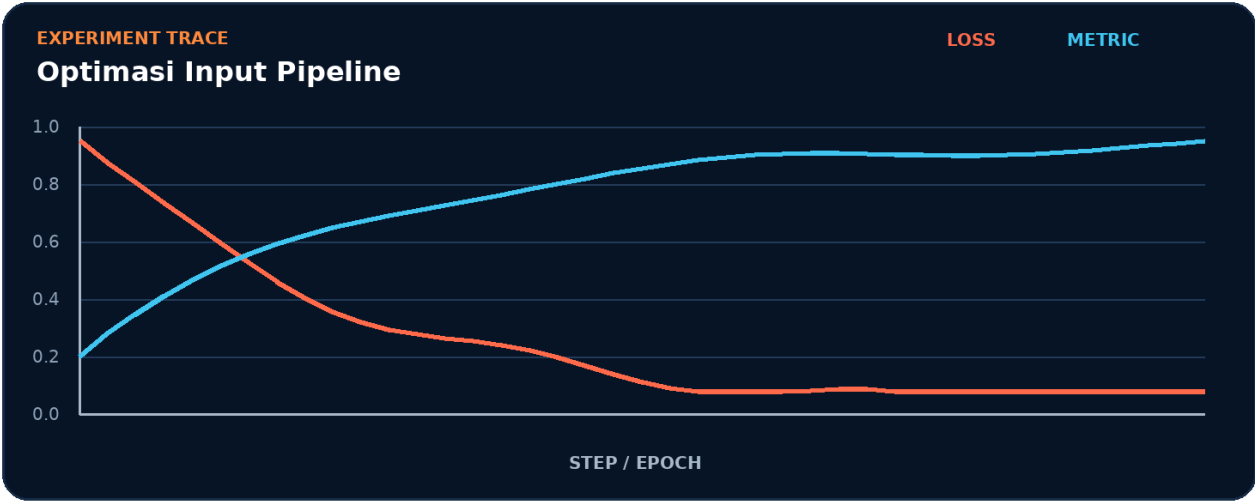


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk num_workers.
2. Terapkan transformasi utama: pin_memory.
3. Kelola state atau kontrol melalui prefetch.
4. Ukur hasil akhir memakai persistent_workers.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur num_workers -> pin_memory -> prefetch -> persistent_workers tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk persistent_workers.
- 2. Ubah satu nilai yang memengaruhi pin_memory.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada pin_memory akan memengaruhi persistent_workers.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	persistent_workers
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah num_workers menjadi bottleneck.
- Pisahkan biaya setup dari steady-state pin_memory.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Menambah worker tanpa mengukur bottleneck i/o.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait num_workers.
3. Isolasi	Nonaktifkan sementara pin_memory dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```

1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()

```

Mini Project: Benchmark throughput loader

Bangun artefak kecil yang membuktikan Anda dapat mengurangi waktu tunggu GPU akibat data loading. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk `persistent_workers`.
2. Implementasikan baseline memakai `num_workers` dan `pin_memory`.
3. Tambahkan logging untuk prefetch.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan `num_workers` tanpa menggunakan istilah `pin_memory`.
2. Apa `shape`, `dtype`, dan `device` yang diharapkan pada batas `pin_memory`?
3. Kapan `prefetch` berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas `persistent_workers`?
5. Bagaimana Anda mereproduksi kegagalan: menambah worker tanpa mengukur bottleneck I/O?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Benchmark throughput loader?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk `pin_memory`, lalu buat tabel yang membandingkan `persistent_workers`, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
<code>num_workers</code>	Peran inti dalam mengurangi waktu tunggu GPU akibat data loading.
<code>pin_memory</code>	Peran inti dalam mengurangi waktu tunggu GPU akibat data loading.
<code>prefetch</code>	Peran inti dalam mengurangi waktu tunggu GPU akibat data loading.
<code>persistent_workers</code>	Peran inti dalam mengurangi waktu tunggu GPU akibat data loading.

Checklist siap pakai

- Verifikasi `num_workers` sebelum eksekusi utama.
- Catat perubahan pada `pin_memory` dan `prefetch`.
- Ukur `persistent_workers` dengan baseline yang adil.
- Tambahkan test untuk mencegah: menambah worker tanpa mengukur bottleneck I/O.
- Simpan artefak proyek: Benchmark throughput loader.

API yang muncul

`torch.utils.data`, `DataLoader`, `torch.cuda.is_available`

SATU KALIMAT Bab 12 mengajarkan cara mengurangi waktu tunggu GPU akibat data loading dengan kontrak eksplisit dan bukti terukur.

BAGIAN 4 - JARINGAN SARAF DAN TRAINING

Bab 13. Mendesain nn.Module

Menyusun layer menjadi model yang mudah diuji.

Hasil belajar

- Menjelaskan peran Module dan hubungannya dengan forward.
- Menulis notebook Colab untuk menyusun layer menjadi model yang mudah diuji.
- Mendiagnosis risiko: layer disimpan dalam list Python biasa sehingga tidak terdaftar.
- Menghasilkan proyek mini: MLP modular.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: Module, forward, parameters, state_dict.

Parameter	Target
Level	Dasar
Waktu	60-120 menit
Artefak	MLP modular
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 13

Mendesain nn.Module



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah menyusun layer menjadi model yang mudah diuji. Alur di atas memperlakukan Module sebagai input keputusan, forward sebagai transformasi utama, parameters sebagai state atau mekanisme kontrol, dan state_dict sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika forward berubah, bagaimana dampaknya terhadap state_dict? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk menyusun layer menjadi model yang mudah diuji. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch import nn
3 class MLP(nn.Module):
4     def __init__(self, d_in=20, d_hidden=64, n_class=3):
5         super().__init__()
6         self.net = nn.Sequential(nn.Linear(d_in, d_hidden), nn.GELU(),
7                                   nn.LayerNorm(d_hidden), nn.Linear(d_hidden, n_class))
8     def forward(self, x): return self.net(x)
9 model = MLP(); print(sum(p.numel() for p in model.parameters()))
10 print(model(torch.randn(8, 20)).shape)
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi state_dict dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
MLP	Mewakili Module; catat input, output, dtype, device, serta state yang berubah.
nn.Module	Mewakili forward; catat input, output, dtype, device, serta state yang berubah.
nn.Sequential	Mewakili parameters; catat input, output, dtype, device, serta state yang berubah.
nn.Linear	Mewakili state_dict; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Layer disimpan dalam list python biasa sehingga tidak terdaftar.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 13

Mendesain nn.Module

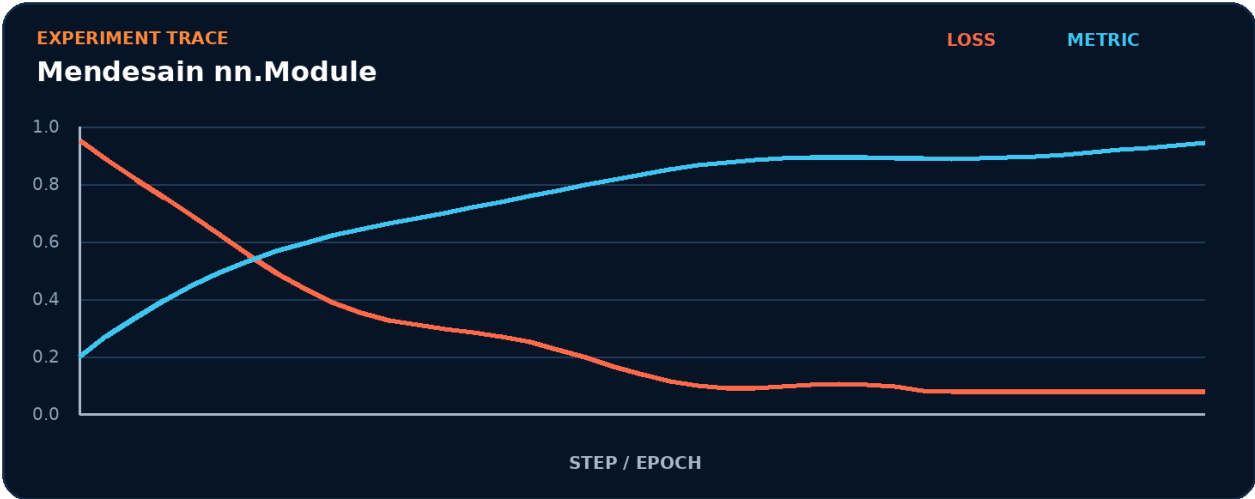


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk Module.
2. Terapkan transformasi utama: forward.
3. Kelola state atau kontrol melalui parameters.
4. Ukur hasil akhir memakai state_dict.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur Module -> forward -> parameters -> state_dict tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk state_dict.
- 2. Ubah satu nilai yang memengaruhi forward.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada forward akan memengaruhi state_dict.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	state_dict
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah Module menjadi bottleneck.
- Pisahkan biaya setup dari steady-state forward.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Layer disimpan dalam list python biasa sehingga tidak terdaftar.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait Module.
3. Isolasi	Nonaktifkan sementara forward dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```

1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()

```

Mini Project: MLP modular

Bangun artefak kecil yang membuktikan Anda dapat menyusun layer menjadi model yang mudah diuji. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk `state_dict`.
2. Implementasikan baseline memakai `Module` dan `forward`.
3. Tambahkan logging untuk parameters.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan Module tanpa menggunakan istilah forward.
2. Apa shape, dtype, dan device yang diharapkan pada batas forward?
3. Kapan parameters berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas state_dict?
5. Bagaimana Anda mereproduksi kegagalan: layer disimpan dalam list Python biasa sehingga tidak terdaftar?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek MLP modular?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk forward, lalu buat tabel yang membandingkan state_dict, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
Module	Peran inti dalam menyusun layer menjadi model yang mudah diuji.
forward	Peran inti dalam menyusun layer menjadi model yang mudah diuji.
parameters	Peran inti dalam menyusun layer menjadi model yang mudah diuji.
state_dict	Peran inti dalam menyusun layer menjadi model yang mudah diuji.

Checklist siap pakai

- Verifikasi Module sebelum eksekusi utama.
- Catat perubahan pada forward dan parameters.
- Ukur state_dict dengan baseline yang adil.
- Tambahkan test untuk mencegah: layer disimpan dalam list Python biasa sehingga tidak terdaftar.
- Simpan artefak proyek: MLP modular.

API yang muncul

MLP, nn.Module, nn.Sequential, nn.Linear, nn.GELU, nn.LayerNorm

SATU KALIMAT Bab 13 mengajarkan cara menyusun layer menjadi model yang mudah diuji dengan kontrak eksplisit dan bukti terukur.

BAGIAN 4 - JARINGAN SARAF DAN TRAINING

Bab 14. Loss Function dan Optimizer

Memilih objective dan aturan update yang tepat.

Hasil belajar

- Menjelaskan peran cross_entropy dan hubungannya dengan MSE.
- Menulis notebook Colab untuk memilih objective dan aturan update yang tepat.
- Mendiagnosis risiko: menerapkan softmax sebelum CrossEntropyLoss.
- Menghasilkan proyek mini: Eksperimen optimizer.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: cross_entropy, MSE, AdamW, SGD.

Parameter	Target
Level	Dasar
Waktu	60-120 menit
Artefak	Eksperimen optimizer
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 14

Loss Function dan Optimizer



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah memilih objective dan aturan update yang tepat. Alur di atas memperlakukan `cross_entropy` sebagai input keputusan, MSE sebagai transformasi utama, AdamW sebagai state atau mekanisme kontrol, dan SGD sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika MSE berubah, bagaimana dampaknya terhadap SGD? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk memilih objective dan aturan update yang tepat. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch import nn
3 logits = torch.randn(32, 5, requires_grad=True)
4 target = torch.randint(0, 5, (32,))
5 loss = nn.CrossEntropyLoss(label_smoothing=0.1)(logits, target)
6 opt = torch.optim.AdamW([logits], lr=1e-3, weight_decay=1e-2)
7 opt.zero_grad(set_to_none=True); loss.backward(); opt.step()
8 print(float(loss))
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi SGD dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.randn	Mewakili cross_entropy; catat input, output, dtype, device, serta state yang berubah.
True	Mewakili MSE; catat input, output, dtype, device, serta state yang berubah.
torch.randint	Mewakili AdamW; catat input, output, dtype, device, serta state yang berubah.
nn.CrossEntropyLoss	Mewakili SGD; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Menerapkan softmax sebelum crossentropyloss.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 14

Loss Function dan Optimizer

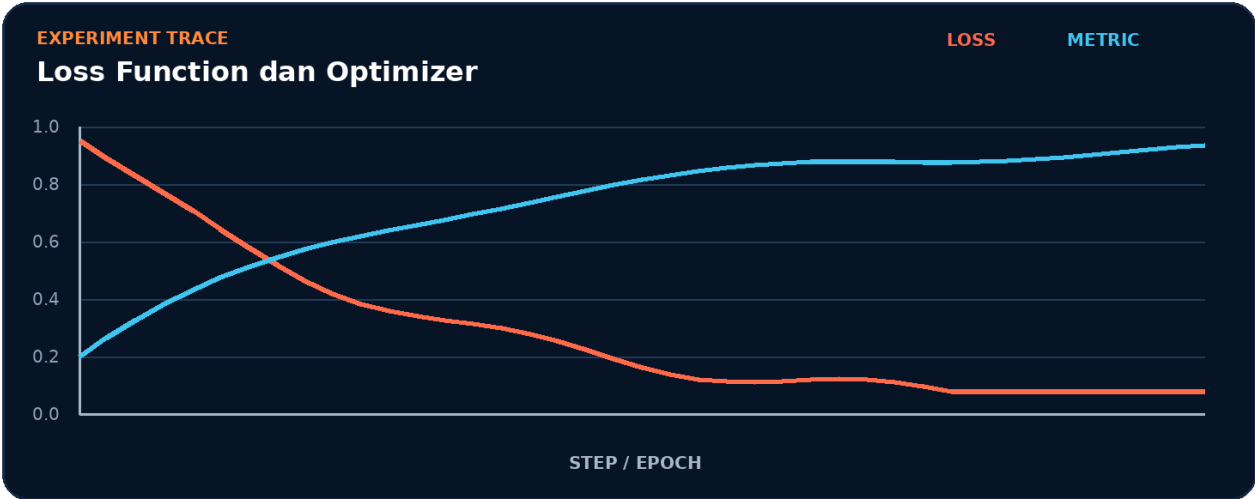


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk `cross_entropy`.
2. Terapkan transformasi utama: MSE.
3. Kelola state atau kontrol melalui AdamW.
4. Ukur hasil akhir memakai SGD.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur `cross_entropy` -> MSE -> AdamW -> SGD tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk SGD.
- 2. Ubah satu nilai yang memengaruhi MSE.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada MSE akan memengaruhi SGD.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	SGD
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah cross_entropy menjadi bottleneck.
- Pisahkan biaya setup dari steady-state MSE.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Menerapkan softmax sebelum crossentropyloss.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait cross_entropy.
3. Isolasi	Nonaktifkan sementara MSE dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

```
COLAB - DEBUG CELL
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Eksperimen optimizer

Bangun artefak kecil yang membuktikan Anda dapat memilih objective dan aturan update yang tepat. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk SGD.
2. Implementasikan baseline memakai `cross_entropy` dan `MSE`.
3. Tambahkan logging untuk AdamW.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan `cross_entropy` tanpa menggunakan istilah MSE.
2. Apa `shape`, `dtype`, dan `device` yang diharapkan pada batas MSE?
3. Kapan AdamW berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas SGD?
5. Bagaimana Anda mereproduksi kegagalan: menerapkan softmax sebelum `CrossEntropyLoss`?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Eksperimen optimizer?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk MSE, lalu buat tabel yang membandingkan SGD, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
cross_entropy	Peran inti dalam memilih objective dan aturan update yang tepat.
MSE	Peran inti dalam memilih objective dan aturan update yang tepat.
AdamW	Peran inti dalam memilih objective dan aturan update yang tepat.
SGD	Peran inti dalam memilih objective dan aturan update yang tepat.

Checklist siap pakai

- Verifikasi `cross_entropy` sebelum eksekusi utama.
- Catat perubahan pada MSE dan AdamW.
- Ukur SGD dengan baseline yang adil.
- Tambahkan test untuk mencegah: menerapkan softmax sebelum `CrossEntropyLoss`.
- Simpan artefak proyek: Eksperimen optimizer.

API yang muncul

`torch.randn`, `True`, `torch.randint`, `nn.CrossEntropyLoss`, `torch.optim.AdamW`

SATU KALIMAT Bab 14 mengajarkan cara memilih objective dan aturan update yang tepat dengan kontrak eksplisit dan bukti terukur.

BAGIAN 4 - JARINGAN SARAF DAN TRAINING

Bab 15. Training Loop yang Benar

Membangun loop train-validasi yang dapat diaudit.

Hasil belajar

- Menjelaskan peran zero_grad dan hubungannya dengan backward.
- Menulis notebook Colab untuk membangun loop train-validasi yang dapat diaudit.
- Mendiagnosis risiko: gradien menumpuk karena zero_grad terlewat.
- Menghasilkan proyek mini: Trainer minimal reusable.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: zero_grad, backward, step, no_grad.

Parameter	Target
Level	Dasar
Waktu	60-120 menit
Artefak	Trainer minimal reusable
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 15

Training Loop yang Benar



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah membangun loop train-validasi yang dapat diaudit. Alur di atas memperlakukan `zero_grad` sebagai input keputusan, `backward` sebagai transformasi utama, `step` sebagai state atau mekanisme kontrol, dan `no_grad` sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika `backward` berubah, bagaimana dampaknya terhadap `no_grad`? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk membangun loop train-validasi yang dapat diaudit. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 def run_epoch(model, loader, optimizer=None, device='cpu'):
2     training = optimizer is not None; model.train(training)
3     total_loss = total_correct = total = 0
4     context = torch.enable_grad() if training else torch.inference_mode()
5     with context:
6         for x, y in loader:
7             x, y = x.to(device), y.to(device); logits = model(x)
8             loss = torch.nn.functional.cross_entropy(logits, y)
9             if training:
10                 optimizer.zero_grad(set_to_none=True); loss.backward(); optimizer.step()
11                 total_loss += loss.item()*len(y); total_correct += (logits.argmax(1)==y).sum().item();
12 total += len(y)
13 return total_loss/total, total_correct/total
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi no_grad dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
None	Mewakili zero_grad; catat input, output, dtype, device, serta state yang berubah.
torch.enable_grad	Mewakili backward; catat input, output, dtype, device, serta state yang berubah.
torch.inference_mode	Mewakili step; catat input, output, dtype, device, serta state yang berubah.
torch.nn.functional.cross_entropy	Mewakili no_grad; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Gradien menumpuk karena zero_grad terlewat.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 15

Training Loop yang Benar

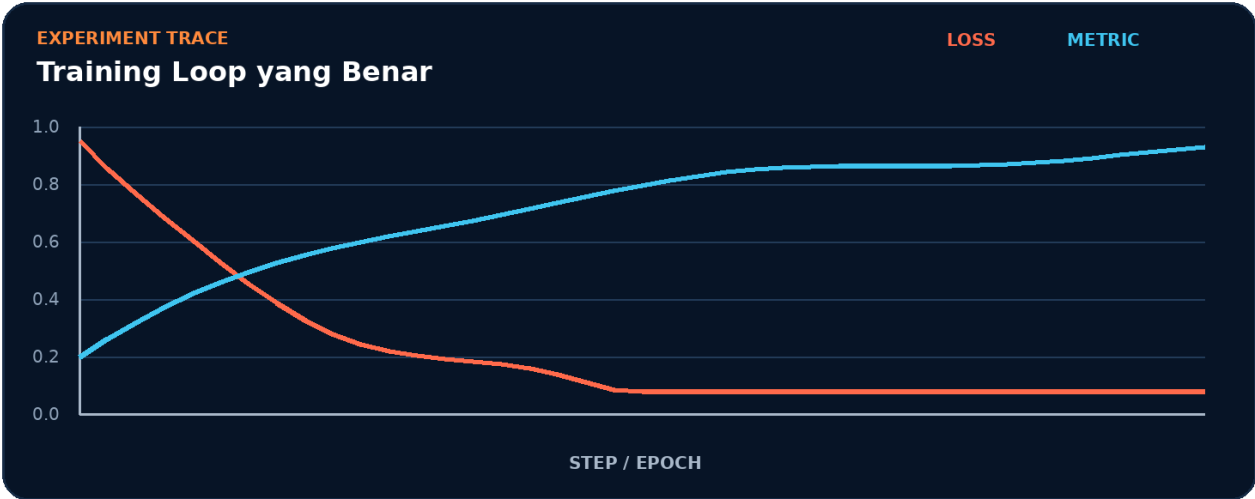


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk zero_grad.
2. Terapkan transformasi utama: backward.
3. Kelola state atau kontrol melalui step.
4. Ukur hasil akhir memakai no_grad.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur zero_grad -> backward -> step -> no_grad tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk no_grad.
- 2. Ubah satu nilai yang memengaruhi backward.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada backward akan memengaruhi no_grad.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	no_grad
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah zero_grad menjadi bottleneck.
- Pisahkan biaya setup dari steady-state backward.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Gradien menumpuk karena zero_grad terlewat.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait zero_grad.
3. Isolasi	Nonaktifkan sementara backward dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```

1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()

```

Mini Project: Trainer minimal reusable

Bangun artefak kecil yang membuktikan Anda dapat membangun loop train-validasi yang dapat diaudit. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk no_grad.
2. Implementasikan baseline memakai zero_grad dan backward.
3. Tambahkan logging untuk step.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan zero_grad tanpa menggunakan istilah backward.
2. Apa shape, dtype, dan device yang diharapkan pada batas backward?
3. Kapan step berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas no_grad?
5. Bagaimana Anda mereproduksi kegagalan: gradien menumpuk karena zero_grad terlewat?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Trainer minimal reusable?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk backward, lalu buat tabel yang membandingkan no_grad, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
zero_grad	Peran inti dalam membangun loop train-validasi yang dapat diaudit.
backward	Peran inti dalam membangun loop train-validasi yang dapat diaudit.
step	Peran inti dalam membangun loop train-validasi yang dapat diaudit.
no_grad	Peran inti dalam membangun loop train-validasi yang dapat diaudit.

Checklist siap pakai

- Verifikasi `zero_grad` sebelum eksekusi utama.
- Catat perubahan pada `backward` dan `step`.
- Ukur `no_grad` dengan baseline yang adil.
- Tambahkan test untuk mencegah: gradien menumpuk karena `zero_grad` terlewat.
- Simpan artefak proyek: Trainer minimal reusable.

API yang muncul

None, torch.enable_grad, torch.inference_mode, torch.nn.functional.cross_entropy, True

SATU KALIMAT Bab 15 mengajarkan cara membangun loop train-validasi yang dapat diaudit dengan kontrak eksplisit dan bukti terukur.

BAGIAN 4 - JARINGAN SARAF DAN TRAINING

Bab 16. Regularisasi, Inisialisasi, dan Normalisasi

Meningkatkan stabilitas serta generalisasi model.

Hasil belajar

- Menjelaskan peran dropout dan hubungannya dengan `weight_decay`.
- Menulis notebook Colab untuk meningkatkan stabilitas serta generalisasi model.
- Mendiagnosis risiko: membandingkan eksperimen dengan lebih dari satu variabel berubah.
- Menghasilkan proyek mini: Ablation regularisasi.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: dropout, `weight_decay`, initialization, normalization.

Parameter	Target
Level	Dasar
Waktu	60-120 menit
Artefak	Ablation regularisasi
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 16

Regularisasi, Inisialisasi, dan Normalisasi



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah meningkatkan stabilitas serta generalisasi model. Alur di atas memperlakukan dropout sebagai input keputusan, weight_decay sebagai transformasi utama, initialization sebagai state atau mekanisme kontrol, dan normalization sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika weight_decay berubah, bagaimana dampaknya terhadap normalization? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk meningkatkan stabilitas serta generalisasi model. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch import nn
3 block = nn.Sequential(nn.Linear(128, 256), nn.LayerNorm(256),
4                       nn.GELU(), nn.Dropout(0.2), nn.Linear(256, 10))
5 for m in block.modules():
6     if isinstance(m, nn.Linear):
7         nn.init.kaiming_uniform_(m.weight, nonlinearity='relu')
8         nn.init.zeros_(m.bias)
9 opt = torch.optim.AdamW(block.parameters(), lr=3e-4, weight_decay=0.05)
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi normalization dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
nn.Sequential	Mewakili dropout; catat input, output, dtype, device, serta state yang berubah.
nn.Linear	Mewakili weight_decay; catat input, output, dtype, device, serta state yang berubah.
nn.LayerNorm	Mewakili initialization; catat input, output, dtype, device, serta state yang berubah.
nn.GELU	Mewakili normalization; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Membandingkan eksperimen dengan lebih dari satu variabel berubah.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 16

Regularisasi, Inisialisasi, dan Normalisasi

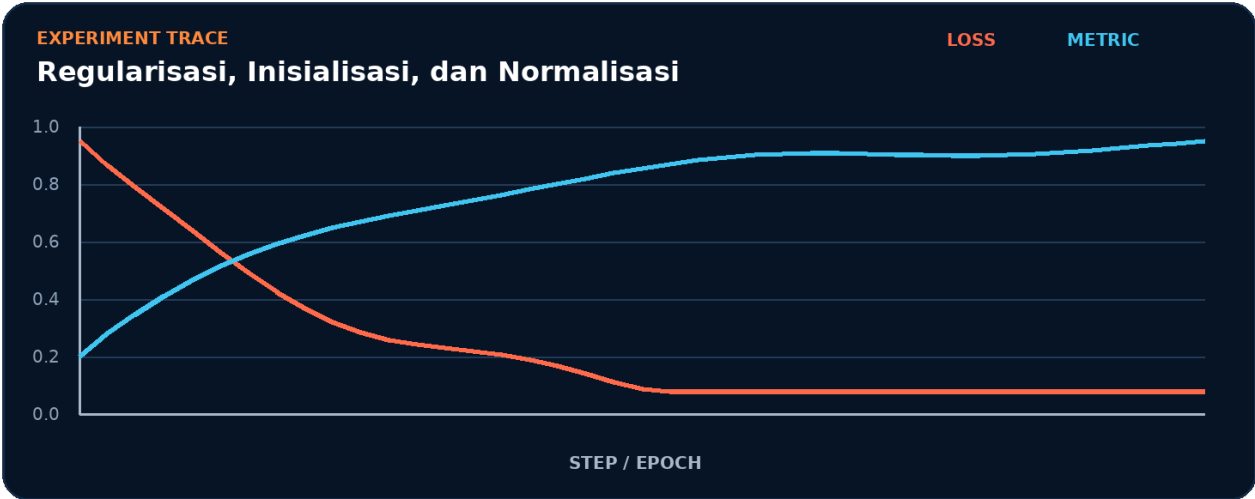


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk dropout.
2. Terapkan transformasi utama: `weight_decay`.
3. Kelola state atau kontrol melalui `initialization`.
4. Ukur hasil akhir memakai `normalization`.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur dropout -> `weight_decay` -> `initialization` -> `normalization` tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk normalization.
- 2. Ubah satu nilai yang memengaruhi weight_decay.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada weight_decay akan memengaruhi normalization.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	normalization
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah dropout menjadi bottleneck.
- Pisahkan biaya setup dari steady-state weight_decay.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Membandingkan eksperimen dengan lebih dari satu variabel berubah.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait dropout.
3. Isolasi	Nonaktifkan sementara weight_decay dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Ablation regularisasi

Bangun artefak kecil yang membuktikan Anda dapat meningkatkan stabilitas serta generalisasi model. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk normalization.
2. Implementasikan baseline memakai dropout dan weight_decay.
3. Tambahkan logging untuk initialization.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan dropout tanpa menggunakan istilah `weight_decay`.
2. Apa `shape`, `dtype`, dan `device` yang diharapkan pada batas `weight_decay`?
3. Kapan `initialization` berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas `normalization`?
5. Bagaimana Anda mereproduksi kegagalan: membandingkan eksperimen dengan lebih dari satu variabel berubah?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Ablation regularisasi?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk `weight_decay`, lalu buat tabel yang membandingkan `normalization`, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
dropout	Peran inti dalam meningkatkan stabilitas serta generalisasi model.
weight_decay	Peran inti dalam meningkatkan stabilitas serta generalisasi model.
initialization	Peran inti dalam meningkatkan stabilitas serta generalisasi model.
normalization	Peran inti dalam meningkatkan stabilitas serta generalisasi model.

Checklist siap pakai

- Verifikasi dropout sebelum eksekusi utama.
- Catat perubahan pada weight_decay dan initialization.
- Ukur normalization dengan baseline yang adil.
- Tambahkan test untuk mencegah: membandingkan eksperimen dengan lebih dari satu variabel berubah.
- Simpan artefak proyek: Ablation regularisasi.

API yang muncul

nn.Sequential, nn.Linear, nn.LayerNorm, nn.GELU, nn.Dropout, nn.init

SATU KALIMAT Bab 16 mengajarkan cara meningkatkan stabilitas serta generalisasi model dengan kontrak eksplisit dan bukti terukur.

BAGIAN 5 - COMPUTER VISION

Bab 17. Convolutional Neural Network

Memahami receptive field dan ekstraksi fitur spasial.

Hasil belajar

- Menjelaskan peran convolution dan hubungannya dengan kernel.
- Menulis notebook Colab untuk memahami receptive field dan ekstraksi fitur spasial.
- Mendiagnosis risiko: dimensi flatten dihitung secara manual dan salah.
- Menghasilkan proyek mini: CNN klasifikasi citra.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: convolution, kernel, pooling, channels.

Parameter	Target
Level	Menengah
Waktu	60-120 menit
Artefak	CNN klasifikasi citra
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 17

Convolutional Neural Network



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah memahami receptive field dan ekstraksi fitur spasial. Alur di atas memperlakukan convolution sebagai input keputusan, kernel sebagai transformasi utama, pooling sebagai state atau mekanisme kontrol, dan channels sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika kernel berubah, bagaimana dampaknya terhadap channels? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk memahami receptive field dan ekstraksi fitur spasial. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch import nn
3 class SmallCNN(nn.Module):
4     def __init__(self, classes=10):
5         super().__init__()
6         self.features = nn.Sequential(nn.Conv2d(3,32,3,padding=1), nn.ReLU(), nn.MaxPool2d(2),
7                                     nn.Conv2d(32,64,3,padding=1), nn.ReLU(),
8                                     nn.AdaptiveAvgPool2d(1))
9         self.head = nn.Linear(64, classes)
10    def forward(self, x): return self.head(self.features(x).flatten(1))
11 print(SmallCNN()(torch.randn(4,3,64,64)).shape)
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi channels dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
SmallCNN	Mewakili convolution; catat input, output, dtype, device, serta state yang berubah.
nn.Module	Mewakili kernel; catat input, output, dtype, device, serta state yang berubah.
nn.Sequential	Mewakili pooling; catat input, output, dtype, device, serta state yang berubah.
nn.Conv2d	Mewakili channels; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Dimensi flatten dihitung secara manual dan salah.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 17

Convolutional Neural Network

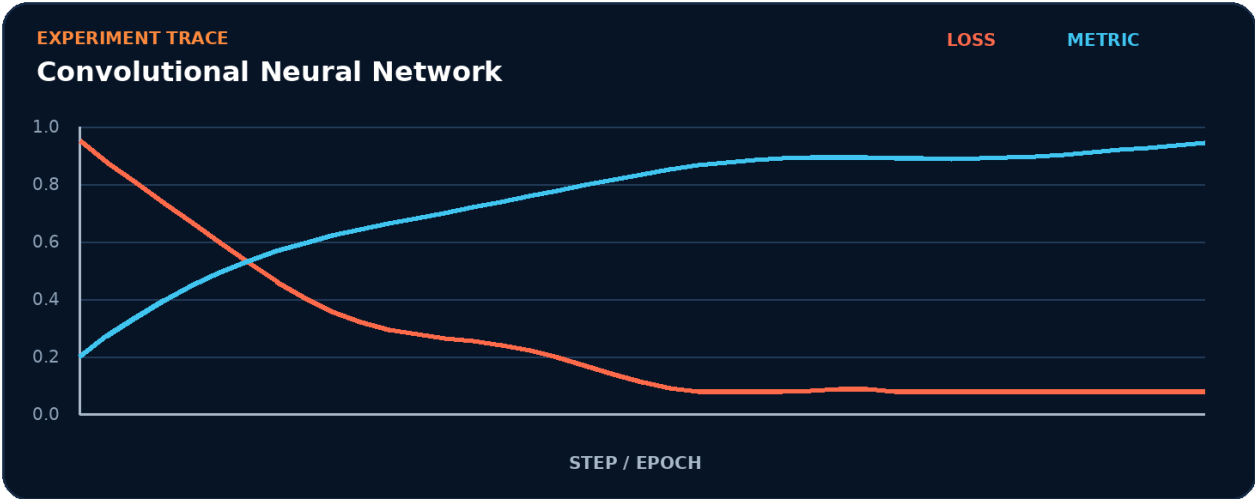


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk convolution.
2. Terapkan transformasi utama: kernel.
3. Kelola state atau kontrol melalui pooling.
4. Ukur hasil akhir memakai channels.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur convolution -> kernel -> pooling -> channels tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk channels.
- 2. Ubah satu nilai yang memengaruhi kernel.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada kernel akan memengaruhi channels.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	channels
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah convolution menjadi bottleneck.
- Pisahkan biaya setup dari steady-state kernel.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Dimensi flatten dihitung secara manual dan salah.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait convolution.
3. Isolasi	Nonaktifkan sementara kernel dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: CNN klasifikasi citra

Bangun artefak kecil yang membuktikan Anda dapat memahami receptive field dan ekstraksi fitur spasial. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk channels.
2. Implementasikan baseline memakai convolution dan kernel.
3. Tambahkan logging untuk pooling.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan convolution tanpa menggunakan istilah kernel.
2. Apa shape, dtype, dan device yang diharapkan pada batas kernel?
3. Kapan pooling berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas channels?
5. Bagaimana Anda mereproduksi kegagalan: dimensi flatten dihitung secara manual dan salah?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek CNN klasifikasi citra?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk kernel, lalu buat tabel yang membandingkan channels, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
convolution	Peran inti dalam memahami receptive field dan ekstraksi fitur spasial.
kernel	Peran inti dalam memahami receptive field dan ekstraksi fitur spasial.
pooling	Peran inti dalam memahami receptive field dan ekstraksi fitur spasial.
channels	Peran inti dalam memahami receptive field dan ekstraksi fitur spasial.

Checklist siap pakai

- Verifikasi convolution sebelum eksekusi utama.
- Catat perubahan pada kernel dan pooling.
- Ukur channels dengan baseline yang adil.
- Tambahkan test untuk mencegah: dimensi flatten dihitung secara manual dan salah.
- Simpan artefak proyek: CNN klasifikasi citra.

API yang muncul

SmallCNN, nn.Module, nn.Sequential, nn.Conv2d, nn.ReLU, nn.MaxPool2d

SATU KALIMAT Bab 17 mengajarkan cara memahami receptive field dan ekstraksi fitur spasial dengan kontrak eksplisit dan bukti terukur.

BAGIAN 5 - COMPUTER VISION

Bab 18. Transfer Learning

Memanfaatkan backbone pralatih secara efisien.

Hasil belajar

- Menjelaskan peran backbone dan hubungannya dengan freeze.
- Menulis notebook Colab untuk memanfaatkan backbone pralatih secara efisien.
- Mendiagnosis risiko: learning rate backbone terlalu besar.
- Menghasilkan proyek mini: Fine-tuning image classifier.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: backbone, freeze, head, fine-tuning.

Parameter	Target
Level	Menengah
Waktu	60-120 menit
Artefak	Fine-tuning image classifier
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 18

Transfer Learning



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah memanfaatkan backbone pralatih secara efisien. Alur di atas memperlakukan backbone sebagai input keputusan, freeze sebagai transformasi utama, head sebagai state atau mekanisme kontrol, dan fine-tuning sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika freeze berubah, bagaimana dampaknya terhadap fine-tuning? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk memanfaatkan backbone prelatih secara efisien. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch import nn
3 from torchvision.models import resnet18, ResNet18_Weights
4 model = resnet18(weights=ResNet18_Weights.DEFAULT)
5 for p in model.parameters(): p.requires_grad = False
6 model.fc = nn.Linear(model.fc.in_features, 5)
7 optimizer = torch.optim.AdamW(model.fc.parameters(), lr=1e-3)
8 print([n for n, p in model.named_parameters() if p.requires_grad])
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi fine-tuning dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
DEFAULT	Mewakili backbone; catat input, output, dtype, device, serta state yang berubah.
False	Mewakili freeze; catat input, output, dtype, device, serta state yang berubah.
nn.Linear	Mewakili head; catat input, output, dtype, device, serta state yang berubah.
torch.optim.AdamW	Mewakili fine-tuning; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Learning rate backbone terlalu besar.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 18

Transfer Learning

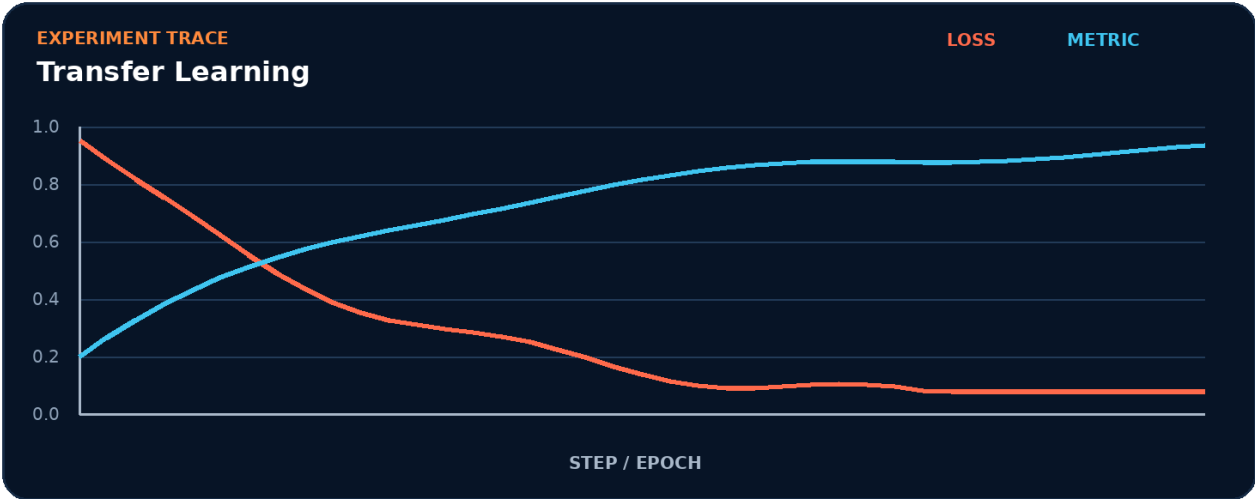


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk backbone.
2. Terapkan transformasi utama: freeze.
3. Kelola state atau kontrol melalui head.
4. Ukur hasil akhir memakai fine-tuning.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur backbone -> freeze -> head -> fine-tuning tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk fine-tuning.
- 2. Ubah satu nilai yang memengaruhi freeze.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada freeze akan memengaruhi fine-tuning.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	fine-tuning
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah backbone menjadi bottleneck.
- Pisahkan biaya setup dari steady-state freeze.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Learning rate backbone terlalu besar.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait backbone.
3. Isolasi	Nonaktifkan sementara freeze dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Fine-tuning image classifier

Bangun artefak kecil yang membuktikan Anda dapat memanfaatkan backbone pralatih secara efisien. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk fine-tuning.
2. Implementasikan baseline memakai backbone dan freeze.
3. Tambahkan logging untuk head.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan backbone tanpa menggunakan istilah freeze.
2. Apa shape, dtype, dan device yang diharapkan pada batas freeze?
3. Kapan head berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas fine-tuning?
5. Bagaimana Anda mereproduksi kegagalan: learning rate backbone terlalu besar?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Fine-tuning image classifier?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk freeze, lalu buat tabel yang membandingkan fine-tuning, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
backbone	Peran inti dalam memanfaatkan backbone pralatih secara efisien.
freeze	Peran inti dalam memanfaatkan backbone pralatih secara efisien.
head	Peran inti dalam memanfaatkan backbone pralatih secara efisien.
fine-tuning	Peran inti dalam memanfaatkan backbone pralatih secara efisien.

Checklist siap pakai

- Verifikasi backbone sebelum eksekusi utama.
- Catat perubahan pada freeze dan head.
- Ukur fine-tuning dengan baseline yang adil.
- Tambahkan test untuk mencegah: learning rate backbone terlalu besar.
- Simpan artefak proyek: Fine-tuning image classifier.

API yang muncul

DEFAULT, False, nn.Linear, torch.optim.AdamW

SATU KALIMAT Bab 18 mengajarkan cara memanfaatkan backbone pralatih secara efisien dengan kontrak eksplisit dan bukti terukur.

BAGIAN 5 - COMPUTER VISION

Bab 19. Semantic Segmentation

Memprediksi kelas pada setiap piksel.

Hasil belajar

- Menjelaskan peran mask dan hubungannya dengan encoder-decoder.
- Menulis notebook Colab untuk memprediksi kelas pada setiap piksel.
- Mendiagnosis risiko: mask di-resize dengan interpolasi bilinear.
- Menghasilkan proyek mini: Segmentasi objek sederhana.

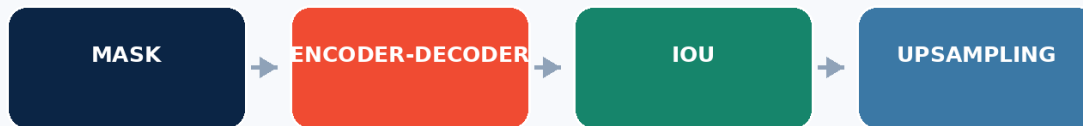
PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: mask, encoder-decoder, IoU, upsampling.

Parameter	Target
Level	Menengah
Waktu	60-120 menit
Artefak	Segmentasi objek sederhana
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 19

Semantic Segmentation



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah memprediksi kelas pada setiap piksel. Alur di atas memperlakukan mask sebagai input keputusan, encoder-decoder sebagai transformasi utama, IoU sebagai state atau mekanisme kontrol, dan upsampling sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika encoder-decoder berubah, bagaimana dampaknya terhadap upsampling? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk memprediksi kelas pada setiap piksel. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch import nn
3 class TinySeg(nn.Module):
4     def __init__(self, classes=3):
5         super().__init__()
6 self.enc=nn.Sequential(nn.Conv2d(3,16,3,padding=1),nn.ReLU(),nn.MaxPool2d(2))
7 self.dec=nn.Sequential(nn.ConvTranspose2d(16,16,2,stride=2),nn.ReLU(),nn.Conv2d(16,classes,1))
8     def forward(self,x): return self.dec(self.enc(x))
9 logits = TinySeg()(torch.randn(2,3,128,128))
10 mask = torch.randint(0,3,(2,128,128)); print(logits.shape, nn.CrossEntropyLoss()(logits,mask))
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi upsampling dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
TinySeg	Mewakili mask; catat input, output, dtype, device, serta state yang berubah.
nn.Module	Mewakili encoder-decoder; catat input, output, dtype, device, serta state yang berubah.
nn.Sequential	Mewakili IoU; catat input, output, dtype, device, serta state yang berubah.
nn.Conv2d	Mewakili upsampling; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Mask di-resize dengan interpolasi bilinear.

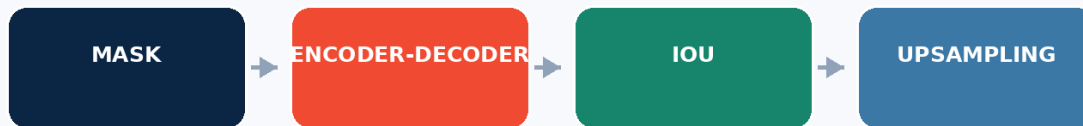
Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 19

Semantic Segmentation

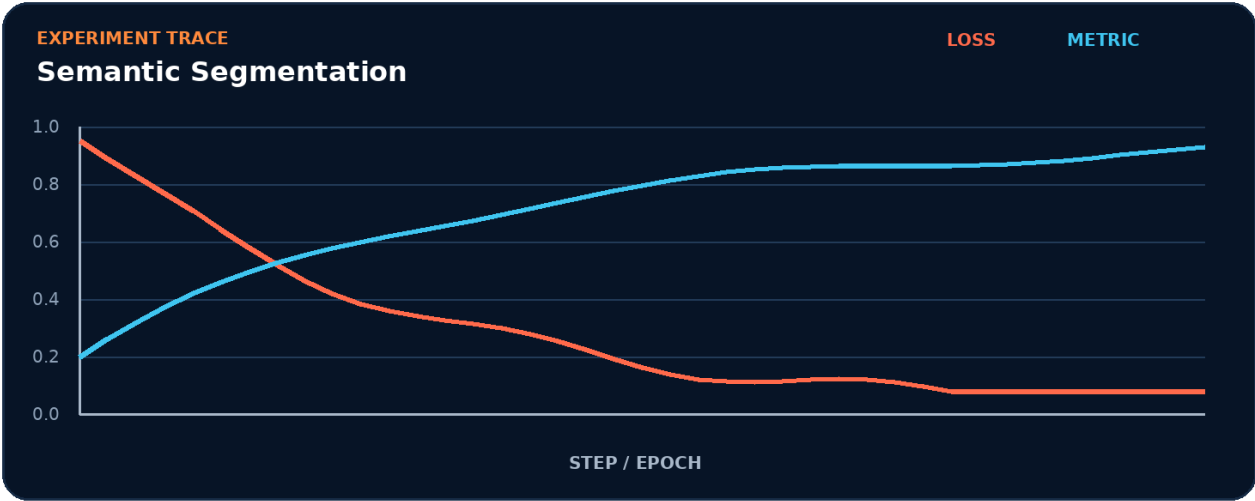


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk mask.
2. Terapkan transformasi utama: encoder-decoder.
3. Kelola state atau kontrol melalui IoU.
4. Ukur hasil akhir memakai upsampling.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur mask -> encoder-decoder -> IoU -> upsampling tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk upsampling.
- 2. Ubah satu nilai yang memengaruhi encoder-decoder.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada encoder-decoder akan memengaruhi upsampling.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	upsampling
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah mask menjadi bottleneck.
- Pisahkan biaya setup dari steady-state encoder-decoder.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Mask di-resize dengan interpolasi bilinear.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait mask.
3. Isolasi	Nonaktifkan sementara encoder-decoder dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Segmentasi objek sederhana

Bangun artefak kecil yang membuktikan Anda dapat memprediksi kelas pada setiap piksel. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk upsampling.
2. Implementasikan baseline memakai mask dan encoder-decoder.
3. Tambahkan logging untuk IoU.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan mask tanpa menggunakan istilah encoder-decoder.
2. Apa shape, dtype, dan device yang diharapkan pada batas encoder-decoder?
3. Kapan IoU berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas upsampling?
5. Bagaimana Anda mereproduksi kegagalan: mask di-resize dengan interpolasi bilinear?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Segmentasi objek sederhana?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk encoder-decoder, lalu buat tabel yang membandingkan upsampling, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
mask	Peran inti dalam memprediksi kelas pada setiap piksel.
encoder-decoder	Peran inti dalam memprediksi kelas pada setiap piksel.
IoU	Peran inti dalam memprediksi kelas pada setiap piksel.
upsampling	Peran inti dalam memprediksi kelas pada setiap piksel.

Checklist siap pakai

- Verifikasi mask sebelum eksekusi utama.
- Catat perubahan pada encoder-decoder dan IoU.
- Ukur upsampling dengan baseline yang adil.
- Tambahkan test untuk mencegah: mask di-resize dengan interpolasi bilinear.
- Simpan artefak proyek: Segmentasi objek sederhana.

API yang muncul

TinySeg, nn.Module, nn.Sequential, nn.Conv2d, nn.ReLU, nn.MaxPool2d

SATU KALIMAT Bab 19 mengajarkan cara memprediksi kelas pada setiap piksel dengan kontrak eksplisit dan bukti terukur.

BAGIAN 5 - COMPUTER VISION

Bab 20. Detection dan Vision Transformer

Membandingkan paradigma region dan token gambar.

Hasil belajar

- Menjelaskan peran bounding_box dan hubungannya dengan NMS.
- Menulis notebook Colab untuk membandingkan paradigma region dan token gambar.
- Mendiagnosis risiko: koordinat bounding box memakai konvensi berbeda.
- Menghasilkan proyek mini: Eksperimen patch embedding.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: bounding_box, NMS, patch, attention.

Parameter	Target
Level	Menengah
Waktu	60-120 menit
Artefak	Eksperimen patch embedding
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 20

Detection dan Vision Transformer



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah membandingkan paradigma region dan token gambar. Alur di atas memperlakukan bounding_box sebagai input keputusan, NMS sebagai transformasi utama, patch sebagai state atau mekanisme kontrol, dan attention sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika NMS berubah, bagaimana dampaknya terhadap attention? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk membandingkan paradigma region dan token gambar. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch import nn
3 class PatchEmbed(nn.Module):
4     def __init__(self, patch=16, dim=192):
5         super().__init__(); self.proj=nn.Conv2d(3,dim,kernel_size=patch,stroke=patch)
6     def forward(self,x): return self.proj(x).flatten(2).transpose(1,2)
7 tokens = PatchEmbed()(torch.randn(2,3,224,224))
8 attn = nn.MultiheadAttention(192, 6, batch_first=True)
9 out, weights = attn(tokens,tokens,tokens,need_weights=True)
10 print(tokens.shape, out.shape, weights.shape)
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi attention dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
PatchEmbed	Mewakili bounding_box; catat input, output, dtype, device, serta state yang berubah.
nn.Module	Mewakili NMS; catat input, output, dtype, device, serta state yang berubah.
nn.Conv2d	Mewakili patch; catat input, output, dtype, device, serta state yang berubah.
torch.randn	Mewakili attention; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Koordinat bounding box memakai konvensi berbeda.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 20

Detection dan Vision Transformer

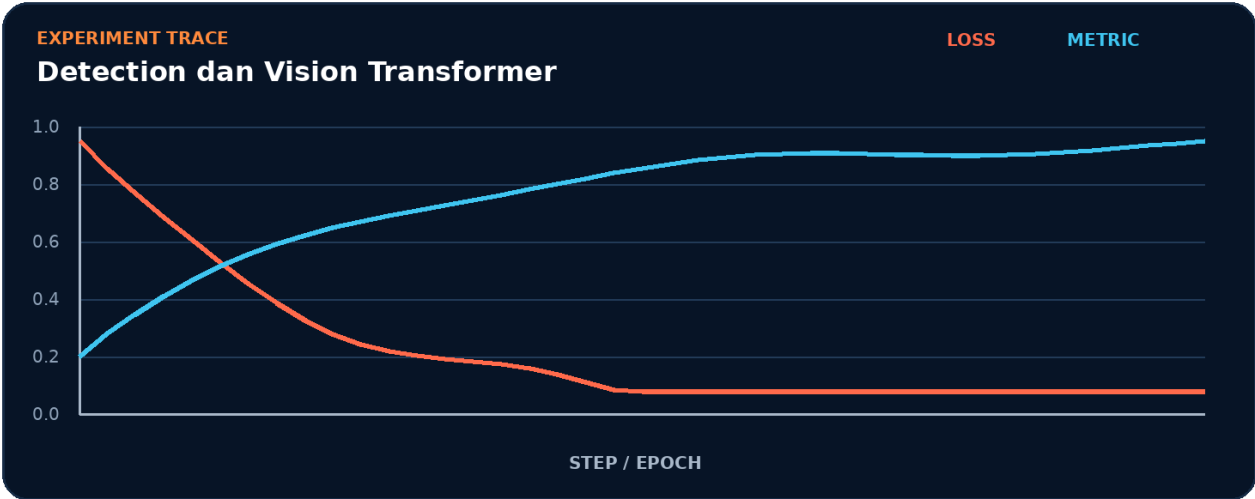


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk bounding_box.
2. Terapkan transformasi utama: NMS.
3. Kelola state atau kontrol melalui patch.
4. Ukur hasil akhir memakai attention.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur bounding_box -> NMS -> patch -> attention tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk attention.
- 2. Ubah satu nilai yang memengaruhi NMS.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada NMS akan memengaruhi attention.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	attention
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah bounding_box menjadi bottleneck.
- Pisahkan biaya setup dari steady-state NMS.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Koordinat bounding box memakai konvensi berbeda.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait bounding_box.
3. Isolasi	Nonaktifkan sementara NMS dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```

1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()

```

Mini Project: Eksperimen patch embedding

Bangun artefak kecil yang membuktikan Anda dapat membandingkan paradigma region dan token gambar. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

- 1. Definisikan input, target, dan batas keberhasilan untuk attention.
- 2. Implementasikan baseline memakai bounding_box dan NMS.
- 3. Tambahkan logging untuk patch.
- 4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
- 5. Simpan config, metric, diagram, dan checkpoint bila relevan.
- 6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan `bounding_box` tanpa menggunakan istilah NMS.
2. Apa `shape`, `dtype`, dan `device` yang diharapkan pada batas NMS?
3. Kapan patch berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas `attention`?
5. Bagaimana Anda mereproduksi kegagalan: koordinat bounding box memakai konvensi berbeda?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Eksperimen patch embedding?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk NMS, lalu buat tabel yang membandingkan `attention`, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
bounding_box	Peran inti dalam membandingkan paradigma region dan token gambar.
NMS	Peran inti dalam membandingkan paradigma region dan token gambar.
patch	Peran inti dalam membandingkan paradigma region dan token gambar.
attention	Peran inti dalam membandingkan paradigma region dan token gambar.

Checklist siap pakai

- Verifikasi bounding_box sebelum eksekusi utama.
- Catat perubahan pada NMS dan patch.
- Ukur attention dengan baseline yang adil.
- Tambahkan test untuk mencegah: koordinat bounding box memakai konvensi berbeda.
- Simpan artefak proyek: Eksperimen patch embedding.

API yang muncul

PatchEmbed, nn.Module, nn.Conv2d, torch.randn, nn.MultiheadAttention, True

SATU KALIMAT Bab 20 mengajarkan cara membandingkan paradigma region dan token gambar dengan kontrak eksplisit dan bukti terukur.

BAGIAN 6 - NLP DAN TRANSFORMER

Bab 21. Representasi Teks dan Embedding

Mengubah token menjadi representasi numerik yang dapat dilatih.

Hasil belajar

- Menjelaskan peran token dan hubungannya dengan vocabulary.
- Menulis notebook Colab untuk mengubah token menjadi representasi numerik yang dapat dilatih.
- Mendiagnosis risiko: padding ikut dihitung dalam pooling.
- Menghasilkan proyek mini: Klasifier sentimen sederhana.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: token, vocabulary, embedding, padding.

Parameter	Target
Level	Menengah
Waktu	60-120 menit
Artefak	Klasifier sentimen sederhana
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 21

Representasi Teks dan Embedding



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah mengubah token menjadi representasi numerik yang dapat dilatih. Alur di atas memperlakukan token sebagai input keputusan, vocabulary sebagai transformasi utama, embedding sebagai state atau mekanisme kontrol, dan padding sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika vocabulary berubah, bagaimana dampaknya terhadap padding? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk mengubah token menjadi representasi numerik yang dapat dilatih. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch import nn
3 tokens = torch.tensor([[4,8,2,0,0],[7,3,9,5,0]])
4 embedding = nn.Embedding(20, 16, padding_idx=0)
5 x = embedding(tokens)
6 mask = tokens.ne(0).unsqueeze(-1)
7 pooled = (x * mask).sum(1) / mask.sum(1).clamp_min(1)
8 classifier = nn.Linear(16, 2)
9 print(classifier(pooled).shape)
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi padding dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.tensor	Mewakili token; catat input, output, dtype, device, serta state yang berubah.
nn.Embedding	Mewakili vocabulary; catat input, output, dtype, device, serta state yang berubah.
nn.Linear	Mewakili embedding; catat input, output, dtype, device, serta state yang berubah.
torch.tensor	Mewakili padding; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Padding ikut dihitung dalam pooling.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 21

Representasi Teks dan Embedding

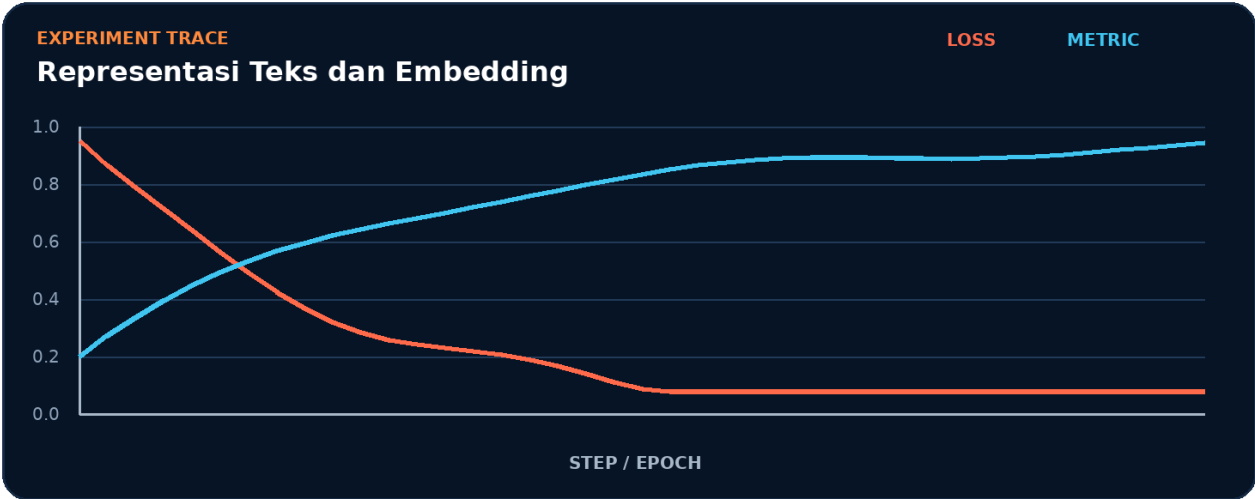


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk token.
2. Terapkan transformasi utama: vocabulary.
3. Kelola state atau kontrol melalui embedding.
4. Ukur hasil akhir memakai padding.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur token -> vocabulary -> embedding -> padding tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk padding.
- 2. Ubah satu nilai yang memengaruhi vocabulary.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada vocabulary akan memengaruhi padding.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	padding
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah token menjadi bottleneck.
- Pisahkan biaya setup dari steady-state vocabulary.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Padding ikut dihitung dalam pooling.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait token.
3. Isolasi	Nonaktifkan sementara vocabulary dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

```
COLAB - DEBUG CELL
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Klasifier sentimen sederhana

Bangun artefak kecil yang membuktikan Anda dapat mengubah token menjadi representasi numerik yang dapat dilatih. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk padding.
2. Implementasikan baseline memakai token dan vocabulary.
3. Tambahkan logging untuk embedding.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan token tanpa menggunakan istilah vocabulary.
2. Apa shape, dtype, dan device yang diharapkan pada batas vocabulary?
3. Kapan embedding berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas padding?
5. Bagaimana Anda mereproduksi kegagalan: padding ikut dihitung dalam pooling?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Klasifier sentimen sederhana?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk vocabulary, lalu buat tabel yang membandingkan padding, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
token	Peran inti dalam mengubah token menjadi representasi numerik yang dapat dilatih.
vocabulary	Peran inti dalam mengubah token menjadi representasi numerik yang dapat dilatih.
embedding	Peran inti dalam mengubah token menjadi representasi numerik yang dapat dilatih.
padding	Peran inti dalam mengubah token menjadi representasi numerik yang dapat dilatih.

Checklist siap pakai

- Verifikasi token sebelum eksekusi utama.
- Catat perubahan pada vocabulary dan embedding.
- Ukur padding dengan baseline yang adil.
- Tambahkan test untuk mencegah: padding ikut dihitung dalam pooling.
- Simpan artefak proyek: Klasifier sentimen sederhana.

API yang muncul

torch.tensor, nn.Embedding, nn.Linear

SATU KALIMAT Bab 21 mengajarkan cara mengubah token menjadi representasi numerik yang dapat dilatih dengan kontrak eksplisit dan bukti terukur.

BAGIAN 6 - NLP DAN TRANSFORMER

Bab 22. RNN, LSTM, dan GRU

Memodelkan urutan dengan hidden state.

Hasil belajar

- Menjelaskan peran sequence dan hubungannya dengan hidden_state.
- Menulis notebook Colab untuk memodelkan urutan dengan hidden state.
- Mendiagnosis risiko: dimensi batch_first tidak konsisten.
- Menghasilkan proyek mini: Prediksi deret waktu.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: sequence, hidden_state, LSTM, GRU.

Parameter	Target
Level	Menengah
Waktu	60-120 menit
Artefak	Prediksi deret waktu
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 22

RNN, LSTM, dan GRU



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah memodelkan urutan dengan hidden state. Alur di atas memperlakukan sequence sebagai input keputusan, hidden_state sebagai transformasi utama, LSTM sebagai state atau mekanisme kontrol, dan GRU sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika hidden_state berubah, bagaimana dampaknya terhadap GRU? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk memodelkan urutan dengan hidden state. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch import nn
3 x = torch.randn(32, 50, 8)
4 lstm = nn.LSTM(input_size=8, hidden_size=64, num_layers=2,
5               batch_first=True, dropout=0.1, bidirectional=True)
6 out, (h, c) = lstm(x)
7 head = nn.Linear(128, 1)
8 pred = head(out[:, -1])
9 print(out.shape, h.shape, pred.shape)
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi GRU dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.randn	Mewakili sequence; catat input, output, dtype, device, serta state yang berubah.
nn.LSTM	Mewakili hidden_state; catat input, output, dtype, device, serta state yang berubah.
True	Mewakili LSTM; catat input, output, dtype, device, serta state yang berubah.
nn.Linear	Mewakili GRU; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Dimensi batch_first tidak konsisten.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 22

RNN, LSTM, dan GRU

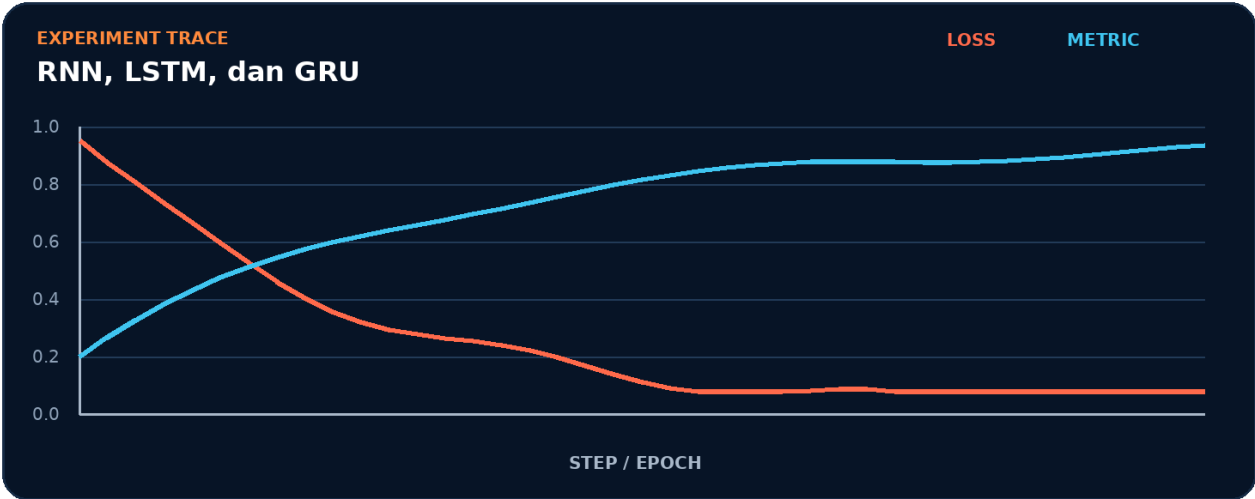


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk sequence.
2. Terapkan transformasi utama: hidden_state.
3. Kelola state atau kontrol melalui LSTM.
4. Ukur hasil akhir memakai GRU.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur sequence -> hidden_state -> LSTM -> GRU tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk GRU.
- 2. Ubah satu nilai yang memengaruhi hidden_state.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada hidden_state akan memengaruhi GRU.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	GRU
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah sequence menjadi bottleneck.
- Pisahkan biaya setup dari steady-state hidden_state.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Dimensi batch_first tidak konsisten.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait sequence.
3. Isolasi	Nonaktifkan sementara hidden_state dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Prediksi deret waktu

Bangun artefak kecil yang membuktikan Anda dapat memodelkan urutan dengan hidden state. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk GRU.
2. Implementasikan baseline memakai sequence dan hidden_state.
3. Tambahkan logging untuk LSTM.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan sequence tanpa menggunakan istilah `hidden_state`.
2. Apa shape, dtype, dan device yang diharapkan pada batas `hidden_state`?
3. Kapan LSTM berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas GRU?
5. Bagaimana Anda mereproduksi kegagalan: dimensi `batch_first` tidak konsisten?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Prediksi deret waktu?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk `hidden_state`, lalu buat tabel yang membandingkan GRU, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
sequence	Peran inti dalam memodelkan urutan dengan hidden state.
hidden_state	Peran inti dalam memodelkan urutan dengan hidden state.
LSTM	Peran inti dalam memodelkan urutan dengan hidden state.
GRU	Peran inti dalam memodelkan urutan dengan hidden state.

Checklist siap pakai

- Verifikasi sequence sebelum eksekusi utama.
- Catat perubahan pada hidden_state dan LSTM.
- Ukur GRU dengan baseline yang adil.
- Tambahkan test untuk mencegah: dimensi batch_first tidak konsisten.
- Simpan artefak proyek: Prediksi deret waktu.

API yang muncul

torch.randn, nn.LSTM, True, nn.Linear

SATU KALIMAT Bab 22 mengajarkan cara memodelkan urutan dengan hidden state dengan kontrak eksplisit dan bukti terukur.

BAGIAN 6 - NLP DAN TRANSFORMER

Bab 23. Attention dan Multi-Head Attention

Menghitung dependensi global antar token.

Hasil belajar

- Menjelaskan peran query dan hubungannya dengan key.
- Menulis notebook Colab untuk menghitung dependensi global antar token.
- Mendiagnosis risiko: mask memiliki arah atau dtype yang salah.
- Menghasilkan proyek mini: Visualisasi attention map.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: query, key, value, mask.

Parameter	Target
Level	Menengah
Waktu	60-120 menit
Artefak	Visualisasi attention map
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 23

Attention dan Multi-Head Attention



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah menghitung dependensi global antar token. Alur di atas memperlakukan query sebagai input keputusan, key sebagai transformasi utama, value sebagai state atau mekanisme kontrol, dan mask sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika key berubah, bagaimana dampaknya terhadap mask? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk menghitung dependensi global antar token. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch import nn
3 x = torch.randn(4, 12, 128)
4 mask = torch.triu(torch.ones(12,12,dtype=torch.bool), diagonal=1)
5 attn = nn.MultiheadAttention(128, 8, batch_first=True)
6 y, weights = attn(x, x, x, attn_mask=mask, need_weights=True)
7 print(y.shape, weights.shape)
8 print('row sum:', weights[0,0].sum().item())
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi mask dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.randn	Mewakili query; catat input, output, dtype, device, serta state yang berubah.
torch.triu	Mewakili key; catat input, output, dtype, device, serta state yang berubah.
torch.ones	Mewakili value; catat input, output, dtype, device, serta state yang berubah.
torch.bool	Mewakili mask; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Mask memiliki arah atau dtype yang salah.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 23

Attention dan Multi-Head Attention

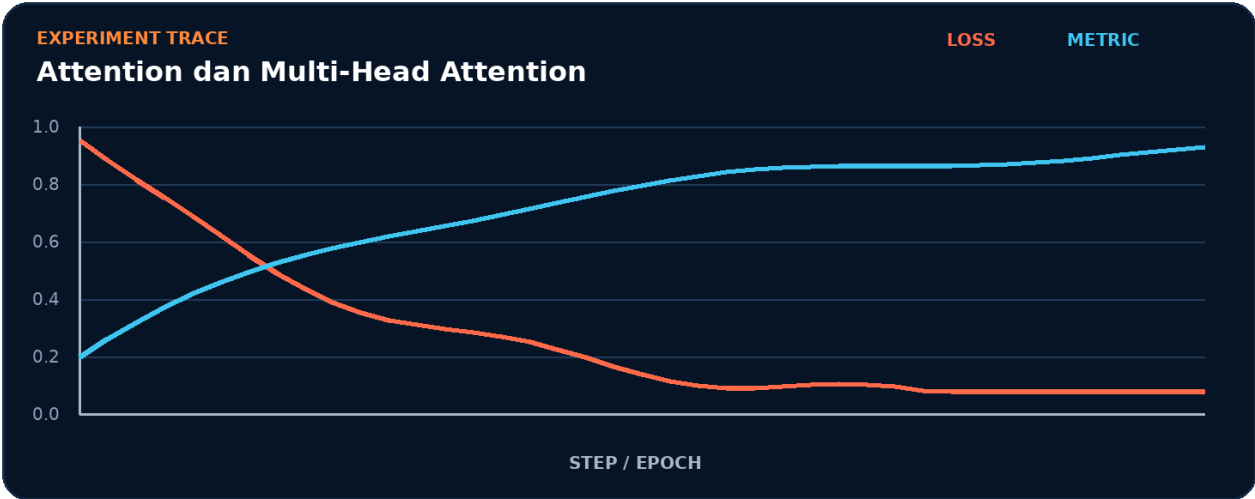


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk query.
2. Terapkan transformasi utama: key.
3. Kelola state atau kontrol melalui value.
4. Ukur hasil akhir memakai mask.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur query -> key -> value -> mask tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk mask.
- 2. Ubah satu nilai yang memengaruhi key.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada key akan memengaruhi mask.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	mask
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah query menjadi bottleneck.
- Pisahkan biaya setup dari steady-state key.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Mask memiliki arah atau dtype yang salah.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait query.
3. Isolasi	Nonaktifkan sementara key dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

```
COLAB - DEBUG CELL
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Visualisasi attention map

Bangun artefak kecil yang membuktikan Anda dapat menghitung dependensi global antar token. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk mask.
2. Implementasikan baseline memakai query dan key.
3. Tambahkan logging untuk value.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan query tanpa menggunakan istilah key.
2. Apa shape, dtype, dan device yang diharapkan pada batas key?
3. Kapan value berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas mask?
5. Bagaimana Anda mereproduksi kegagalan: mask memiliki arah atau dtype yang salah?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Visualisasi attention map?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk key, lalu buat tabel yang membandingkan mask, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
query	Peran inti dalam menghitung dependensi global antar token.
key	Peran inti dalam menghitung dependensi global antar token.
value	Peran inti dalam menghitung dependensi global antar token.
mask	Peran inti dalam menghitung dependensi global antar token.

Checklist siap pakai

- Verifikasi query sebelum eksekusi utama.
- Catat perubahan pada key dan value.
- Ukur mask dengan baseline yang adil.
- Tambahkan test untuk mencegah: mask memiliki arah atau dtype yang salah.
- Simpan artefak proyek: Visualisasi attention map.

API yang muncul

torch.randn, torch.triu, torch.ones, torch.bool, nn.MultiheadAttention, True

SATU KALIMAT Bab 23 mengajarkan cara menghitung dependensi global antar token dengan kontrak eksplisit dan bukti terukur.

BAGIAN 6 - NLP DAN TRANSFORMER

Bab 24. Transformer Encoder

Menyusun self-attention, residual, dan feed-forward.

Hasil belajar

- Menjelaskan peran encoder dan hubungannya dengan residual.
- Menulis notebook Colab untuk menyusun self-attention, residual, dan feed-forward.
- Mendiagnosis risiko: urutan dimensi sequence dan batch tertukar.
- Menghasilkan proyek mini: Encoder untuk klasifikasi teks.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: encoder, residual, layer_norm, positional_encoding.

Parameter	Target
Level	Menengah
Waktu	60-120 menit
Artefak	Encoder untuk klasifikasi teks
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 24

Transformer Encoder



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah menyusun self-attention, residual, dan feed-forward. Alur di atas memperlakukan encoder sebagai input keputusan, residual sebagai transformasi utama, layer_norm sebagai state atau mekanisme kontrol, dan positional_encoding sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika residual berubah, bagaimana dampaknya terhadap positional_encoding? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk menyusun self-attention, residual, dan feed-forward. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch import nn
3 layer = nn.TransformerEncoderLayer(d_model=256, nhead=8,
4                                     dim_feedforward=1024, dropout=0.1,
5                                     batch_first=True, norm_first=True)
6 encoder = nn.TransformerEncoder(layer, num_layers=4)
7 x = torch.randn(8, 128, 256)
8 padding_mask = torch.zeros(8, 128, dtype=torch.bool)
9 y = encoder(x, src_key_padding_mask=padding_mask)
10 print(y.shape)
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi positional_encoding dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
nn.TransformerEncoderLayer	Mewakili encoder; catat input, output, dtype, device, serta state yang berubah.
True	Mewakili residual; catat input, output, dtype, device, serta state yang berubah.
nn.TransformerEncoder	Mewakili layer_norm; catat input, output, dtype, device, serta state yang berubah.
torch.randn	Mewakili positional_encoding; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Urutan dimensi sequence dan batch tertukar.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 24

Transformer Encoder

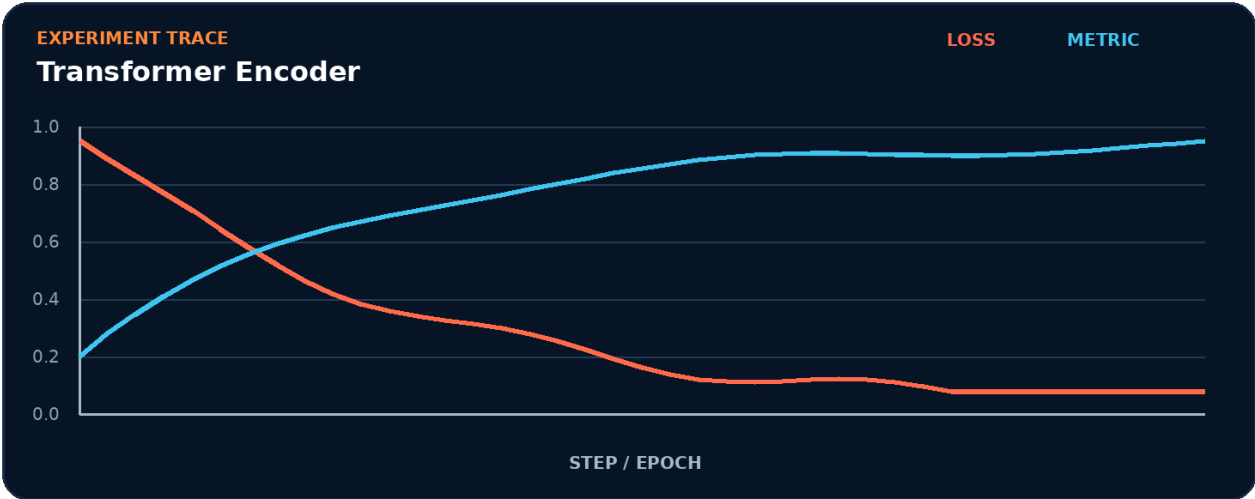


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk encoder.
2. Terapkan transformasi utama: residual.
3. Kelola state atau kontrol melalui layer_norm.
4. Ukur hasil akhir memakai positional_encoding.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur encoder -> residual -> layer_norm -> positional_encoding tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk positional_encoding.
- 2. Ubah satu nilai yang memengaruhi residual.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada residual akan memengaruhi positional_encoding.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	positional_encoding
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah encoder menjadi bottleneck.
- Pisahkan biaya setup dari steady-state residual.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Urutan dimensi sequence dan batch tertukar.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait encoder.
3. Isolasi	Nonaktifkan sementara residual dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Encoder untuk klasifikasi teks

Bangun artefak kecil yang membuktikan Anda dapat menyusun self-attention, residual, dan feed-forward. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

- 1. Definisikan input, target, dan batas keberhasilan untuk positional_encoding.
- 2. Implementasikan baseline memakai encoder dan residual.
- 3. Tambahkan logging untuk layer_norm.
- 4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
- 5. Simpan config, metric, diagram, dan checkpoint bila relevan.
- 6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan encoder tanpa menggunakan istilah residual.
2. Apa shape, dtype, dan device yang diharapkan pada batas residual?
3. Kapan layer_norm berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas positional_encoding?
5. Bagaimana Anda mereproduksi kegagalan: urutan dimensi sequence dan batch tertukar?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Encoder untuk klasifikasi teks?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk residual, lalu buat tabel yang membandingkan positional_encoding, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
encoder	Peran inti dalam menyusun self-attention, residual, dan feed-forward.
residual	Peran inti dalam menyusun self-attention, residual, dan feed-forward.
layer_norm	Peran inti dalam menyusun self-attention, residual, dan feed-forward.
positional_encoding	Peran inti dalam menyusun self-attention, residual, dan feed-forward.

Checklist siap pakai

- Verifikasi encoder sebelum eksekusi utama.
- Catat perubahan pada residual dan layer_norm.
- Ukur positional_encoding dengan baseline yang adil.
- Tambahkan test untuk mencegah: urutan dimensi sequence dan batch tertukar.
- Simpan artefak proyek: Encoder untuk klasifikasi teks.

API yang muncul

nn.TransformerEncoderLayer, True, nn.TransformerEncoder, torch.randn, torch.zeros, torch.bool

SATU KALIMAT Bab 24 mengajarkan cara menyusun self-attention, residual, dan feed-forward dengan kontrak eksplisit dan bukti terukur.

BAGIAN 7 - TRAINING LANJUTAN

Bab 25. Automatic Mixed Precision

Mempercepat training gpu dengan presisi campuran.

Hasil belajar

- Menjelaskan peran autocast dan hubungannya dengan GradScaler.
- Menulis notebook Colab untuk mempercepat training GPU dengan presisi campuran.
- Mendiagnosis risiko: menskalakan loss tetapi lupa scaler.step.
- Menghasilkan proyek mini: Benchmark AMP.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: autocast, GradScaler, fp16, bf16.

Parameter	Target
Level	Menengah
Waktu	60-120 menit
Artefak	Benchmark AMP
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 25

Automatic Mixed Precision



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah mempercepat training GPU dengan presisi campuran. Alur di atas memperlakukan autocast sebagai input keputusan, GradScaler sebagai transformasi utama, fp16 sebagai state atau mekanisme kontrol, dan bf16 sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika GradScaler berubah, bagaimana dampaknya terhadap bf16? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk mempercepat training GPU dengan presisi campuran. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 scaler = torch.amp.GradScaler('cuda', enabled=torch.cuda.is_available())
2 for x, y in loader:
3     x, y = x.to(device), y.to(device)
4     optimizer.zero_grad(set_to_none=True)
5     with torch.amp.autocast('cuda', enabled=torch.cuda.is_available()):
6         logits = model(x); loss = torch.nn.functional.cross_entropy(logits, y)
7     scaler.scale(loss).backward()
8     scaler.step(optimizer); scaler.update()
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi bf16 dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.amp.GradScaler	Mewakili autocast; catat input, output, dtype, device, serta state yang berubah.
torch.cuda.is_available	Mewakili GradScaler; catat input, output, dtype, device, serta state yang berubah.
True	Mewakili fp16; catat input, output, dtype, device, serta state yang berubah.
torch.amp.autocast	Mewakili bf16; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Menskalakan loss tetapi lupa scaler.step.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 25

Automatic Mixed Precision

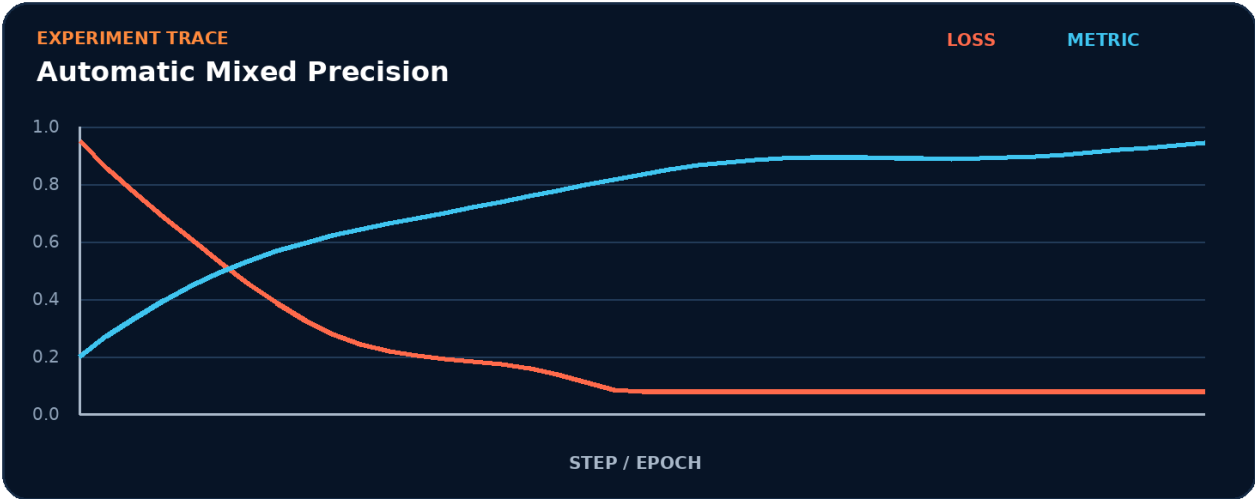


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk autocast.
2. Terapkan transformasi utama: GradScaler.
3. Kelola state atau kontrol melalui fp16.
4. Ukur hasil akhir memakai bf16.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur autocast -> GradScaler -> fp16 -> bf16 tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk bf16.
- 2. Ubah satu nilai yang memengaruhi GradScaler.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada GradScaler akan memengaruhi bf16.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	bf16
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah autocast menjadi bottleneck.
- Pisahkan biaya setup dari steady-state GradScaler.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Menskalakan loss tetapi lupa scaler.step.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait autocast.
3. Isolasi	Nonaktifkan sementara GradScaler dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Benchmark AMP

Bangun artefak kecil yang membuktikan Anda dapat mempercepat training GPU dengan presisi campuran. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk bf16.
2. Implementasikan baseline memakai autocast dan GradScaler.
3. Tambahkan logging untuk fp16.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan autocast tanpa menggunakan istilah GradScaler.
2. Apa shape, dtype, dan device yang diharapkan pada batas GradScaler?
3. Kapan fp16 berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas bf16?
5. Bagaimana Anda mereproduksi kegagalan: menskalakan loss tetapi lupa scaler.step?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Benchmark AMP?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk GradScaler, lalu buat tabel yang membandingkan bf16, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
autocast	Peran inti dalam mempercepat training GPU dengan presisi campuran.
GradScaler	Peran inti dalam mempercepat training GPU dengan presisi campuran.
fp16	Peran inti dalam mempercepat training GPU dengan presisi campuran.
bf16	Peran inti dalam mempercepat training GPU dengan presisi campuran.

Checklist siap pakai

- Verifikasi autocast sebelum eksekusi utama.
- Catat perubahan pada GradScaler dan fp16.
- Ukur bf16 dengan baseline yang adil.
- Tambahkan test untuk mencegah: menskalakan loss tetapi lupa scaler.step.
- Simpan artefak proyek: Benchmark AMP.

API yang muncul

torch.amp.GradScaler, torch.cuda.is_available, True, torch.amp.autocast, torch.nn.functional.cross_entropy

SATU KALIMAT Bab 25 mengajarkan cara mempercepat training GPU dengan presisi campuran dengan kontrak eksplisit dan bukti terukur.

BAGIAN 7 - TRAINING LANJUTAN

Bab 26. Scheduler, Gradient Accumulation, dan Clipping

Mengontrol dinamika update pada batch efektif besar.

Hasil belajar

- Menjelaskan peran scheduler dan hubungannya dengan accumulation.
- Menulis notebook Colab untuk mengontrol dinamika update pada batch efektif besar.
- Mendiagnosis risiko: scheduler dijalankan pada frekuensi yang salah.
- Menghasilkan proyek mini: Trainer batch efektif.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: scheduler, accumulation, clip_grad, warmup.

Parameter	Target
Level	Menengah
Waktu	60-120 menit
Artefak	Trainer batch efektif
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 26

Scheduler, Gradient Accumulation, dan Clipping



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah mengontrol dinamika update pada batch efektif besar. Alur di atas memperlakukan scheduler sebagai input keputusan, accumulation sebagai transformasi utama, clip_grad sebagai state atau mekanisme kontrol, dan warmup sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika accumulation berubah, bagaimana dampaknya terhadap warmup? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk mengontrol dinamika update pada batch efektif besar. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 accum_steps = 4
2 optimizer.zero_grad(set_to_none=True)
3 for step, (x, y) in enumerate(loader):
4     loss = criterion(model(x.to(device)), y.to(device)) / accum_steps
5     loss.backward()
6     if (step + 1) % accum_steps == 0:
7         torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
8         optimizer.step(); scheduler.step()
9         optimizer.zero_grad(set_to_none=True)
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi warmup dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
True	Mewakili scheduler; catat input, output, dtype, device, serta state yang berubah.
torch.nn.utils.clip_grad_norm_	Mewakili accumulation; catat input, output, dtype, device, serta state yang berubah.
True	Mewakili clip_grad; catat input, output, dtype, device, serta state yang berubah.
torch.nn.utils.clip_grad_norm_	Mewakili warmup; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Scheduler dijalankan pada frekuensi yang salah.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 26

Scheduler, Gradient Accumulation, dan Clipping

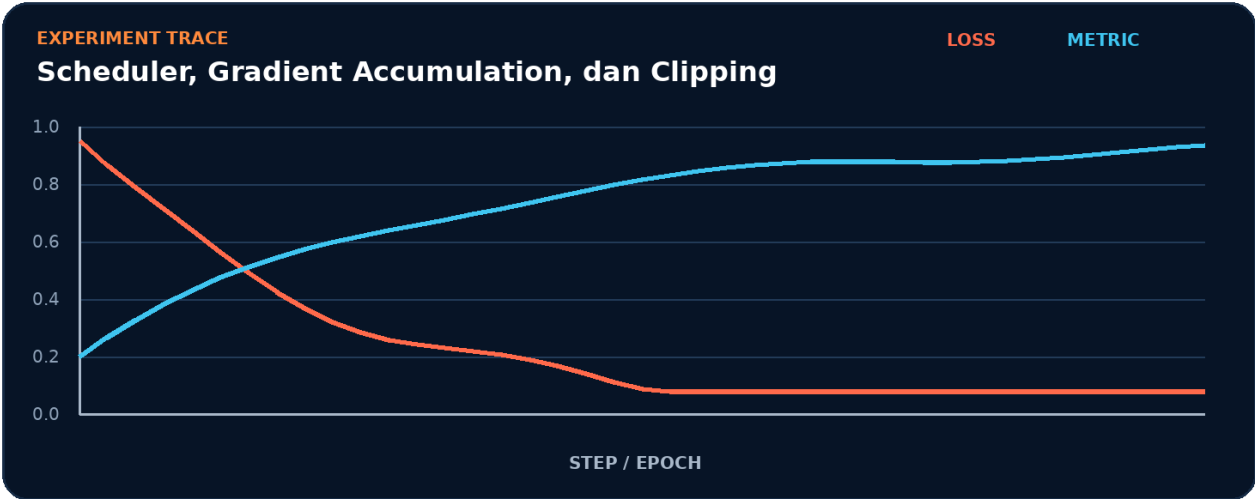


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk scheduler.
2. Terapkan transformasi utama: accumulation.
3. Kelola state atau kontrol melalui clip_grad.
4. Ukur hasil akhir memakai warmup.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur scheduler -> accumulation -> clip_grad -> warmup tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk warmup.
- 2. Ubah satu nilai yang memengaruhi accumulation.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada accumulation akan memengaruhi warmup.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	warmup
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah scheduler menjadi bottleneck.
- Pisahkan biaya setup dari steady-state accumulation.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Scheduler dijalankan pada frekuensi yang salah.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait scheduler.
3. Isolasi	Nonaktifkan sementara accumulation dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Trainer batch efektif

Bangun artefak kecil yang membuktikan Anda dapat mengontrol dinamika update pada batch efektif besar. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk warmup.
2. Implementasikan baseline memakai scheduler dan accumulation.
3. Tambahkan logging untuk clip_grad.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan scheduler tanpa menggunakan istilah accumulation.
2. Apa shape, dtype, dan device yang diharapkan pada batas accumulation?
3. Kapan clip_grad berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas warmup?
5. Bagaimana Anda mereproduksi kegagalan: scheduler dijalankan pada frekuensi yang salah?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Trainer batch efektif?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk accumulation, lalu buat tabel yang membandingkan warmup, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
scheduler	Peran inti dalam mengontrol dinamika update pada batch efektif besar.
accumulation	Peran inti dalam mengontrol dinamika update pada batch efektif besar.
clip_grad	Peran inti dalam mengontrol dinamika update pada batch efektif besar.
warmup	Peran inti dalam mengontrol dinamika update pada batch efektif besar.

Checklist siap pakai

- Verifikasi scheduler sebelum eksekusi utama.
- Catat perubahan pada accumulation dan clip_grad.
- Ukur warmup dengan baseline yang adil.
- Tambahkan test untuk mencegah: scheduler dijalankan pada frekuensi yang salah.
- Simpan artefak proyek: Trainer batch efektif.

API yang muncul

True, torch.nn.utils.clip_grad_norm_

SATU KALIMAT Bab 26 mengajarkan cara mengontrol dinamika update pada batch efektif besar dengan kontrak eksplisit dan bukti terukur.

BAGIAN 7 - TRAINING LANJUTAN

Bab 27. Checkpoint, Resume, dan Early Stopping

Memulihkan eksperimen tanpa kehilangan state penting.

Hasil belajar

- Menjelaskan peran checkpoint dan hubungannya dengan optimizer_state.
- Menulis notebook Colab untuk memulihkan eksperimen tanpa kehilangan state penting.
- Mendiagnosis risiko: hanya menyimpan bobot model tanpa optimizer.
- Menghasilkan proyek mini: Training yang tahan putus runtime.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: checkpoint, optimizer_state, epoch, early_stopping.

Parameter	Target
Level	Menengah
Waktu	60-120 menit
Artefak	Training yang tahan putus runtime
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 27

Checkpoint, Resume, dan Early Stopping



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah memulihkan eksperimen tanpa kehilangan state penting. Alur di atas memperlakukan checkpoint sebagai input keputusan, optimizer_state sebagai transformasi utama, epoch sebagai state atau mekanisme kontrol, dan early_stopping sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika optimizer_state berubah, bagaimana dampaknya terhadap early_stopping? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk memulihkan eksperimen tanpa kehilangan state penting. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 checkpoint = {
2     'epoch': epoch, 'model': model.state_dict(),
3     'optimizer': optimizer.state_dict(), 'scheduler': scheduler.state_dict(),
4     'best_metric': best_metric, 'rng': torch.get_rng_state(),
5 }
6 torch.save(checkpoint, '/content/checkpoint.pt')
7 state = torch.load('/content/checkpoint.pt', map_location=device, weights_only=False)
8 model.load_state_dict(state['model']); optimizer.load_state_dict(state['optimizer'])
9 start_epoch = state['epoch'] + 1
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi early_stopping dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.get_rng_state	Mewakili checkpoint; catat input, output, dtype, device, serta state yang berubah.
torch.save	Mewakili optimizer_state; catat input, output, dtype, device, serta state yang berubah.
torch.load	Mewakili epoch; catat input, output, dtype, device, serta state yang berubah.
False	Mewakili early_stopping; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Hanya menyimpan bobot model tanpa optimizer.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 27

Checkpoint, Resume, dan Early Stopping

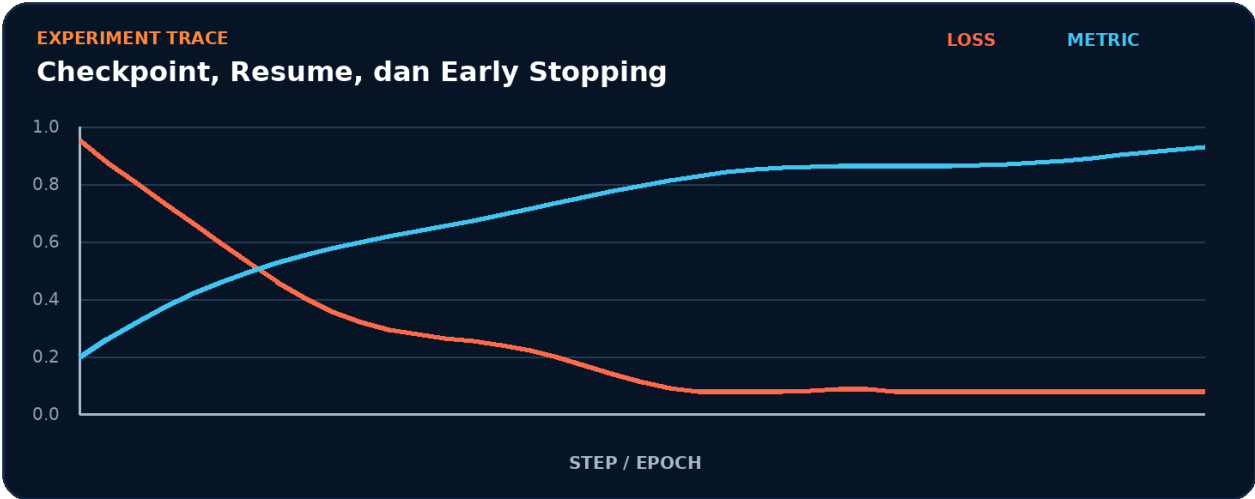


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk checkpoint.
2. Terapkan transformasi utama: `optimizer_state`.
3. Kelola state atau kontrol melalui epoch.
4. Ukur hasil akhir memakai `early_stopping`.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur checkpoint -> optimizer_state -> epoch -> early_stopping tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk early_stopping.
- 2. Ubah satu nilai yang memengaruhi optimizer_state.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada optimizer_state akan memengaruhi early_stopping.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	early_stopping
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah checkpoint menjadi bottleneck.
- Pisahkan biaya setup dari steady-state optimizer_state.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Hanya menyimpan bobot model tanpa optimizer.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait checkpoint.
3. Isolasi	Nonaktifkan sementara optimizer_state dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Training yang tahan putus runtime

Bangun artefak kecil yang membuktikan Anda dapat memulihkan eksperimen tanpa kehilangan state penting. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk `early_stopping`.
2. Implementasikan baseline memakai `checkpoint` dan `optimizer_state`.
3. Tambahkan logging untuk epoch.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan checkpoint tanpa menggunakan istilah `optimizer_state`.
2. Apa `shape`, `dtype`, dan `device` yang diharapkan pada batas `optimizer_state`?
3. Kapan epoch berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas `early_stopping`?
5. Bagaimana Anda mereproduksi kegagalan: hanya menyimpan bobot model tanpa optimizer?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Training yang tahan putus runtime?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk `optimizer_state`, lalu buat tabel yang membandingkan `early_stopping`, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
checkpoint	Peran inti dalam memulihkan eksperimen tanpa kehilangan state penting.
optimizer_state	Peran inti dalam memulihkan eksperimen tanpa kehilangan state penting.
epoch	Peran inti dalam memulihkan eksperimen tanpa kehilangan state penting.
early_stopping	Peran inti dalam memulihkan eksperimen tanpa kehilangan state penting.

Checklist siap pakai

- Verifikasi checkpoint sebelum eksekusi utama.
- Catat perubahan pada optimizer_state dan epoch.
- Ukur early_stopping dengan baseline yang adil.
- Tambahkan test untuk mencegah: hanya menyimpan bobot model tanpa optimizer.
- Simpan artefak proyek: Training yang tahan putus runtime.

API yang muncul

torch.get_rng_state, torch.save, torch.load, False

SATU KALIMAT Bab 27 mengajarkan cara memulihkan eksperimen tanpa kehilangan state penting dengan kontrak eksplisit dan bukti terukur.

BAGIAN 7 - TRAINING LANJUTAN

Bab 28. Hyperparameter Tuning dan Experiment Tracking

Membandingkan run secara disiplin.

Hasil belajar

- Menjelaskan peran search_space dan hubungannya dengan metric.
- Menulis notebook Colab untuk membandingkan run secara disiplin.
- Mendiagnosis risiko: memilih model berdasarkan test set.
- Menghasilkan proyek mini: Sweep learning rate.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: search_space, metric, trial, tracking.

Parameter	Target
Level	Menengah
Waktu	60-120 menit
Artefak	Sweep learning rate
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 28

Hyperparameter Tuning dan Experiment Tracking



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah membandingkan run secara disiplin. Alur di atas memperlakukan `search_space` sebagai input keputusan, `metric` sebagai transformasi utama, `trial` sebagai state atau mekanisme kontrol, dan `tracking` sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika `metric` berubah, bagaimana dampaknya terhadap `tracking`? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk membandingkan run secara disiplin. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 def trial(lr, weight_decay):
2     model = build_model().to(device)
3     opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay)
4     for _ in range(3): train_one_epoch(model, train_loader, opt)
5     return evaluate(model, valid_loader)['accuracy']
6 results = []
7 for lr in [1e-4, 3e-4, 1e-3]:
8     for wd in [0.0, 0.01]: results.append((trial(lr, wd), lr, wd))
9 print(max(results))
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi tracking dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.optim.AdamW	Mewakili search_space; catat input, output, dtype, device, serta state yang berubah.
torch.optim.AdamW	Mewakili metric; catat input, output, dtype, device, serta state yang berubah.
torch.optim.AdamW	Mewakili trial; catat input, output, dtype, device, serta state yang berubah.
torch.optim.AdamW	Mewakili tracking; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Memilih model berdasarkan test set.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 28

Hyperparameter Tuning dan Experiment Tracking

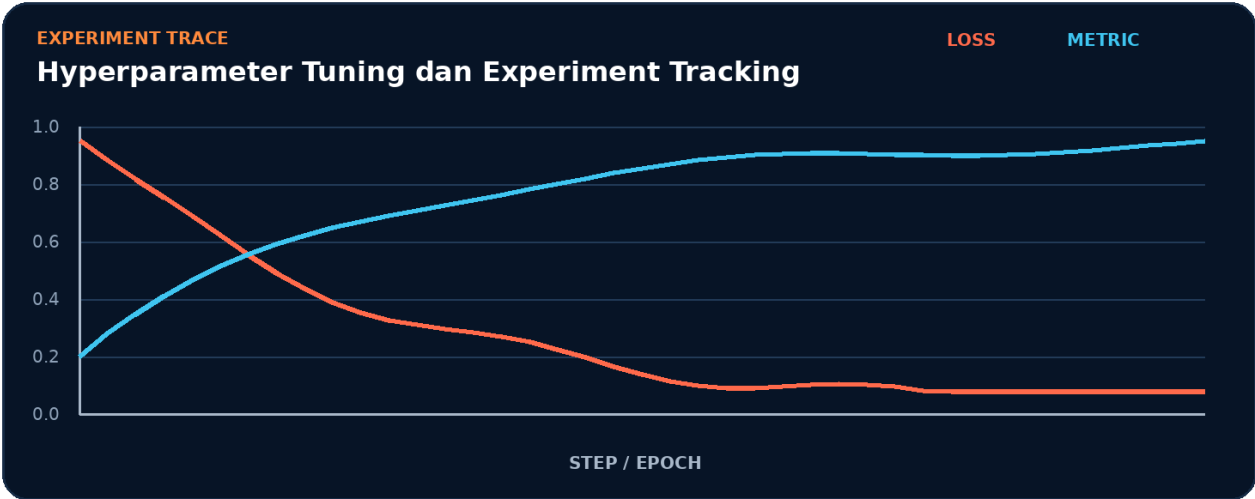


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk `search_space`.
2. Terapkan transformasi utama: `metric`.
3. Kelola state atau kontrol melalui `trial`.
4. Ukur hasil akhir memakai `tracking`.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur `search_space` -> `metric` -> `trial` -> `tracking` tanpa melihat halaman ini.

Eksperimen Terpandu



1. Simpan baseline untuk tracking.
2. Ubah satu nilai yang memengaruhi metric.
3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada metric akan memengaruhi tracking.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	tracking
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah search_space menjadi bottleneck.
- Pisahkan biaya setup dari steady-state metric.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Memilih model berdasarkan test set.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait search_space.
3. Isolasi	Nonaktifkan sementara metric dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Sweep learning rate

Bangun artefak kecil yang membuktikan Anda dapat membandingkan run secara disiplin. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk tracking.
2. Implementasikan baseline memakai `search_space` dan `metric`.
3. Tambahkan logging untuk trial.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan `search_space` tanpa menggunakan istilah `metric`.
2. Apa `shape`, `dtype`, dan `device` yang diharapkan pada batas `metric`?
3. Kapan `trial` berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas `tracking`?
5. Bagaimana Anda mereproduksi kegagalan: memilih model berdasarkan test set?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum `profiling`?
7. Bagaimana desain `checkpoint` atau test untuk proyek `Sweep learning rate`?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk `metric`, lalu buat tabel yang membandingkan `tracking`, waktu eksekusi, dan `peak memory`. Pertahankan `seed` serta jumlah `step`.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak `tensor`, `state`, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
search_space	Peran inti dalam membandingkan run secara disiplin.
metric	Peran inti dalam membandingkan run secara disiplin.
trial	Peran inti dalam membandingkan run secara disiplin.
tracking	Peran inti dalam membandingkan run secara disiplin.

Checklist siap pakai

- Verifikasi `search_space` sebelum eksekusi utama.
- Catat perubahan pada `metric` dan `trial`.
- Ukur `tracking` dengan baseline yang adil.
- Tambahkan test untuk mencegah: memilih model berdasarkan test set.
- Simpan artefak proyek: Sweep learning rate.

API yang muncul

torch.optim.AdamW

SATU KALIMAT Bab 28 mengajarkan cara membandingkan run secara disiplin dengan kontrak eksplisit dan bukti terukur.

BAGIAN 8 - KOMPILASI DAN KINERJA

Bab 29. torch.compile dan Graph Capture

Mengoptimalkan eksekusi model dengan compiler stack.

Hasil belajar

- Menjelaskan peran Dynamo dan hubungannya dengan AOTAutograd.
- Menulis notebook Colab untuk mengoptimalkan eksekusi model dengan compiler stack.
- Mendiagnosis risiko: mengukur compile time sebagai steady-state latency.
- Menghasilkan proyek mini: Benchmark eager vs compile.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: Dynamo, AOTAutograd, Inductor, graph_break.

Parameter	Target
Level	Menengah
Waktu	60-120 menit
Artefak	Benchmark eager vs compile
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 29

torch.compile dan Graph Capture



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah mengoptimalkan eksekusi model dengan compiler stack. Alur di atas memperlakukan Dynamo sebagai input keputusan, AOTAutograd sebagai transformasi utama, Inductor sebagai state atau mekanisme kontrol, dan graph_break sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika AOTAutograd berubah, bagaimana dampaknya terhadap graph_break? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk mengoptimalkan eksekusi model dengan compiler stack. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import time, torch
2 model = build_model().to(device).eval()
3 x = torch.randn(64, 128, device=device)
4 compiled = torch.compile(model, mode='default')
5 for _ in range(5): compiled(x)
6 if device.type == 'cuda': torch.cuda.synchronize()
7 start = time.perf_counter()
8 for _ in range(100): compiled(x)
9 if device.type == 'cuda': torch.cuda.synchronize()
10 print('ms/iter:', (time.perf_counter()-start)*10)
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi graph_break dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.randn	Mewakili Dynamo; catat input, output, dtype, device, serta state yang berubah.
torch.compile	Mewakili AOTAutograd; catat input, output, dtype, device, serta state yang berubah.
torch.cuda.synchronize	Mewakili Inductor; catat input, output, dtype, device, serta state yang berubah.
torch.randn	Mewakili graph_break; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Mengukur compile time sebagai steady-state latency.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 29

torch.compile dan Graph Capture

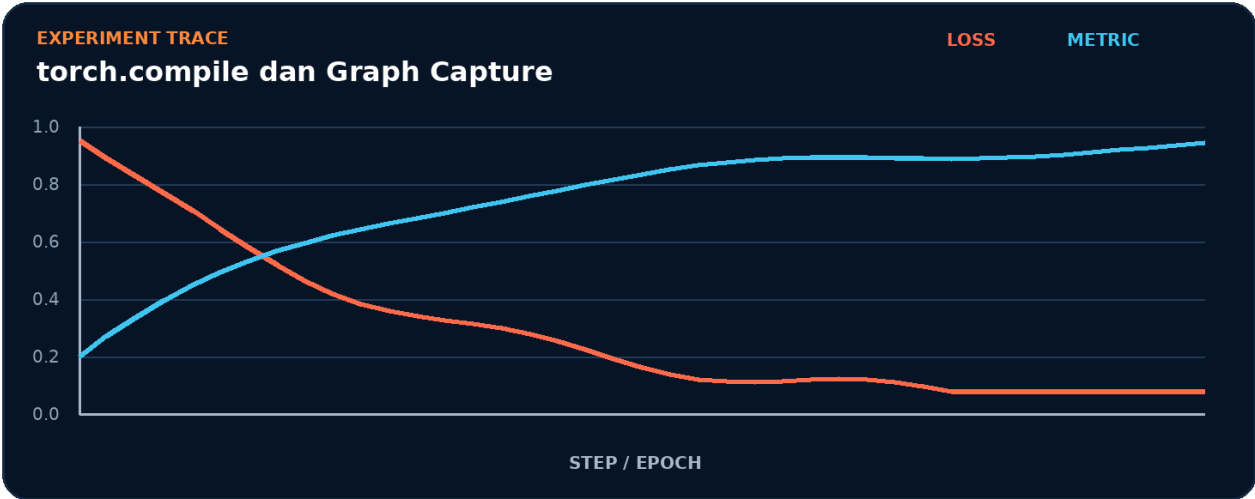


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk Dynamo.
2. Terapkan transformasi utama: AOTAutograd.
3. Kelola state atau kontrol melalui Inductor.
4. Ukur hasil akhir memakai graph_break.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur Dynamo -> AOTAutograd -> Inductor -> graph_break tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk graph_break.
- 2. Ubah satu nilai yang memengaruhi AOTAutograd.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada AOTAutograd akan memengaruhi graph_break.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	graph_break
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah Dynamo menjadi bottleneck.
- Pisahkan biaya setup dari steady-state AOTAutograd.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Mengukur compile time sebagai steady-state latency.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait Dynamo.
3. Isolasi	Nonaktifkan sementara AOTAutograd dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```

1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()

```

Mini Project: Benchmark eager vs compile

Bangun artefak kecil yang membuktikan Anda dapat mengoptimalkan eksekusi model dengan compiler stack. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk `graph_break`.
2. Implementasikan baseline memakai `Dynamo` dan `AOTAutograd`.
3. Tambahkan logging untuk `Inductor`.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan Dynamo tanpa menggunakan istilah AOTAutograd.
2. Apa shape, dtype, dan device yang diharapkan pada batas AOTAutograd?
3. Kapan Inductor berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas graph_break?
5. Bagaimana Anda mereproduksi kegagalan: mengukur compile time sebagai steady-state latency?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Benchmark eager vs compile?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk AOTAutograd, lalu buat tabel yang membandingkan graph_break, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
Dynamo	Peran inti dalam mengoptimalkan eksekusi model dengan compiler stack.
AOTAutograd	Peran inti dalam mengoptimalkan eksekusi model dengan compiler stack.
Inductor	Peran inti dalam mengoptimalkan eksekusi model dengan compiler stack.
graph_break	Peran inti dalam mengoptimalkan eksekusi model dengan compiler stack.

Checklist siap pakai

- Verifikasi Dynamo sebelum eksekusi utama.
- Catat perubahan pada AOTAutograd dan Inductor.
- Ukur graph_break dengan baseline yang adil.
- Tambahkan test untuk mencegah: mengukur compile time sebagai steady-state latency.
- Simpan artefak proyek: Benchmark eager vs compile.

API yang muncul

torch.randn, torch.compile, torch.cuda.synchronize

SATU KALIMAT Bab 29 mengajarkan cara mengoptimalkan eksekusi model dengan compiler stack dengan kontrak eksplisit dan bukti terukur.

BAGIAN 8 - KOMPILASI DAN KINERJA

Bab 30. Profiler dan Analisis Bottleneck

Mengukur waktu cpu, cuda, memori, dan kernel.

Hasil belajar

- Menjelaskan peran profiler dan hubungannya dengan trace.
- Menulis notebook Colab untuk mengukur waktu CPU, CUDA, memori, dan kernel.
- Mendiagnosis risiko: mengoptimalkan tanpa warm-up dan sinkronisasi.
- Menghasilkan proyek mini: Laporan top operator.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: profiler, trace, operator, bottleneck.

Parameter	Target
Level	Menengah
Waktu	60-120 menit
Artefak	Laporan top operator
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 30

Profiler dan Analisis Bottleneck



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah mengukur waktu CPU, CUDA, memori, dan kernel. Alur di atas memperlakukan profiler sebagai input keputusan, trace sebagai transformasi utama, operator sebagai state atau mekanisme kontrol, dan bottleneck sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika trace berubah, bagaimana dampaknya terhadap bottleneck? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk mengukur waktu CPU, CUDA, memori, dan kernel. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch.profiler import profile, ProfilerActivity, record_function
3 activities = [ProfilerActivity.CPU] + ([ProfilerActivity.CUDA] if torch.cuda.is_available() else [])
4 with profile(activities=activities, record_shapes=True, profile_memory=True) as prof:
5     with record_function('train_step'):
6         logits=model(x); loss=criterion(logits,y)
7         optimizer.zero_grad(set_to_none=True); loss.backward(); optimizer.step()
8 print(prof.key_averages().table(sort_by='self_cpu_time_total', row_limit=8))
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi bottleneck dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.profiler	Mewakili profiler; catat input, output, dtype, device, serta state yang berubah.
ProfilerActivity	Mewakili trace; catat input, output, dtype, device, serta state yang berubah.
CPU	Mewakili operator; catat input, output, dtype, device, serta state yang berubah.
CUDA	Mewakili bottleneck; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Mengoptimalkan tanpa warm-up dan sinkronisasi.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 30

Profiler dan Analisis Bottleneck

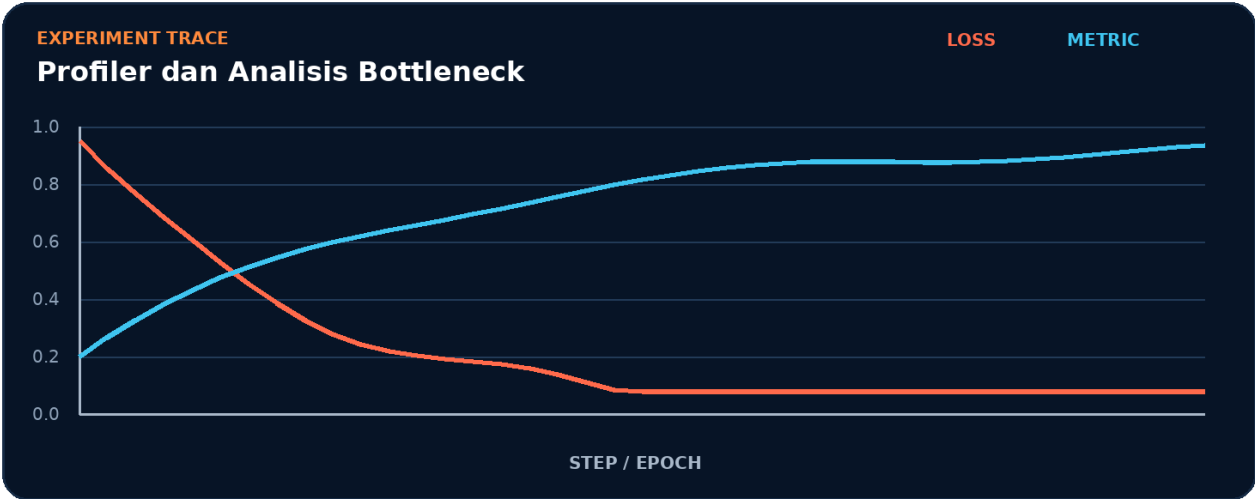


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk profiler.
2. Terapkan transformasi utama: trace.
3. Kelola state atau kontrol melalui operator.
4. Ukur hasil akhir memakai bottleneck.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur profiler -> trace -> operator -> bottleneck tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk bottleneck.
- 2. Ubah satu nilai yang memengaruhi trace.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada trace akan memengaruhi bottleneck.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	bottleneck
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah profiler menjadi bottleneck.
- Pisahkan biaya setup dari steady-state trace.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Mengoptimalkan tanpa warm-up dan sinkronisasi.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait profiler.
3. Isolasi	Nonaktifkan sementara trace dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```

1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()

```

Mini Project: Laporan top operator

Bangun artefak kecil yang membuktikan Anda dapat mengukur waktu CPU, CUDA, memori, dan kernel. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk bottleneck.
2. Implementasikan baseline memakai profiler dan trace.
3. Tambahkan logging untuk operator.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan profiler tanpa menggunakan istilah trace.
2. Apa shape, dtype, dan device yang diharapkan pada batas trace?
3. Kapan operator berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas bottleneck?
5. Bagaimana Anda mereproduksi kegagalan: mengoptimalkan tanpa warm-up dan sinkronisasi?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Laporan top operator?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk trace, lalu buat tabel yang membandingkan bottleneck, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
profiler	Peran inti dalam mengukur waktu CPU, CUDA, memori, dan kernel.
trace	Peran inti dalam mengukur waktu CPU, CUDA, memori, dan kernel.
operator	Peran inti dalam mengukur waktu CPU, CUDA, memori, dan kernel.
bottleneck	Peran inti dalam mengukur waktu CPU, CUDA, memori, dan kernel.

Checklist siap pakai

- Verifikasi profiler sebelum eksekusi utama.
- Catat perubahan pada trace dan operator.
- Ukur bottleneck dengan baseline yang adil.
- Tambahkan test untuk mencegah: mengoptimalkan tanpa warm-up dan sinkronisasi.
- Simpan artefak proyek: Laporan top operator.

API yang muncul

torch.profiler, ProfilerActivity, CPU, CUDA, torch.cuda.is_available, True

SATU KALIMAT Bab 30 mengajarkan cara mengukur waktu CPU, CUDA, memori, dan kernel dengan kontrak eksplisit dan bukti terukur.

BAGIAN 8 - KOMPILASI DAN KINERJA

Bab 31. Efisiensi Memori

Menekan peak memory tanpa merusak numerik.

Hasil belajar

- Menjelaskan peran `activation_checkpointing` dan hubungannya dengan `in-place`.
- Menulis notebook Colab untuk menekan peak memory tanpa merusak numerik.
- Mendiagnosis risiko: `in-place operation` merusak tensor yang dibutuhkan `backward`.
- Menghasilkan proyek mini: Model lebih besar pada GPU sama.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: `activation_checkpointing`, `in-place`, `gradient`, `memory_snapshot`.

Parameter	Target
Level	Menengah
Waktu	60-120 menit
Artefak	Model lebih besar pada GPU sama
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 31

Efisiensi Memori



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah menekan peak memory tanpa merusak numerik. Alur di atas memperlakukan `activation_checkpointing` sebagai input keputusan, `in-place` sebagai transformasi utama, `gradient` sebagai state atau mekanisme kontrol, dan `memory_snapshot` sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika `in-place` berubah, bagaimana dampaknya terhadap `memory_snapshot`? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk menekan peak memory tanpa merusak numerik. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch.utils.checkpoint import checkpoint
3 class BigBlock(torch.nn.Module):
4     def __init__(self, d=1024):
5         super().__init__()
6 self.net=torch.nn.Sequential(torch.nn.Linear(d,4*d),torch.nn.GELU(),torch.nn.Linear(4*d,d))
7     def forward(self,x): return self.net(x)
8 block = BigBlock().to(device)
9 x = torch.randn(8,256,1024,device=device,requires_grad=True)
10 y = checkpoint(block, x, use_reentrant=False); y.mean().backward()
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi memory_snapshot dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.utils.checkpoint	Mewakili activation_checkpointing; catat input, output, dtype, device, serta state yang berubah.
BigBlock	Mewakili in-place; catat input, output, dtype, device, serta state yang berubah.
torch.nn.Module	Mewakili gradient; catat input, output, dtype, device, serta state yang berubah.
torch.nn.Sequential	Mewakili memory_snapshot; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	In-place operation merusak tensor yang dibutuhkan backward.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 31

Efisiensi Memori

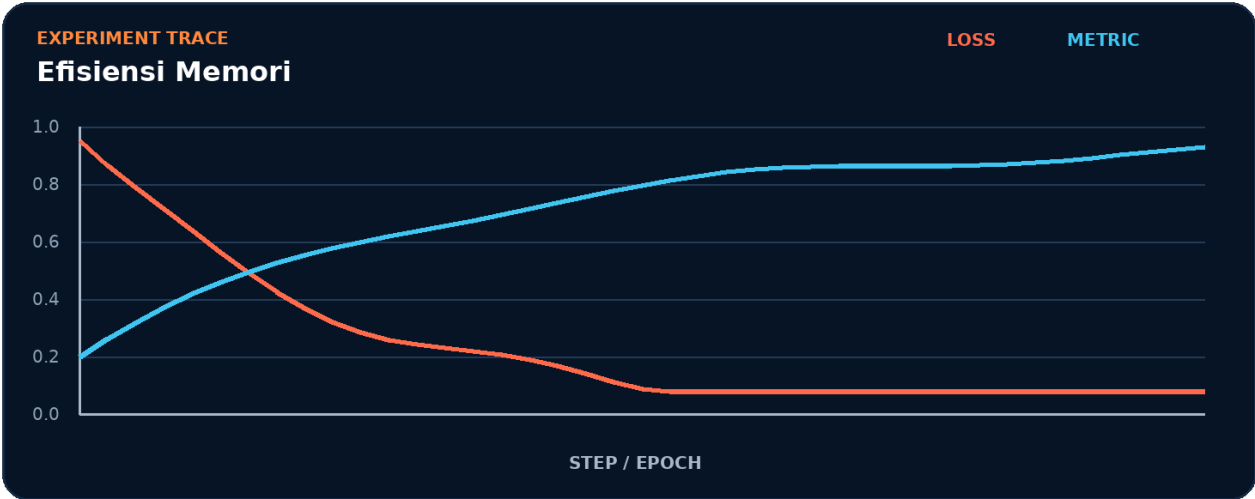


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk `activation_checkpointing`.
2. Terapkan transformasi utama: `in-place`.
3. Kelola state atau kontrol melalui `gradient`.
4. Ukur hasil akhir memakai `memory_snapshot`.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur `activation_checkpointing -> in-place -> gradient -> memory_snapshot` tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk memory_snapshot.
- 2. Ubah satu nilai yang memengaruhi in-place.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada in-place akan memengaruhi memory_snapshot.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	memory_snapshot
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah activation_checkpointing menjadi bottleneck.
- Pisahkan biaya setup dari steady-state in-place.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA In-place operation merusak tensor yang dibutuhkan backward.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait activation_checkpointing.
3. Isolasi	Nonaktifkan sementara in-place dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Model lebih besar pada GPU sama

Bangun artefak kecil yang membuktikan Anda dapat menekan peak memory tanpa merusak numerik. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk `memory_snapshot`.
2. Implementasikan baseline memakai `activation_checkpointing` dan `in-place`.
3. Tambahkan logging untuk gradient.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan `activation_checkpointing` tanpa menggunakan istilah `in-place`.
2. Apa `shape`, `dtype`, dan `device` yang diharapkan pada batas `in-place`?
3. Kapan `gradient` berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas `memory_snapshot`?
5. Bagaimana Anda mereproduksi kegagalan: `in-place operation` merusak tensor yang dibutuhkan `backward`?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum `profiling`?
7. Bagaimana desain `checkpoint` atau `test` untuk proyek Model lebih besar pada GPU sama?
8. Sebutkan satu `trade-off` antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi `baseline` agar menerima dua konfigurasi berbeda untuk `in-place`, lalu buat tabel yang membandingkan `memory_snapshot`, waktu eksekusi, dan `peak memory`. Pertahankan `seed` serta jumlah `step`.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
activation_checkpointing	Peran inti dalam menekan peak memory tanpa merusak numerik.
in-place	Peran inti dalam menekan peak memory tanpa merusak numerik.
gradient	Peran inti dalam menekan peak memory tanpa merusak numerik.
memory_snapshot	Peran inti dalam menekan peak memory tanpa merusak numerik.

Checklist siap pakai

- Verifikasi `activation_checkpointing` sebelum eksekusi utama.
- Catat perubahan pada `in-place` dan `gradient`.
- Ukur `memory_snapshot` dengan baseline yang adil.
- Tambahkan test untuk mencegah: `in-place operation` merusak tensor yang dibutuhkan backward.
- Simpan artefak proyek: Model lebih besar pada GPU sama.

API yang muncul

`torch.utils.checkpoint`, `BigBlock`, `torch.nn.Module`, `torch.nn.Sequential`, `torch.nn.Linear`, `torch.nn.GELU`

SATU KALIMAT Bab 31 mengajarkan cara menekan peak memory tanpa merusak numerik dengan kontrak eksplisit dan bukti terukur.

BAGIAN 8 - KOMPILASI DAN KINERJA

Bab 32. Quantization dan Pruning

Mengecilkan model untuk inference yang lebih efisien.

Hasil belajar

- Menjelaskan peran quantization dan hubungannya dengan int8.
- Menulis notebook Colab untuk mengecilkan model untuk inference yang lebih efisien.
- Mendiagnosis risiko: kalibrasi memakai data yang tidak representatif.
- Menghasilkan proyek mini: Model terkompresi dengan evaluasi akurasi.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: quantization, int8, pruning, calibration.

Parameter	Target
Level	Menengah
Waktu	60-120 menit
Artefak	Model terkompresi dengan evaluasi akurasi
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 32

Quantization dan Pruning



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah mengecilkan model untuk inference yang lebih efisien. Alur di atas memperlakukan quantization sebagai input keputusan, int8 sebagai transformasi utama, pruning sebagai state atau mekanisme kontrol, dan calibration sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika int8 berubah, bagaimana dampaknya terhadap calibration? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk mengecilkan model untuk inference yang lebih efisien. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch.ao.quantization import quantize_dynamic
3 model = torch.nn.Sequential(torch.nn.Linear(128,256),torch.nn.ReLU(),torch.nn.Linear(256,10)).eval()
4 qmodel = quantize_dynamic(model, {torch.nn.Linear}, dtype=torch.qint8)
5 x = torch.randn(32,128)
6 with torch.inference_mode():
7     ref, pred = model(x), qmodel(x)
8     print('mean abs error:', (ref-pred).abs().mean().item())
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi calibration dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.ao.quantization	Mewakili quantization; catat input, output, dtype, device, serta state yang berubah.
torch.nn.Sequential	Mewakili int8; catat input, output, dtype, device, serta state yang berubah.
torch.nn.Linear	Mewakili pruning; catat input, output, dtype, device, serta state yang berubah.
torch.nn.ReLU	Mewakili calibration; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Kalibrasi memakai data yang tidak representatif.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 32

Quantization dan Pruning

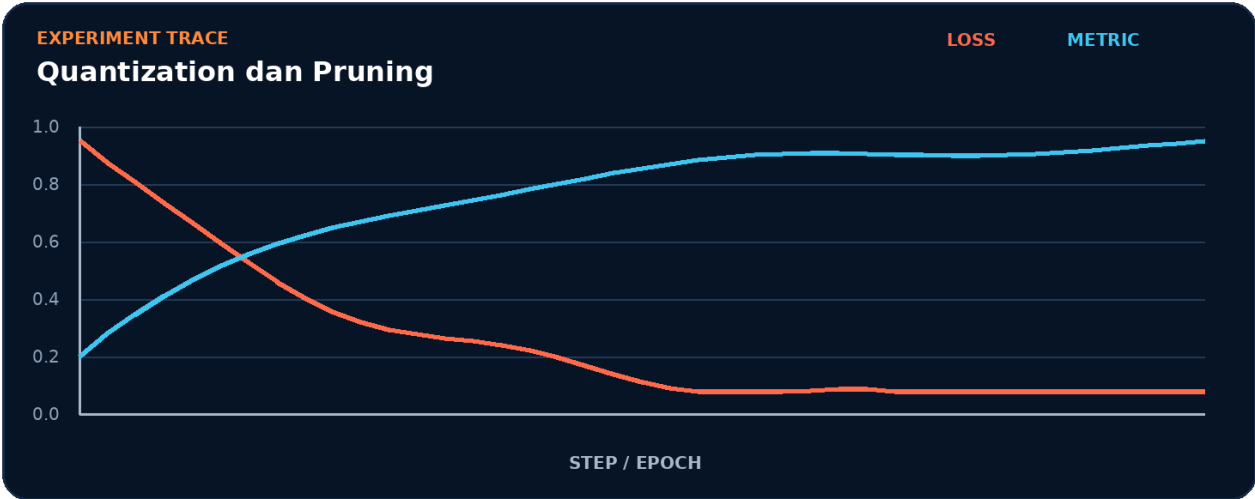


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk quantization.
2. Terapkan transformasi utama: int8.
3. Kelola state atau kontrol melalui pruning.
4. Ukur hasil akhir memakai calibration.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur quantization -> int8 -> pruning -> calibration tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk calibration.
- 2. Ubah satu nilai yang memengaruhi int8.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada int8 akan memengaruhi calibration.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	calibration
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah quantization menjadi bottleneck.
- Pisahkan biaya setup dari steady-state int8.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Kalibrasi memakai data yang tidak representatif.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait quantization.
3. Isolasi	Nonaktifkan sementara int8 dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Model terkompresi dengan evaluasi akurasi

Bangun artefak kecil yang membuktikan Anda dapat mengecilkan model untuk inference yang lebih efisien. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk calibration.
2. Implementasikan baseline memakai quantization dan int8.
3. Tambahkan logging untuk pruning.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan quantization tanpa menggunakan istilah int8.
2. Apa shape, dtype, dan device yang diharapkan pada batas int8?
3. Kapan pruning berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas calibration?
5. Bagaimana Anda mereproduksi kegagalan: kalibrasi memakai data yang tidak representatif?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Model terkompresi dengan evaluasi akurasi?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk int8, lalu buat tabel yang membandingkan calibration, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
quantization	Peran inti dalam mengecilkan model untuk inference yang lebih efisien.
int8	Peran inti dalam mengecilkan model untuk inference yang lebih efisien.
pruning	Peran inti dalam mengecilkan model untuk inference yang lebih efisien.
calibration	Peran inti dalam mengecilkan model untuk inference yang lebih efisien.

Checklist siap pakai

- Verifikasi quantization sebelum eksekusi utama.
- Catat perubahan pada int8 dan pruning.
- Ukur calibration dengan baseline yang adil.
- Tambahkan test untuk mencegah: kalibrasi memakai data yang tidak representatif.
- Simpan artefak proyek: Model terkompresi dengan evaluasi akurasi.

API yang muncul

torch.ao.quantization, torch.nn.Sequential, torch.nn.Linear, torch.nn.ReLU, torch.qint8, torch.randn

SATU KALIMAT Bab 32 mengajarkan cara mengecilkan model untuk inference yang lebih efisien dengan kontrak eksplisit dan bukti terukur.

BAGIAN 9 - DISTRIBUTED DAN SCALE-OUT

Bab 33. DistributedDataParallel

Melatih satu model pada banyak gpu dengan sinkronisasi gradien.

Hasil belajar

- Menjelaskan peran process_group dan hubungannya dengan rank.
- Menulis notebook Colab untuk melatih satu model pada banyak GPU dengan sinkronisasi gradien.
- Mendiagnosis risiko: set_epoch tidak dipanggil pada sampler.
- Menghasilkan proyek mini: Template DDP single-node.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: process_group, rank, all_reduce, DistributedSampler.

Parameter	Target
Level	Advanced
Waktu	60-120 menit
Artefak	Template DDP single-node
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 33

DistributedDataParallel



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah melatih satu model pada banyak GPU dengan sinkronisasi gradien. Alur di atas memperlakukan `process_group` sebagai input keputusan, `rank` sebagai transformasi utama, `all_reduce` sebagai state atau mekanisme kontrol, dan `DistributedSampler` sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika rank berubah, bagaimana dampaknya terhadap `DistributedSampler`? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk melatih satu model pada banyak GPU dengan sinkronisasi gradien. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 # jalankan: torchrun --standalone --nproc_per_node=2 train.py
2 import os, torch
3 import torch.distributed as dist
4 from torch.nn.parallel import DistributedDataParallel as DDP
5 dist.init_process_group('nccl')
6 local_rank = int(os.environ['LOCAL_RANK']); torch.cuda.set_device(local_rank)
7 model = DDP(build_model().cuda(local_rank), device_ids=[local_rank])
8 sampler = torch.utils.data.DistributedSampler(dataset, shuffle=True)
9 loader = torch.utils.data.DataLoader(dataset, sampler=sampler, batch_size=64)
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi DistributedSampler dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.distributed	Mewakili process_group; catat input, output, dtype, device, serta state yang berubah.
torch.nn.parallel	Mewakili rank; catat input, output, dtype, device, serta state yang berubah.
DistributedDataParallel	Mewakili all_reduce; catat input, output, dtype, device, serta state yang berubah.
DDP	Mewakili DistributedSampler; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Set_epoch tidak dipanggil pada sampler.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 33

DistributedDataParallel

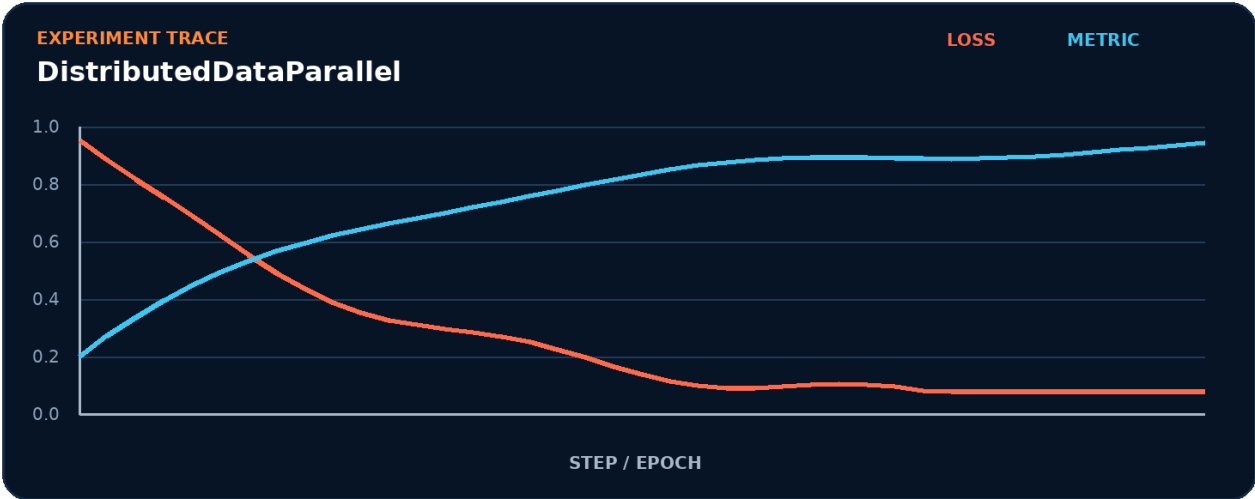


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk process_group.
2. Terapkan transformasi utama: rank.
3. Kelola state atau kontrol melalui all_reduce.
4. Ukur hasil akhir memakai DistributedSampler.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur process_group -> rank -> all_reduce -> DistributedSampler tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk DistributedSampler.
- 2. Ubah satu nilai yang memengaruhi rank.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada rank akan memengaruhi DistributedSampler.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	DistributedSampler
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah process_group menjadi bottleneck.
- Pisahkan biaya setup dari steady-state rank.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Set_epoch tidak dipanggil pada sampler.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait process_group.
3. Isolasi	Nonaktifkan sementara rank dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Template DDP single-node

Bangun artefak kecil yang membuktikan Anda dapat melatih satu model pada banyak GPU dengan sinkronisasi gradien. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

- 1. Definisikan input, target, dan batas keberhasilan untuk DistributedSampler.
- 2. Implementasikan baseline memakai process_group dan rank.
- 3. Tambahkan logging untuk all_reduce.
- 4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
- 5. Simpan config, metric, diagram, dan checkpoint bila relevan.
- 6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan `process_group` tanpa menggunakan istilah rank.
2. Apa shape, dtype, dan device yang diharapkan pada batas rank?
3. Kapan `all_reduce` berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas `DistributedSampler`?
5. Bagaimana Anda mereproduksi kegagalan: `set_epoch` tidak dipanggil pada sampler?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Template DDP single-node?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk rank, lalu buat tabel yang membandingkan `DistributedSampler`, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
<code>process_group</code>	Peran inti dalam melatih satu model pada banyak GPU dengan sinkronisasi gradien.
<code>rank</code>	Peran inti dalam melatih satu model pada banyak GPU dengan sinkronisasi gradien.
<code>all_reduce</code>	Peran inti dalam melatih satu model pada banyak GPU dengan sinkronisasi gradien.
<code>DistributedSampler</code>	Peran inti dalam melatih satu model pada banyak GPU dengan sinkronisasi gradien.

Checklist siap pakai

- Verifikasi `process_group` sebelum eksekusi utama.
- Catat perubahan pada `rank` dan `all_reduce`.
- Ukur `DistributedSampler` dengan baseline yang adil.
- Tambahkan test untuk mencegah: `set_epoch` tidak dipanggil pada sampler.
- Simpan artefak proyek: Template DDP single-node.

API yang muncul

`torch.distributed`, `torch.nn.parallel`, `DistributedDataParallel`, `DDP`, `torch.cuda.set_device`, `torch.utils.data.DistributedSampler`

SATU KALIMAT Bab 33 mengajarkan cara melatih satu model pada banyak GPU dengan sinkronisasi gradien dengan kontrak eksplisit dan bukti terukur.

BAGIAN 9 - DISTRIBUTED DAN SCALE-OUT

Bab 34. Fully Sharded Data Parallel

Membagi parameter, gradien, dan optimizer state.

Hasil belajar

- Menjelaskan peran sharding dan hubungannya dengan FSDP.
- Menulis notebook Colab untuk membagi parameter, gradien, dan optimizer state.
- Mendiagnosis risiko: menyimpan full state pada setiap rank tanpa perencanaan memori.
- Menghasilkan proyek mini: Rancangan training model besar.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: sharding, FSDP, wrap_policy, state_dict.

Parameter	Target
Level	Advanced
Waktu	60-120 menit
Artefak	Rancangan training model besar
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 34

Fully Sharded Data Parallel



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah membagi parameter, gradien, dan optimizer state. Alur di atas memperlakukan sharding sebagai input keputusan, FSDP sebagai transformasi utama, wrap_policy sebagai state atau mekanisme kontrol, dan state_dict sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika FSDP berubah, bagaimana dampaknya terhadap state_dict? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk membagi parameter, gradien, dan optimizer state. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch.distributed.fsdp import FullyShardedDataParallel as FSDP
3 from torch.distributed.fsdp.wrap import size_based_auto_wrap_policy
4 from functools import partial
5 policy = partial(size_based_auto_wrap_policy, min_num_params=1_000_000)
6 model = FSDP(build_model().cuda(), auto_wrap_policy=policy,
7             device_id=torch.cuda.current_device(), use_orig_params=True)
8 optimizer = torch.optim.AdamW(model.parameters(), lr=2e-4)
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi state_dict dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.distributed.fsdp	Mewakili sharding; catat input, output, dtype, device, serta state yang berubah.
FullyShardedDataParallel	Mewakili FSDP; catat input, output, dtype, device, serta state yang berubah.
FSDP	Mewakili wrap_policy; catat input, output, dtype, device, serta state yang berubah.
torch.distributed.fsdp.wrap	Mewakili state_dict; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Menyimpan full state pada setiap rank tanpa perencanaan memori.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 34

Fully Sharded Data Parallel

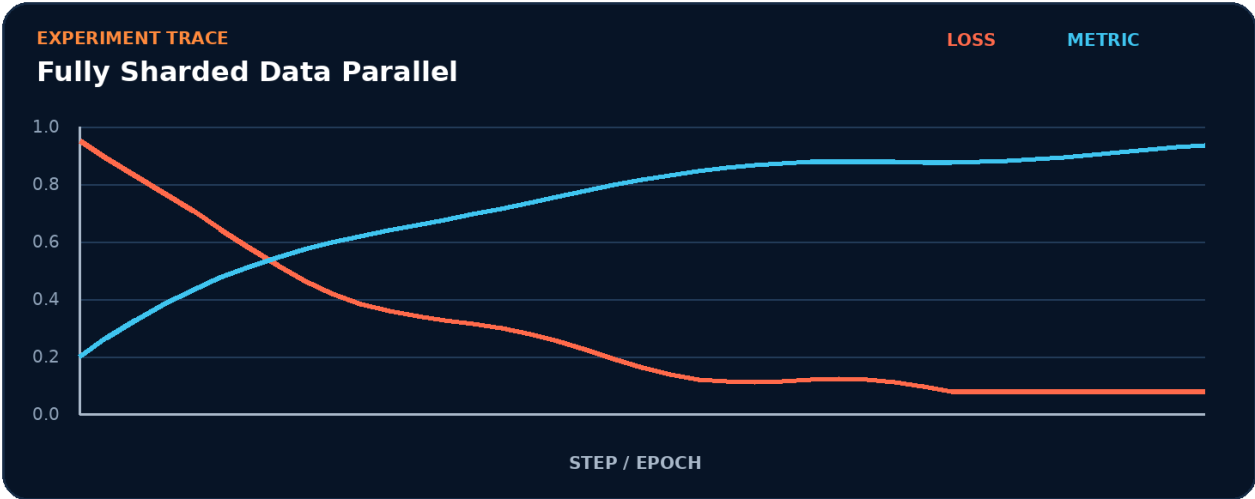


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk sharding.
2. Terapkan transformasi utama: FSDP.
3. Kelola state atau kontrol melalui wrap_policy.
4. Ukur hasil akhir memakai state_dict.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur sharding -> FSDP -> wrap_policy -> state_dict tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk state_dict.
- 2. Ubah satu nilai yang memengaruhi FSDP.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada FSDP akan memengaruhi state_dict.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	state_dict
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah sharding menjadi bottleneck.
- Pisahkan biaya setup dari steady-state FSDP.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Menyimpan full state pada setiap rank tanpa perencanaan memori.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait sharding.
3. Isolasi	Nonaktifkan sementara FSDP dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Rancangan training model besar

Bangun artefak kecil yang membuktikan Anda dapat membagi parameter, gradien, dan optimizer state. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk `state_dict`.
2. Implementasikan baseline memakai sharding dan FSDP.
3. Tambahkan logging untuk `wrap_policy`.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan sharding tanpa menggunakan istilah FSDP.
2. Apa shape, dtype, dan device yang diharapkan pada batas FSDP?
3. Kapan wrap_policy berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas state_dict?
5. Bagaimana Anda mereproduksi kegagalan: menyimpan full state pada setiap rank tanpa perencanaan memori?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Rancangan training model besar?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk FSDP, lalu buat tabel yang membandingkan state_dict, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
sharding	Peran inti dalam membagi parameter, gradien, dan optimizer state.
FSDP	Peran inti dalam membagi parameter, gradien, dan optimizer state.
wrap_policy	Peran inti dalam membagi parameter, gradien, dan optimizer state.
state_dict	Peran inti dalam membagi parameter, gradien, dan optimizer state.

Checklist siap pakai

- Verifikasi sharding sebelum eksekusi utama.
- Catat perubahan pada FSDP dan wrap_policy.
- Ukur state_dict dengan baseline yang adil.
- Tambahkan test untuk mencegah: menyimpan full state pada setiap rank tanpa perencanaan memori.
- Simpan artefak proyek: Rancangan training model besar.

API yang muncul

torch.distributed.fsdp, FullyShardedDataParallel, FSDP, torch.distributed.fsdp.wrap, torch.cuda.current_device, True

SATU KALIMAT Bab 34 mengajarkan cara membagi parameter, gradien, dan optimizer state dengan kontrak eksplisit dan bukti terukur.

BAGIAN 9 - DISTRIBUTED DAN SCALE-OUT

Bab 35. Sharding Data dan Tensor

Memetakan data serta tensor ke mesh perangkat.

Hasil belajar

- Menjelaskan peran shard dan hubungannya dengan replicate.
- Menulis notebook Colab untuk memetakan data serta tensor ke mesh perangkat.
- Mendiagnosis risiko: dimensi shard tidak habis dibagi ukuran mesh.
- Menghasilkan proyek mini: Simulasi strategi placement.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: shard, replicate, device_mesh, placement.

Parameter	Target
Level	Advanced
Waktu	60-120 menit
Artefak	Simulasi strategi placement
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 35

Sharding Data dan Tensor



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah memetakan data serta tensor ke mesh perangkat. Alur di atas memperlakukan shard sebagai input keputusan, replicate sebagai transformasi utama, device_mesh sebagai state atau mekanisme kontrol, dan placement sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika replicate berubah, bagaimana dampaknya terhadap placement? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk memetakan data serta tensor ke mesh perangkat. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch.distributed.device_mesh import init_device_mesh
3 from torch.distributed.tensor import distribute_tensor, Shard, Replicate
4 mesh = init_device_mesh('cuda', (2,), mesh_dim_names=('dp',))
5 x = torch.arange(32, device='cuda').reshape(8,4)
6 x_shard = distribute_tensor(x, mesh, placements=[Shard(0)])
7 x_rep = x_shard.redistribute(placements=[Replicate()])
8 print(x_shard.to_local().shape, x_rep.to_local().shape)
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi placement dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.distributed.device_mesh	Mewakili shard; catat input, output, dtype, device, serta state yang berubah.
torch.distributed.tensor	Mewakili replicate; catat input, output, dtype, device, serta state yang berubah.
Shard	Mewakili device_mesh; catat input, output, dtype, device, serta state yang berubah.
Replicate	Mewakili placement; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Dimensi shard tidak habis dibagi ukuran mesh.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 35

Sharding Data dan Tensor

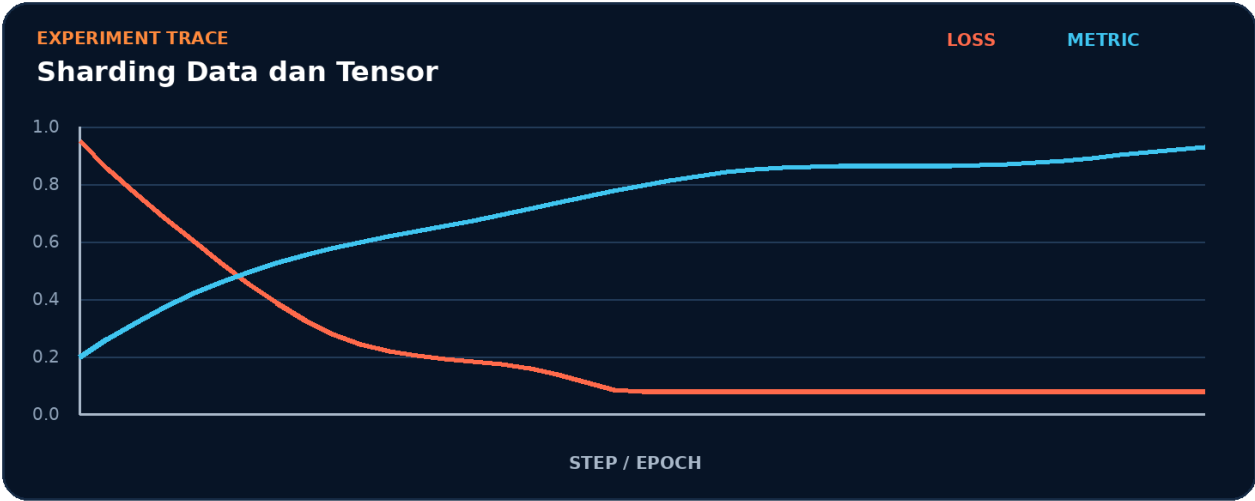


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk shard.
2. Terapkan transformasi utama: replicate.
3. Kelola state atau kontrol melalui device_mesh.
4. Ukur hasil akhir memakai placement.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur shard -> replicate -> device_mesh -> placement tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk placement.
- 2. Ubah satu nilai yang memengaruhi replicate.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada replicate akan memengaruhi placement.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	placement
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah shard menjadi bottleneck.
- Pisahkan biaya setup dari steady-state replicate.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Dimensi shard tidak habis dibagi ukuran mesh.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait shard.
3. Isolasi	Nonaktifkan sementara replicate dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

```
COLAB - DEBUG CELL
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Simulasi strategi placement

Bangun artefak kecil yang membuktikan Anda dapat memetakan data serta tensor ke mesh perangkat. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk placement.
2. Implementasikan baseline memakai shard dan replicate.
3. Tambahkan logging untuk device_mesh.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan shard tanpa menggunakan istilah replicate.
2. Apa shape, dtype, dan device yang diharapkan pada batas replicate?
3. Kapan device_mesh berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas placement?
5. Bagaimana Anda mereproduksi kegagalan: dimensi shard tidak habis dibagi ukuran mesh?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Simulasi strategi placement?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk replicate, lalu buat tabel yang membandingkan placement, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
shard	Peran inti dalam memetakan data serta tensor ke mesh perangkat.
replicate	Peran inti dalam memetakan data serta tensor ke mesh perangkat.
device_mesh	Peran inti dalam memetakan data serta tensor ke mesh perangkat.
placement	Peran inti dalam memetakan data serta tensor ke mesh perangkat.

Checklist siap pakai

- Verifikasi shard sebelum eksekusi utama.
- Catat perubahan pada replicate dan device_mesh.
- Ukur placement dengan baseline yang adil.
- Tambahkan test untuk mencegah: dimensi shard tidak habis dibagi ukuran mesh.
- Simpan artefak proyek: Simulasi strategi placement.

API yang muncul

torch.distributed.device_mesh, torch.distributed.tensor, Shard, Replicate, torch.arange

SATU KALIMAT Bab 35 mengajarkan cara memetakan data serta tensor ke mesh perangkat dengan kontrak eksplisit dan bukti terukur.

BAGIAN 9 - DISTRIBUTED DAN SCALE-OUT

Bab 36. Multi-Node dan Fault Tolerance

Merancang job yang dapat pulih dari kegagalan proses.

Hasil belajar

- Menjelaskan peran elastic dan hubungannya dengan rendezvous.
- Menulis notebook Colab untuk merancang job yang dapat pulih dari kegagalan proses.
- Mendiagnosis risiko: mengasumsikan storage lokal dapat dilihat semua node.
- Menghasilkan proyek mini: Runbook distributed training.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: elastic, rendezvous, checkpoint, restart.

Parameter	Target
Level	Advanced
Waktu	60-120 menit
Artefak	Runbook distributed training
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 36

Multi-Node dan Fault Tolerance



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah merancang job yang dapat pulih dari kegagalan proses. Alur di atas memperlakukan elastic sebagai input keputusan, rendezvous sebagai transformasi utama, checkpoint sebagai state atau mekanisme kontrol, dan restart sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika rendezvous berubah, bagaimana dampaknya terhadap restart? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk merancang job yang dapat pulih dari kegagalan proses. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 # torchrun --nnodes=2 --nproc_per_node=8 --rdzv_backend=c10d ...
2 def save_atomic(state, path):
3     tmp = path + '.tmp'
4     torch.save(state, tmp)
5     os.replace(tmp, path)
6 state = {'step': step, 'model': model.state_dict(),
7         'optimizer': optimizer.state_dict(), 'world_size': dist.get_world_size()}
8 if dist.get_rank() == 0: save_atomic(state, shared_checkpoint_path)
9 dist.barrier()
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi restart dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.save	Mewakili elastic; catat input, output, dtype, device, serta state yang berubah.
torch.save	Mewakili rendezvous; catat input, output, dtype, device, serta state yang berubah.
torch.save	Mewakili checkpoint; catat input, output, dtype, device, serta state yang berubah.
torch.save	Mewakili restart; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Mengasumsikan storage lokal dapat dilihat semua node.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 36

Multi-Node dan Fault Tolerance

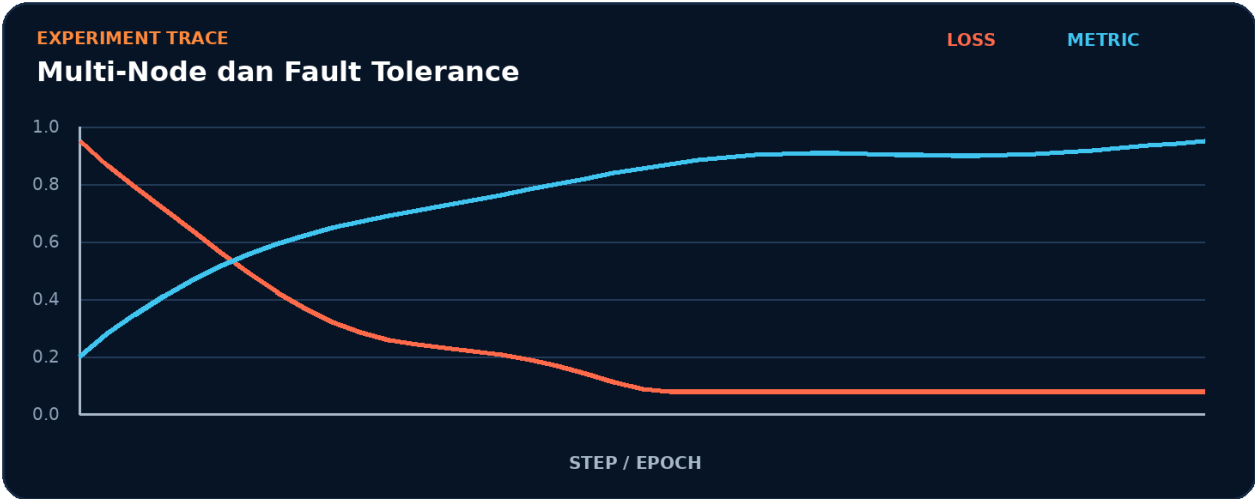


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk elastic.
2. Terapkan transformasi utama: rendezvous.
3. Kelola state atau kontrol melalui checkpoint.
4. Ukur hasil akhir memakai restart.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur elastic -> rendezvous -> checkpoint -> restart tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk restart.
- 2. Ubah satu nilai yang memengaruhi rendezvous.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada rendezvous akan memengaruhi restart.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	restart
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah elastic menjadi bottleneck.
- Pisahkan biaya setup dari steady-state rendezvous.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Mengasumsikan storage lokal dapat dilihat semua node.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait elastic.
3. Isolasi	Nonaktifkan sementara rendezvous dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```

1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()

```

Mini Project: Runbook distributed training

Bangun artefak kecil yang membuktikan Anda dapat merancang job yang dapat pulih dari kegagalan proses. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk restart.
2. Implementasikan baseline memakai elastic dan rendezvous.
3. Tambahkan logging untuk checkpoint.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan elastic tanpa menggunakan istilah rendezvous.
2. Apa shape, dtype, dan device yang diharapkan pada batas rendezvous?
3. Kapan checkpoint berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas restart?
5. Bagaimana Anda mereproduksi kegagalan: mengasumsikan storage lokal dapat dilihat semua node?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Runbook distributed training?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk rendezvous, lalu buat tabel yang membandingkan restart, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
elastic	Peran inti dalam merancang job yang dapat pulih dari kegagalan proses.
rendezvous	Peran inti dalam merancang job yang dapat pulih dari kegagalan proses.
checkpoint	Peran inti dalam merancang job yang dapat pulih dari kegagalan proses.
restart	Peran inti dalam merancang job yang dapat pulih dari kegagalan proses.

Checklist siap pakai

- Verifikasi elastic sebelum eksekusi utama.
- Catat perubahan pada rendezvous dan checkpoint.
- Ukur restart dengan baseline yang adil.
- Tambahkan test untuk mencegah: mengasumsikan storage lokal dapat dilihat semua node.
- Simpan artefak proyek: Runbook distributed training.

API yang muncul

torch.save

SATU KALIMAT Bab 36 mengajarkan cara merancang job yang dapat pulih dari kegagalan proses dengan kontrak eksplisit dan bukti terukur.

BAGIAN 10 - GENERATIVE DAN MULTIMODAL

Bab 37. Autoencoder dan Variational Autoencoder

Belajar representasi laten dan rekonstruksi.

Hasil belajar

- Menjelaskan peran encoder dan hubungannya dengan latent.
- Menulis notebook Colab untuk belajar representasi laten dan rekonstruksi.
- Mendiagnosis risiko: KL loss mendominasi sebelum decoder belajar.
- Menghasilkan proyek mini: VAE citra mini.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: encoder, latent, decoder, KL_divergence.

Parameter	Target
Level	Advanced
Waktu	60-120 menit
Artefak	VAE citra mini
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 37

Autoencoder dan Variational Autoencoder



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah belajar representasi laten dan rekonstruksi. Alur di atas memperlakukan encoder sebagai input keputusan, latent sebagai transformasi utama, decoder sebagai state atau mekanisme kontrol, dan KL_divergence sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika latent berubah, bagaimana dampaknya terhadap KL_divergence? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk belajar representasi laten dan rekonstruksi. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch import nn
3 class VAE(nn.Module):
4     def __init__(self, d=784, z=16):
5         super().__init__(); self.enc=nn.Linear(d,2*z); self.dec=nn.Linear(z,d)
6     def forward(self,x):
7         mu, logvar = self.enc(x).chunk(2,-1); std=(0.5*logvar).exp()
8         z = mu + std*torch.randn_like(std); return self.dec(z), mu, logvar
9 recon,mu,logvar=VAE()(torch.rand(32,784))
10 kl=-0.5*(1+logvar-mu.square()-logvar.exp()).mean(); print(kl.item())
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi KL_divergence dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
VAE	Mewakili encoder; catat input, output, dtype, device, serta state yang berubah.
nn.Module	Mewakili latent; catat input, output, dtype, device, serta state yang berubah.
nn.Linear	Mewakili decoder; catat input, output, dtype, device, serta state yang berubah.
torch.randn_like	Mewakili KL_divergence; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	KI loss mendominasi sebelum decoder belajar.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 37

Autoencoder dan Variational Autoencoder

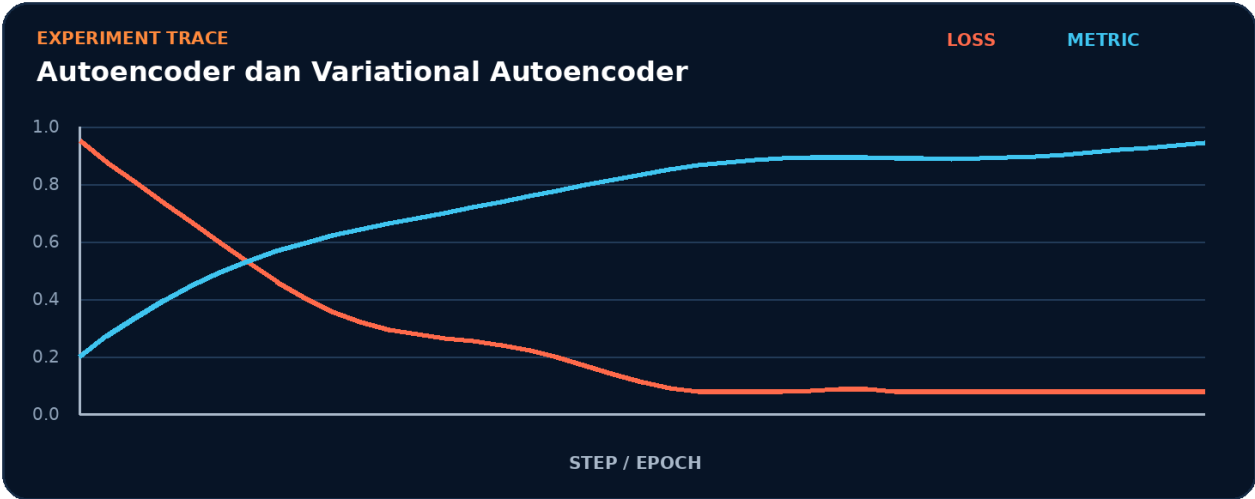


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk encoder.
2. Terapkan transformasi utama: latent.
3. Kelola state atau kontrol melalui decoder.
4. Ukur hasil akhir memakai KL_divergence.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur encoder -> latent -> decoder -> KL_divergence tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk KL_divergence.
- 2. Ubah satu nilai yang memengaruhi latent.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada latent akan memengaruhi KL_divergence.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	KL_divergence
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah encoder menjadi bottleneck.
- Pisahkan biaya setup dari steady-state latent.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA K_I loss mendominasi sebelum decoder belajar.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait encoder.
3. Isolasi	Nonaktifkan sementara latent dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: VAE citra mini

Bangun artefak kecil yang membuktikan Anda dapat belajar representasi laten dan rekonstruksi. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk KL_divergence.
2. Implementasikan baseline memakai encoder dan latent.
3. Tambahkan logging untuk decoder.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan encoder tanpa menggunakan istilah latent.
2. Apa shape, dtype, dan device yang diharapkan pada batas latent?
3. Kapan decoder berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas KL_divergence?
5. Bagaimana Anda mereproduksi kegagalan: KL loss mendominasi sebelum decoder belajar?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek VAE citra mini?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk latent, lalu buat tabel yang membandingkan KL_divergence, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
encoder	Peran inti dalam belajar representasi laten dan rekonstruksi.
latent	Peran inti dalam belajar representasi laten dan rekonstruksi.
decoder	Peran inti dalam belajar representasi laten dan rekonstruksi.
KL_divergence	Peran inti dalam belajar representasi laten dan rekonstruksi.

Checklist siap pakai

- Verifikasi encoder sebelum eksekusi utama.
- Catat perubahan pada latent dan decoder.
- Ukur KL_divergence dengan baseline yang adil.
- Tambahkan test untuk mencegah: KL loss mendominasi sebelum decoder belajar.
- Simpan artefak proyek: VAE citra mini.

API yang muncul

VAE, nn.Module, nn.Linear, torch.randn_like, torch.rand

SATU KALIMAT Bab 37 mengajarkan cara belajar representasi laten dan rekonstruksi dengan kontrak eksplisit dan bukti terukur.

BAGIAN 10 - GENERATIVE DAN MULTIMODAL

Bab 38. Generative Adversarial Network

Menyeimbangkan permainan generator dan discriminator.

Hasil belajar

- Menjelaskan peran generator dan hubungannya dengan discriminator.
- Menulis notebook Colab untuk menyeimbangkan permainan generator dan discriminator.
- Mendiagnosis risiko: satu jaringan tidak di-freeze saat update lawan.
- Menghasilkan proyek mini: GAN 2D sederhana.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: generator, discriminator, adversarial_loss, stability.

Parameter	Target
Level	Advanced
Waktu	60-120 menit
Artefak	GAN 2D sederhana
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 38

Generative Adversarial Network



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah menyeimbangkan permainan generator dan discriminator. Alur di atas memperlakukan generator sebagai input keputusan, discriminator sebagai transformasi utama, adversarial_loss sebagai state atau mekanisme kontrol, dan stability sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika discriminator berubah, bagaimana dampaknya terhadap stability? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk menyeimbangkan permainan generator dan discriminator. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 z = torch.randn(batch, latent_dim, device=device)
2 fake = generator(z)
3 opt_d.zero_grad(set_to_none=True)
4 loss_d = bce(discriminator(real), torch.ones(batch,1,device=device)) + \
5     bce(discriminator(fake.detach()), torch.zeros(batch,1,device=device))
6 loss_d.backward(); opt_d.step()
7 opt_g.zero_grad(set_to_none=True)
8 loss_g = bce(discriminator(fake), torch.ones(batch,1,device=device))
9 loss_g.backward(); opt_g.step()
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi stability dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.randn	Mewakili generator; catat input, output, dtype, device, serta state yang berubah.
True	Mewakili discriminator; catat input, output, dtype, device, serta state yang berubah.
torch.ones	Mewakili adversarial_loss; catat input, output, dtype, device, serta state yang berubah.
torch.zeros	Mewakili stability; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Satu jaringan tidak di-freeze saat update lawan.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 38

Generative Adversarial Network

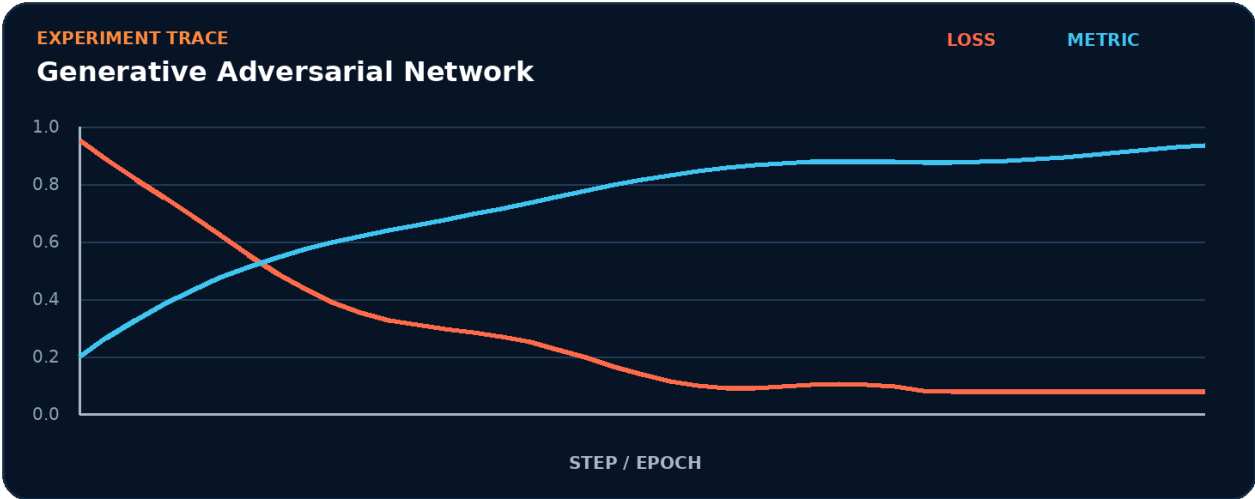


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk generator.
2. Terapkan transformasi utama: discriminator.
3. Kelola state atau kontrol melalui `adversarial_loss`.
4. Ukur hasil akhir memakai `stability`.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur `generator -> discriminator -> adversarial_loss -> stability` tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk stability.
- 2. Ubah satu nilai yang memengaruhi discriminator.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada discriminator akan memengaruhi stability.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	stability
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah generator menjadi bottleneck.
- Pisahkan biaya setup dari steady-state discriminator.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Satu jaringan tidak di-freeze saat update lawan.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait generator.
3. Isolasi	Nonaktifkan sementara discriminator dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```

1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()

```

Mini Project: GAN 2D sederhana

Bangun artefak kecil yang membuktikan Anda dapat menyeimbangkan permainan generator dan discriminator. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk stability.
2. Implementasikan baseline memakai generator dan discriminator.
3. Tambahkan logging untuk adversarial_loss.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan generator tanpa menggunakan istilah discriminator.
2. Apa shape, dtype, dan device yang diharapkan pada batas discriminator?
3. Kapan adversarial_loss berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas stability?
5. Bagaimana Anda mereproduksi kegagalan: satu jaringan tidak di-freeze saat update lawan?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek GAN 2D sederhana?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk discriminator, lalu buat tabel yang membandingkan stability, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
generator	Peran inti dalam menyeimbangkan permainan generator dan discriminator.
discriminator	Peran inti dalam menyeimbangkan permainan generator dan discriminator.
adversarial_loss	Peran inti dalam menyeimbangkan permainan generator dan discriminator.
stability	Peran inti dalam menyeimbangkan permainan generator dan discriminator.

Checklist siap pakai

- Verifikasi generator sebelum eksekusi utama.
- Catat perubahan pada discriminator dan adversarial_loss.
- Ukur stability dengan baseline yang adil.
- Tambahkan test untuk mencegah: satu jaringan tidak di-freeze saat update lawan.
- Simpan artefak proyek: GAN 2D sederhana.

API yang muncul

torch.randn, True, torch.ones, torch.zeros

SATU KALIMAT Bab 38 mengajarkan cara menyeimbangkan permainan generator dan discriminator dengan kontrak eksplisit dan bukti terukur.

BAGIAN 10 - GENERATIVE DAN MULTIMODAL

Bab 39. Diffusion Model

Memahami noising, denoising, dan sampling iteratif.

Hasil belajar

- Menjelaskan peran noise_schedule dan hubungannya dengan timestep.
- Menulis notebook Colab untuk memahami noising, denoising, dan sampling iteratif.
- Mendiagnosis risiko: target prediksi epsilon dan x0 tertukar.
- Menghasilkan proyek mini: Denoiser satu dimensi.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: noise_schedule, timestep, denoiser, sampling.

Parameter	Target
Level	Advanced
Waktu	60-120 menit
Artefak	Denoiser satu dimensi
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 39

Diffusion Model



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah memahami noising, denoising, dan sampling iteratif. Alur di atas memperlakukan noise_schedule sebagai input keputusan, timestep sebagai transformasi utama, denoiser sebagai state atau mekanisme kontrol, dan sampling sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika timestep berubah, bagaimana dampaknya terhadap sampling? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk memahami noising, denoising, dan sampling iteratif. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 T = 1000
3 beta = torch.linspace(1e-4, 0.02, T, device=device)
4 alpha_bar = torch.cumprod(1-beta, dim=0)
5 t = torch.randint(0, T, (x0.size(0),), device=device)
6 noise = torch.randn_like(x0)
7 a = alpha_bar[t].view(-1,1,1,1)
8 xt = a.sqrt()*x0 + (1-a).sqrt()*noise
9 pred_noise = denoiser(xt, t)
10 loss = torch.nn.functional.mse_loss(pred_noise, noise)
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi sampling dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.linspace	Mewakili noise_schedule; catat input, output, dtype, device, serta state yang berubah.
torch.cumprod	Mewakili timestep; catat input, output, dtype, device, serta state yang berubah.
torch.randint	Mewakili denoiser; catat input, output, dtype, device, serta state yang berubah.
torch.randn_like	Mewakili sampling; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Target prediksi epsilon dan x0 tertukar.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 39

Diffusion Model

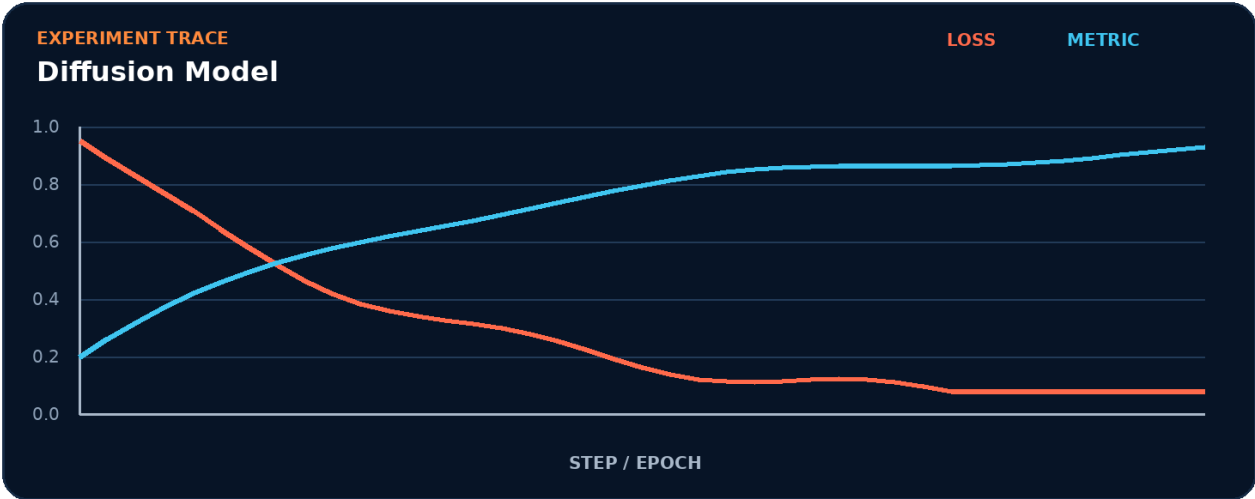


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk noise_schedule.
2. Terapkan transformasi utama: timestep.
3. Kelola state atau kontrol melalui denoiser.
4. Ukur hasil akhir memakai sampling.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur noise_schedule -> timestep -> denoiser -> sampling tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk sampling.
- 2. Ubah satu nilai yang memengaruhi timestep.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada timestep akan memengaruhi sampling.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	sampling
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah noise_schedule menjadi bottleneck.
- Pisahkan biaya setup dari steady-state timestep.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Target prediksi epsilon dan x0 tertukar.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait noise_schedule.
3. Isolasi	Nonaktifkan sementara timestep dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Denoiser satu dimensi

Bangun artefak kecil yang membuktikan Anda dapat memahami noising, denoising, dan sampling iteratif. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

- 1. Definisikan input, target, dan batas keberhasilan untuk sampling.
- 2. Implementasikan baseline memakai noise_schedule dan timestep.
- 3. Tambahkan logging untuk denoiser.
- 4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
- 5. Simpan config, metric, diagram, dan checkpoint bila relevan.
- 6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan noise_schedule tanpa menggunakan istilah timestep.
2. Apa shape, dtype, dan device yang diharapkan pada batas timestep?
3. Kapan denoiser berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas sampling?
5. Bagaimana Anda mereproduksi kegagalan: target prediksi epsilon dan x0 tertukar?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Denoiser satu dimensi?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk timestep, lalu buat tabel yang membandingkan sampling, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
noise_schedule	Peran inti dalam memahami noising, denoising, dan sampling iteratif.
timestep	Peran inti dalam memahami noising, denoising, dan sampling iteratif.
denoiser	Peran inti dalam memahami noising, denoising, dan sampling iteratif.
sampling	Peran inti dalam memahami noising, denoising, dan sampling iteratif.

Checklist siap pakai

- Verifikasi `noise_schedule` sebelum eksekusi utama.
- Catat perubahan pada `timestep` dan `denoiser`.
- Ukur sampling dengan baseline yang adil.
- Tambahkan test untuk mencegah: target prediksi epsilon dan `x0` tertukar.
- Simpan artefak proyek: Denoiser satu dimensi.

API yang muncul

`torch.linspace`, `torch.cumprod`, `torch.randint`, `torch.randn_like`, `torch.nn.functional.mse_loss`

SATU KALIMAT Bab 39 mengajarkan cara memahami noising, denoising, dan sampling iteratif dengan kontrak eksplisit dan bukti terukur.

BAGIAN 10 - GENERATIVE DAN MULTIMODAL

Bab 40. Multimodal dan Contrastive Learning

Menyelaraskan representasi dari dua modalitas.

Hasil belajar

- Menjelaskan peran encoder dan hubungannya dengan projection.
- Menulis notebook Colab untuk menyelaraskan representasi dari dua modalitas.
- Mendiagnosis risiko: label positif tidak selaras antar batch.
- Menghasilkan proyek mini: Pencarian citra-teks mini.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: encoder, projection, contrastive_loss, temperature.

Parameter	Target
Level	Advanced
Waktu	60-120 menit
Artefak	Pencarian citra-teks mini
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 40

Multimodal dan Contrastive Learning



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah menyelaraskan representasi dari dua modalitas. Alur di atas memperlakukan encoder sebagai input keputusan, projection sebagai transformasi utama, contrastive_loss sebagai state atau mekanisme kontrol, dan temperature sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika projection berubah, bagaimana dampaknya terhadap temperature? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk menyelaraskan representasi dari dua modalitas. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 image_z = torch.nn.functional.normalize(image_encoder(images), dim=-1)
2 text_z = torch.nn.functional.normalize(text_encoder(tokens), dim=-1)
3 logits = image_z @ text_z.T / temperature
4 labels = torch.arange(len(images), device=images.device)
5 loss_i = torch.nn.functional.cross_entropy(logits, labels)
6 loss_t = torch.nn.functional.cross_entropy(logits.T, labels)
7 loss = (loss_i + loss_t) / 2
8 print(loss.item())
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi temperature dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.nn.functional.normalize	Mewakili encoder; catat input, output, dtype, device, serta state yang berubah.
torch.arange	Mewakili projection; catat input, output, dtype, device, serta state yang berubah.
torch.nn.functional.cross_entropy	Mewakili contrastive_loss; catat input, output, dtype, device, serta state yang berubah.
torch.nn.functional.normalize	Mewakili temperature; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Label positif tidak selaras antar batch.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 40

Multimodal dan Contrastive Learning

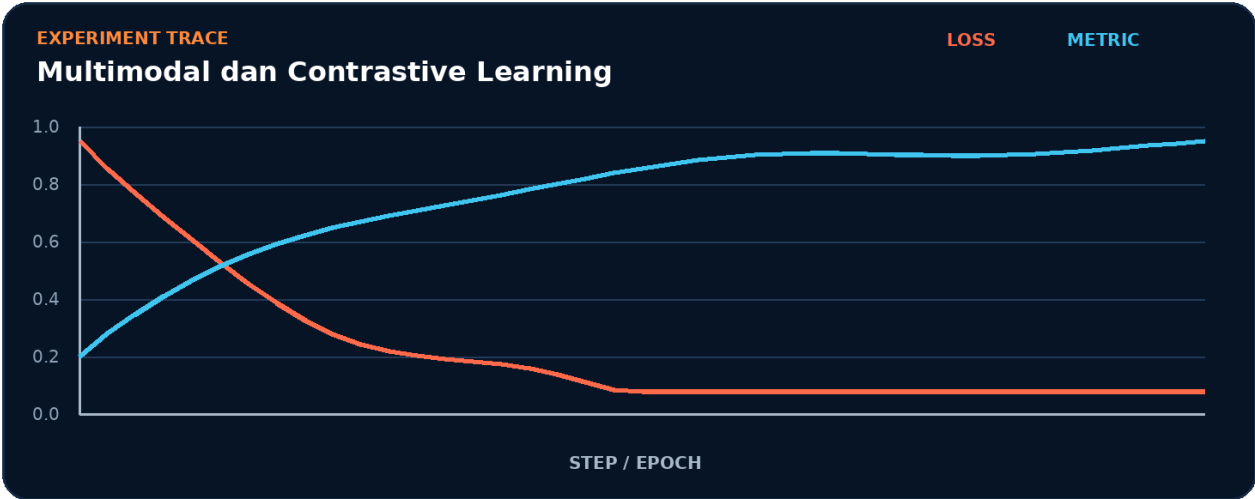


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk encoder.
2. Terapkan transformasi utama: projection.
3. Kelola state atau kontrol melalui `contrastive_loss`.
4. Ukur hasil akhir memakai temperature.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur `encoder -> projection -> contrastive_loss -> temperature` tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk temperature.
- 2. Ubah satu nilai yang memengaruhi projection.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada projection akan memengaruhi temperature.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	temperature
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah encoder menjadi bottleneck.
- Pisahkan biaya setup dari steady-state projection.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Label positif tidak selaras antar batch.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait encoder.
3. Isolasi	Nonaktifkan sementara projection dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```

1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()

```

Mini Project: Pencarian citra-teks mini

Bangun artefak kecil yang membuktikan Anda dapat menyelaraskan representasi dari dua modalitas. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk temperature.
2. Implementasikan baseline memakai encoder dan projection.
3. Tambahkan logging untuk `contrastive_loss`.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan encoder tanpa menggunakan istilah projection.
2. Apa shape, dtype, dan device yang diharapkan pada batas projection?
3. Kapan `contrastive_loss` berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas temperature?
5. Bagaimana Anda mereproduksi kegagalan: label positif tidak selaras antar batch?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Pencarian citra-teks mini?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk projection, lalu buat tabel yang membandingkan temperature, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
encoder	Peran inti dalam menyelaraskan representasi dari dua modalitas.
projection	Peran inti dalam menyelaraskan representasi dari dua modalitas.
contrastive_loss	Peran inti dalam menyelaraskan representasi dari dua modalitas.
temperature	Peran inti dalam menyelaraskan representasi dari dua modalitas.

Checklist siap pakai

- Verifikasi encoder sebelum eksekusi utama.
- Catat perubahan pada projection dan contrastive_loss.
- Ukur temperature dengan baseline yang adil.
- Tambahkan test untuk mencegah: label positif tidak selaras antar batch.
- Simpan artefak proyek: Pencarian citra-teks mini.

API yang muncul

torch.nn.functional.normalize, torch.arange, torch.nn.functional.cross_entropy

SATU KALIMAT Bab 40 mengajarkan cara menyelaraskan representasi dari dua modalitas dengan kontrak eksplisit dan bukti terukur.

BAGIAN 11 - DEPLOYMENT DAN MLOPS

Bab 41. torch.export dan ONNX

Menghasilkan representasi program untuk deployment.

Hasil belajar

- Menjelaskan peran export dan hubungannya dengan graph.
- Menulis notebook Colab untuk menghasilkan representasi program untuk deployment.
- Mendiagnosis risiko: contoh input tidak mewakili bentuk dinamis.
- Menghasilkan proyek mini: Ekspor dan verifikasi model.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: export, graph, dynamic_shapes, ONNX.

Parameter	Target
Level	Advanced
Waktu	60-120 menit
Artefak	Ekspor dan verifikasi model
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 41

torch.export dan ONNX



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah menghasilkan representasi program untuk deployment. Alur di atas memperlakukan export sebagai input keputusan, graph sebagai transformasi utama, dynamic_shapes sebagai state atau mekanisme kontrol, dan ONNX sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika graph berubah, bagaimana dampaknya terhadap ONNX? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk menghasilkan representasi program untuk deployment. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 model = build_model().eval()
3 example = (torch.randn(2, 3, 224, 224),)
4 exported = torch.export.export(model, example)
5 print(exported.graph_module.graph)
6 torch.onnx.export(model, example, 'model.onnx',
7                   input_names=['image'], output_names=['logits'],
8                   dynamo=True)
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi ONNX dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.randn	Mewakili export; catat input, output, dtype, device, serta state yang berubah.
torch.export.export	Mewakili graph; catat input, output, dtype, device, serta state yang berubah.
torch.onnx.export	Mewakili dynamic_shapes; catat input, output, dtype, device, serta state yang berubah.
True	Mewakili ONNX; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Contoh input tidak mewakili bentuk dinamis.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 41

torch.export dan ONNX

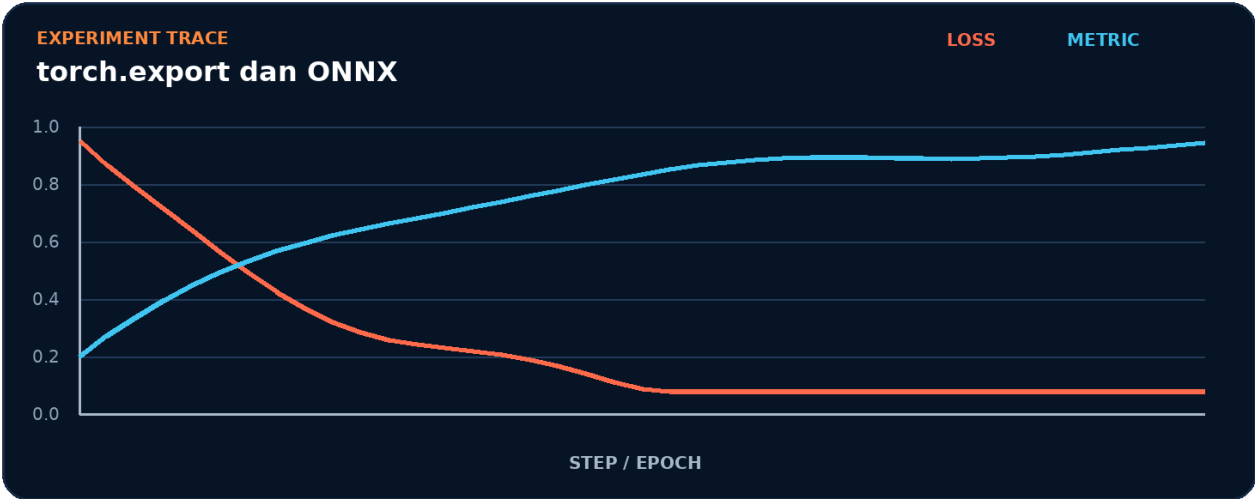


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk export.
2. Terapkan transformasi utama: graph.
3. Kelola state atau kontrol melalui dynamic_shapes.
4. Ukur hasil akhir memakai ONNX.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur export -> graph -> dynamic_shapes -> ONNX tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk ONNX.
- 2. Ubah satu nilai yang memengaruhi graph.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada graph akan memengaruhi ONNX.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	ONNX
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah export menjadi bottleneck.
- Pisahkan biaya setup dari steady-state graph.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Contoh input tidak mewakili bentuk dinamis.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait export.
3. Isolasi	Nonaktifkan sementara graph dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Ekspor dan verifikasi model

Bangun artefak kecil yang membuktikan Anda dapat menghasilkan representasi program untuk deployment. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk ONNX.
2. Implementasikan baseline memakai export dan graph.
3. Tambahkan logging untuk `dynamic_shapes`.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan export tanpa menggunakan istilah graph.
2. Apa shape, dtype, dan device yang diharapkan pada batas graph?
3. Kapan dynamic_shapes berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas ONNX?
5. Bagaimana Anda mereproduksi kegagalan: contoh input tidak mewakili bentuk dinamis?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Ekspor dan verifikasi model?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk graph, lalu buat tabel yang membandingkan ONNX, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
export	Peran inti dalam menghasilkan representasi program untuk deployment.
graph	Peran inti dalam menghasilkan representasi program untuk deployment.
dynamic_shapes	Peran inti dalam menghasilkan representasi program untuk deployment.
ONNX	Peran inti dalam menghasilkan representasi program untuk deployment.

Checklist siap pakai

- Verifikasi export sebelum eksekusi utama.
- Catat perubahan pada graph dan dynamic_shapes.
- Ukur ONNX dengan baseline yang adil.
- Tambahkan test untuk mencegah: contoh input tidak mewakili bentuk dinamis.
- Simpan artefak proyek: Ekspor dan verifikasi model.

API yang muncul

torch.randn, torch.export.export, torch.onnx.export, True

SATU KALIMAT Bab 41 mengajarkan cara menghasilkan representasi program untuk deployment dengan kontrak eksplisit dan bukti terukur.

BAGIAN 11 - DEPLOYMENT DAN MLOPS

Bab 42. Edge AI dengan ExecuTorch

Menyiapkan model untuk perangkat terbatas.

Hasil belajar

- Menjelaskan peran edge dan hubungannya dengan lowering.
- Menulis notebook Colab untuk menyiapkan model untuk perangkat terbatas.
- Mendiagnosis risiko: operator model tidak didukung backend target.
- Menghasilkan proyek mini: Checklist deployment mobile.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: edge, lowering, delegate, runtime.

Parameter	Target
Level	Advanced
Waktu	60-120 menit
Artefak	Checklist deployment mobile
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 42

Edge AI dengan ExecuTorch



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah menyiapkan model untuk perangkat terbatas. Alur di atas memperlakukan edge sebagai input keputusan, lowering sebagai transformasi utama, delegate sebagai state atau mekanisme kontrol, dan runtime sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika lowering berubah, bagaimana dampaknya terhadap runtime? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk menyiapkan model untuk perangkat terbatas. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from executorch.exir import to_edge
3 model = build_model().eval()
4 example = (torch.randn(1, 3, 224, 224),)
5 exported = torch.export.export(model, example)
6 edge_program = to_edge(exported)
7 program = edge_program.to_executorch()
8 with open('model.pt', 'wb') as f:
9     f.write(program.buffer)
10 print('ExecuTorch bytes:', len(program.buffer))
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi runtime dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.randn	Mewakili edge; catat input, output, dtype, device, serta state yang berubah.
torch.export.export	Mewakili lowering; catat input, output, dtype, device, serta state yang berubah.
ExecuTorch	Mewakili delegate; catat input, output, dtype, device, serta state yang berubah.
torch.randn	Mewakili runtime; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Operator model tidak didukung backend target.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 42

Edge AI dengan ExecuTorch

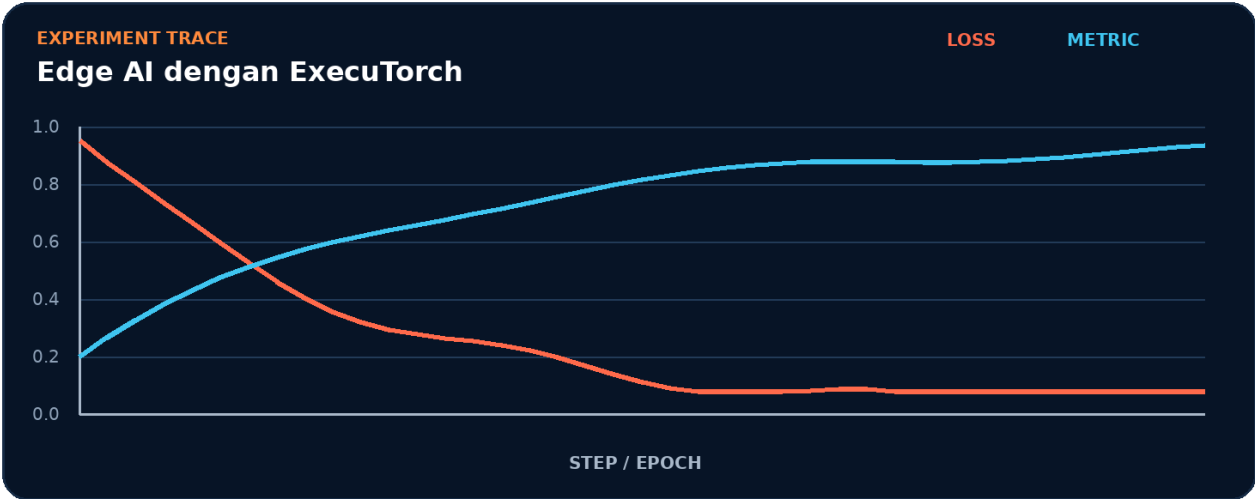


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk edge.
2. Terapkan transformasi utama: lowering.
3. Kelola state atau kontrol melalui delegate.
4. Ukur hasil akhir memakai runtime.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur edge -> lowering -> delegate -> runtime tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk runtime.
- 2. Ubah satu nilai yang memengaruhi lowering.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada lowering akan memengaruhi runtime.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	runtime
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah edge menjadi bottleneck.
- Pisahkan biaya setup dari steady-state lowering.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Operator model tidak didukung backend target.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait edge.
3. Isolasi	Nonaktifkan sementara lowering dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Checklist deployment mobile

Bangun artefak kecil yang membuktikan Anda dapat menyiapkan model untuk perangkat terbatas. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk runtime.
2. Implementasikan baseline memakai edge dan lowering.
3. Tambahkan logging untuk delegate.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan edge tanpa menggunakan istilah lowering.
2. Apa shape, dtype, dan device yang diharapkan pada batas lowering?
3. Kapan delegate berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas runtime?
5. Bagaimana Anda mereproduksi kegagalan: operator model tidak didukung backend target?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Checklist deployment mobile?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk lowering, lalu buat tabel yang membandingkan runtime, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
edge	Peran inti dalam menyiapkan model untuk perangkat terbatas.
lowering	Peran inti dalam menyiapkan model untuk perangkat terbatas.
delegate	Peran inti dalam menyiapkan model untuk perangkat terbatas.
runtime	Peran inti dalam menyiapkan model untuk perangkat terbatas.

Checklist siap pakai

- Verifikasi edge sebelum eksekusi utama.
- Catat perubahan pada lowering dan delegate.
- Ukur runtime dengan baseline yang adil.
- Tambahkan test untuk mencegah: operator model tidak didukung backend target.
- Simpan artefak proyek: Checklist deployment mobile.

API yang muncul

torch.randn, torch.export.export, ExecuTorch

SATU KALIMAT Bab 42 mengajarkan cara menyiapkan model untuk perangkat terbatas dengan kontrak eksplisit dan bukti terukur.

BAGIAN 11 - DEPLOYMENT DAN MLOPS

Bab 43. Serving Model sebagai API

Membangun layanan inference dengan batching dan validasi.

Hasil belajar

- Menjelaskan peran API dan hubungannya dengan batching.
- Menulis notebook Colab untuk membangun layanan inference dengan batching dan validasi.
- Mendiagnosis risiko: model dimuat ulang untuk setiap request.
- Menghasilkan proyek mini: Service inference minimal.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: API, batching, latency, validation.

Parameter	Target
Level	Advanced
Waktu	60-120 menit
Artefak	Service inference minimal
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 43

Serving Model sebagai API



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah membangun layanan inference dengan batching dan validasi. Alur di atas memperlakukan API sebagai input keputusan, batching sebagai transformasi utama, latency sebagai state atau mekanisme kontrol, dan validation sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika batching berubah, bagaimana dampaknya terhadap validation? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk membangun layanan inference dengan batching dan validasi. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 from fastapi import FastAPI
2 from pydantic import BaseModel
3 import torch
4 app = FastAPI(); model = load_model().eval()
5 class Request(BaseModel): features: list[float]
6 @app.post('/predict')
7 def predict(req: Request):
8     x = torch.tensor([req.features], dtype=torch.float32)
9     with torch.inference_mode(): probs = model(x).softmax(-1)[0]
10    return {'class_id': int(probs.argmax()), 'confidence': float(probs.max())}
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi validation dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
FastAPI	Mewakili API; catat input, output, dtype, device, serta state yang berubah.
BaseModel	Mewakili batching; catat input, output, dtype, device, serta state yang berubah.
Request	Mewakili latency; catat input, output, dtype, device, serta state yang berubah.
torch.tensor	Mewakili validation; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Model dimuat ulang untuk setiap request.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 43

Serving Model sebagai API

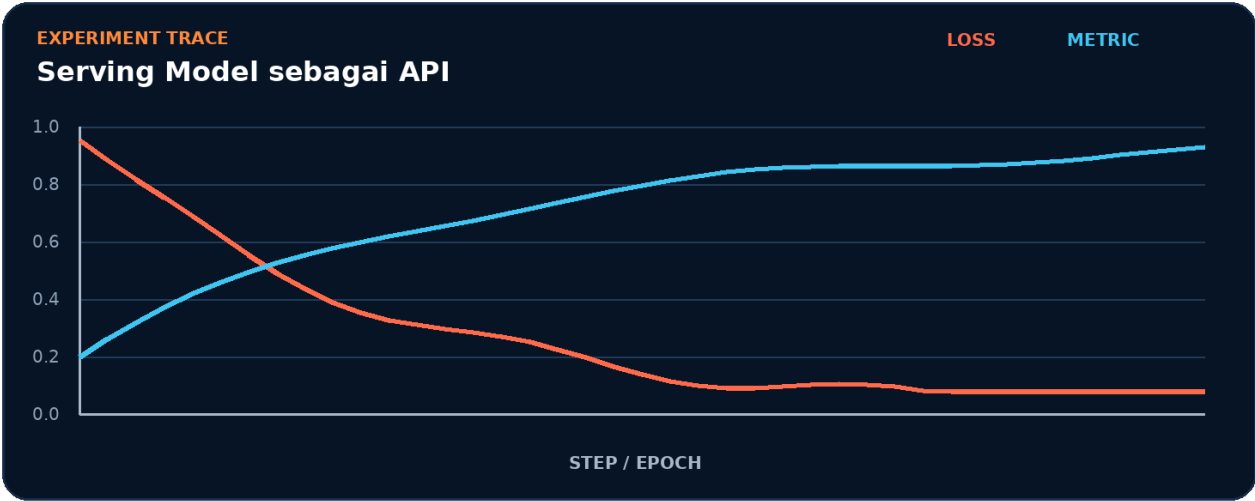


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk API.
2. Terapkan transformasi utama: batching.
3. Kelola state atau kontrol melalui latency.
4. Ukur hasil akhir memakai validation.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur API -> batching -> latency -> validation tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk validation.
- 2. Ubah satu nilai yang memengaruhi batching.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada batching akan memengaruhi validation.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	validation
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah API menjadi bottleneck.
- Pisahkan biaya setup dari steady-state batching.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Model dimuat ulang untuk setiap request.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait API.
3. Isolasi	Nonaktifkan sementara batching dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Service inference minimal

Bangun artefak kecil yang membuktikan Anda dapat membangun layanan inference dengan batching dan validasi. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk validation.
2. Implementasikan baseline memakai API dan batching.
3. Tambahkan logging untuk latency.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan API tanpa menggunakan istilah batching.
2. Apa shape, dtype, dan device yang diharapkan pada batas batching?
3. Kapan latency berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas validation?
5. Bagaimana Anda mereproduksi kegagalan: model dimuat ulang untuk setiap request?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Service inference minimal?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk batching, lalu buat tabel yang membandingkan validation, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
API	Peran inti dalam membangun layanan inference dengan batching dan validasi.
batching	Peran inti dalam membangun layanan inference dengan batching dan validasi.
latency	Peran inti dalam membangun layanan inference dengan batching dan validasi.
validation	Peran inti dalam membangun layanan inference dengan batching dan validasi.

Checklist siap pakai

- Verifikasi API sebelum eksekusi utama.
- Catat perubahan pada batching dan latency.
- Ukur validation dengan baseline yang adil.
- Tambahkan test untuk mencegah: model dimuat ulang untuk setiap request.
- Simpan artefak proyek: Service inference minimal.

API yang muncul

FastAPI, BaseModel, Request, torch.tensor, torch.float32, torch.inference_mode

SATU KALIMAT Bab 43 mengajarkan cara membangun layanan inference dengan batching dan validasi dengan kontrak eksplisit dan bukti terukur.

BAGIAN 11 - DEPLOYMENT DAN MLOPS

Bab 44. Monitoring, Testing, dan CI/CD

Menjaga kualitas model setelah dirilis.

Hasil belajar

- Menjelaskan peran unit_test dan hubungannya dengan drift.
- Menulis notebook Colab untuk menjaga kualitas model setelah dirilis.
- Mendiagnosis risiko: hanya memonitor uptime tanpa kualitas prediksi.
- Menghasilkan proyek mini: Quality gate model.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: unit_test, drift, latency_SLO, pipeline.

Parameter	Target
Level	Advanced
Waktu	60-120 menit
Artefak	Quality gate model
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 44

Monitoring, Testing, dan CI/CD



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah menjaga kualitas model setelah dirilis. Alur di atas memperlakukan `unit_test` sebagai input keputusan, `drift` sebagai transformasi utama, `latency_SLO` sebagai state atau mekanisme kontrol, dan `pipeline` sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika drift berubah, bagaimana dampaknya terhadap pipeline? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk menjaga kualitas model setelah dirilis. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 def test_model_contract():
2     model = build_model().eval(); x = torch.randn(4, 20)
3     with torch.inference_mode(): y = model(x)
4     assert y.shape == (4, 3)
5     assert torch.isfinite(y).all()
6     assert torch.allclose(y, model(x))
7 def drift_score(reference, current):
8     return torch.abs(reference.mean(0) - current.mean(0)).mean().item()
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi pipeline dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.randn	Mewakili unit_test; catat input, output, dtype, device, serta state yang berubah.
torch.inference_mode	Mewakili drift; catat input, output, dtype, device, serta state yang berubah.
torch.isfinite	Mewakili latency_SLO; catat input, output, dtype, device, serta state yang berubah.
torch.allclose	Mewakili pipeline; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Hanya memonitor uptime tanpa kualitas prediksi.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 44

Monitoring, Testing, dan CI/CD

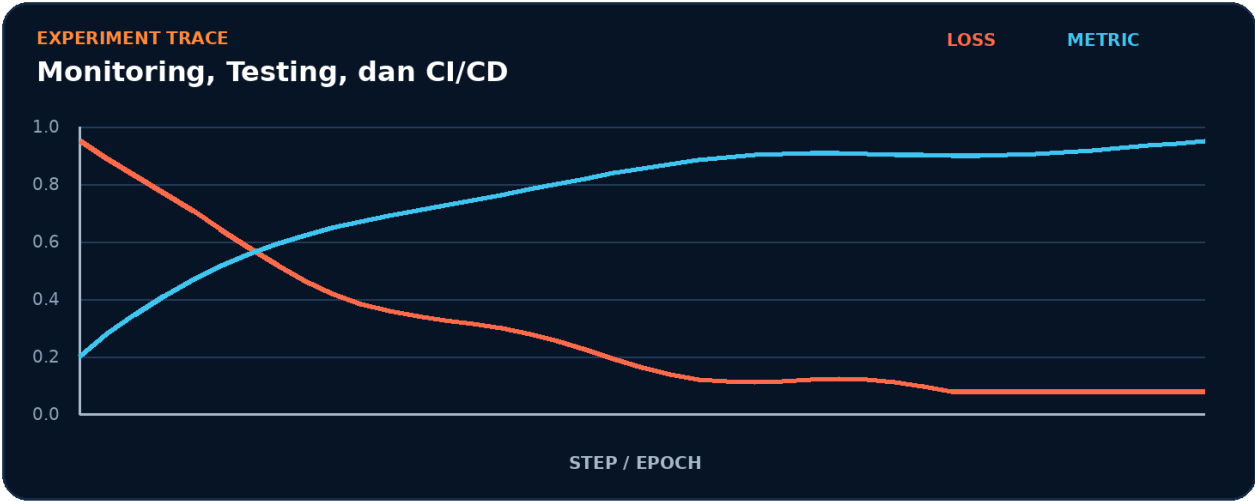


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk `unit_test`.
2. Terapkan transformasi utama: `drift`.
3. Kelola state atau kontrol melalui `latency_SLO`.
4. Ukur hasil akhir memakai `pipeline`.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur `unit_test` -> `drift` -> `latency_SLO` -> `pipeline` tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk pipeline.
- 2. Ubah satu nilai yang memengaruhi drift.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada drift akan memengaruhi pipeline.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	pipeline
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah unit_test menjadi bottleneck.
- Pisahkan biaya setup dari steady-state drift.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Hanya memonitor uptime tanpa kualitas prediksi.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait unit_test.
3. Isolasi	Nonaktifkan sementara drift dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Quality gate model

Bangun artefak kecil yang membuktikan Anda dapat menjaga kualitas model setelah dirilis. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk pipeline.
2. Implementasikan baseline memakai `unit_test` dan drift.
3. Tambahkan logging untuk `latency_SLO`.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan `unit_test` tanpa menggunakan istilah drift.
2. Apa `shape`, `dtype`, dan `device` yang diharapkan pada batas drift?
3. Kapan `latency_SLO` berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas pipeline?
5. Bagaimana Anda mereproduksi kegagalan: hanya memonitor uptime tanpa kualitas prediksi?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Quality gate model?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk drift, lalu buat tabel yang membandingkan pipeline, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
unit_test	Peran inti dalam menjaga kualitas model setelah dirilis.
drift	Peran inti dalam menjaga kualitas model setelah dirilis.
latency_SLO	Peran inti dalam menjaga kualitas model setelah dirilis.
pipeline	Peran inti dalam menjaga kualitas model setelah dirilis.

Checklist siap pakai

- Verifikasi `unit_test` sebelum eksekusi utama.
- Catat perubahan pada drift dan `latency_SLO`.
- Ukur pipeline dengan baseline yang adil.
- Tambahkan test untuk mencegah: hanya memonitor uptime tanpa kualitas prediksi.
- Simpan artefak proyek: Quality gate model.

API yang muncul

`torch.randn`, `torch.inference_mode`, `torch.isfinite`, `torch.allclose`, `torch.abs`

SATU KALIMAT Bab 44 mengajarkan cara menjaga kualitas model setelah dirilis dengan kontrak eksplisit dan bukti terukur.

BAGIAN 12 - RISET, KEAMANAN, DAN CAPSTONE

Bab 45. Interpretability dengan Captum

Menghubungkan prediksi dengan kontribusi fitur.

Hasil belajar

- Menjelaskan peran attribution dan hubungannya dengan IntegratedGradients.
- Menulis notebook Colab untuk menghubungkan prediksi dengan kontribusi fitur.
- Mendiagnosis risiko: baseline tidak sesuai konteks data.
- Menghasilkan proyek mini: Laporan atribusi fitur.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: attribution, IntegratedGradients, baseline, sanity_check.

Parameter	Target
Level	Advanced
Waktu	60-120 menit
Artefak	Laporan atribusi fitur
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 45

Interpretability dengan Captum



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah menghubungkan prediksi dengan kontribusi fitur. Alur di atas memperlakukan attribution sebagai input keputusan, IntegratedGradients sebagai transformasi utama, baseline sebagai state atau mekanisme kontrol, dan sanity_check sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika IntegratedGradients berubah, bagaimana dampaknya terhadap sanity_check? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk menghubungkan prediksi dengan kontribusi fitur. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 from captum.attr import IntegratedGradients
2 model.eval(); x = sample.unsqueeze(0).requires_grad_()
3 ig = IntegratedGradients(model)
4 attribution, delta = ig.attribute(x, baselines=torch.zeros_like(x),
5                                 target=target_class, return_convergence_delta=True)
6 print('attribution:', attribution.shape)
7 print('convergence delta:', float(delta))
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi sanity_check dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
IntegratedGradients	Mewakili attribution; catat input, output, dtype, device, serta state yang berubah.
torch.zeros_like	Mewakili IntegratedGradients; catat input, output, dtype, device, serta state yang berubah.
True	Mewakili baseline; catat input, output, dtype, device, serta state yang berubah.
IntegratedGradients	Mewakili sanity_check; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Baseline tidak sesuai konteks data.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 45

Interpretability dengan Captum

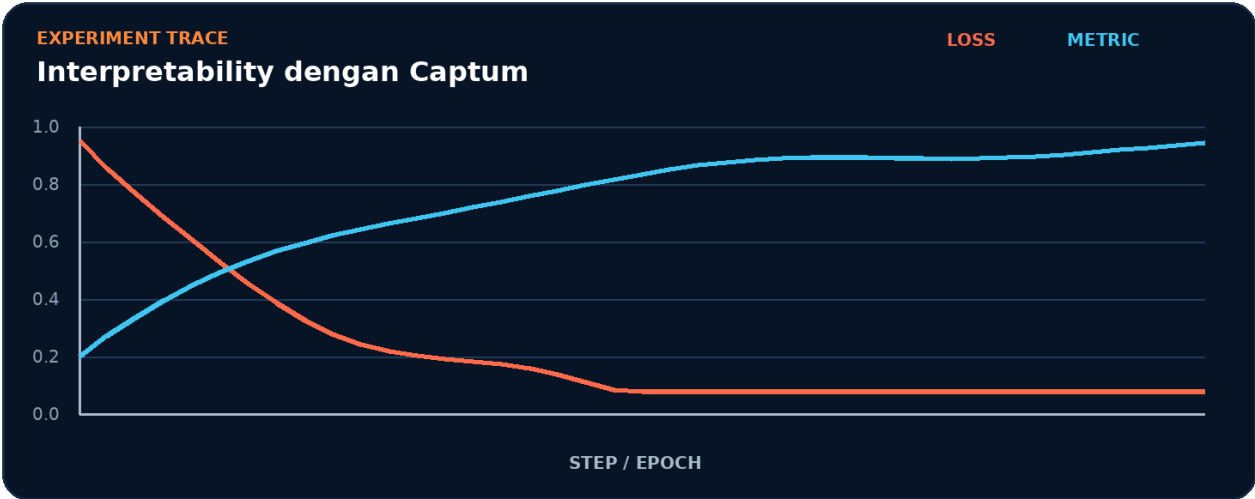


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk attribution.
2. Terapkan transformasi utama: IntegratedGradients.
3. Kelola state atau kontrol melalui baseline.
4. Ukur hasil akhir memakai sanity_check.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur attribution -> IntegratedGradients -> baseline -> sanity_check tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk sanity_check.
- 2. Ubah satu nilai yang memengaruhi IntegratedGradients.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada IntegratedGradients akan memengaruhi sanity_check.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	sanity_check
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah attribution menjadi bottleneck.
- Pisahkan biaya setup dari steady-state IntegratedGradients.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Baseline tidak sesuai konteks data.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait attribution.
3. Isolasi	Nonaktifkan sementara IntegratedGradients dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Laporan atribusi fitur

Bangun artefak kecil yang membuktikan Anda dapat menghubungkan prediksi dengan kontribusi fitur. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk `sanity_check`.
2. Implementasikan baseline memakai `attribution` dan `IntegratedGradients`.
3. Tambahkan logging untuk baseline.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan attribution tanpa menggunakan istilah IntegratedGradients.
2. Apa shape, dtype, dan device yang diharapkan pada batas IntegratedGradients?
3. Kapan baseline berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas sanity_check?
5. Bagaimana Anda mereproduksi kegagalan: baseline tidak sesuai konteks data?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Laporan atribusi fitur?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk IntegratedGradients, lalu buat tabel yang membandingkan sanity_check, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
attribution	Peran inti dalam menghubungkan prediksi dengan kontribusi fitur.
IntegratedGradients	Peran inti dalam menghubungkan prediksi dengan kontribusi fitur.
baseline	Peran inti dalam menghubungkan prediksi dengan kontribusi fitur.
sanity_check	Peran inti dalam menghubungkan prediksi dengan kontribusi fitur.

Checklist siap pakai

- Verifikasi attribution sebelum eksekusi utama.
- Catat perubahan pada IntegratedGradients dan baseline.
- Ukur sanity_check dengan baseline yang adil.
- Tambahkan test untuk mencegah: baseline tidak sesuai konteks data.
- Simpan artefak proyek: Laporan atribusi fitur.

API yang muncul

IntegratedGradients, torch.zeros_like, True

SATU KALIMAT Bab 45 mengajarkan cara menghubungkan prediksi dengan kontribusi fitur dengan kontrak eksplisit dan bukti terukur.

BAGIAN 12 - RISET, KEAMANAN, DAN CAPSTONE

Bab 46. Robustness, Adversarial Testing, dan Fairness

Menguji perilaku model di luar data ideal.

Hasil belajar

- Menjelaskan peran robustness dan hubungannya dengan perturbation.
- Menulis notebook Colab untuk menguji perilaku model di luar data ideal.
- Mendiagnosis risiko: metrik agregat menyembunyikan kegagalan subkelompok.
- Menghasilkan proyek mini: Kartu risiko model.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: robustness, perturbation, subgroup, calibration.

Parameter	Target
Level	Advanced
Waktu	60-120 menit
Artefak	Kartu risiko model
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 46

Robustness, Adversarial Testing, dan Fairness



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah menguji perilaku model di luar data ideal. Alur di atas memperlakukan robustness sebagai input keputusan, perturbation sebagai transformasi utama, subgroup sebagai state atau mekanisme kontrol, dan calibration sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika perturbation berubah, bagaimana dampaknya terhadap calibration? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk menguji perilaku model di luar data ideal. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 def fgsm(model, x, y, eps=0.01):
2     x = x.detach().clone().requires_grad_()
3     loss = torch.nn.functional.cross_entropy(model(x), y)
4     grad, = torch.autograd.grad(loss, x)
5     return (x + eps*grad.sign()).detach()
6 adv = fgsm(model, x, y)
7 with torch.inference_mode():
8     clean_acc=(model(x).argmax(1)==y).float().mean()
9     adv_acc=(model(adv).argmax(1)==y).float().mean()
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi calibration dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.nn.functional.cross_entropy	Mewakili robustness; catat input, output, dtype, device, serta state yang berubah.
torch.autograd.grad	Mewakili perturbation; catat input, output, dtype, device, serta state yang berubah.
torch.inference_mode	Mewakili subgroup; catat input, output, dtype, device, serta state yang berubah.
torch.nn.functional.cross_entropy	Mewakili calibration; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Metrik agregat menyembunyikan kegagalan subkelompok.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 46

Robustness, Adversarial Testing, dan Fairness

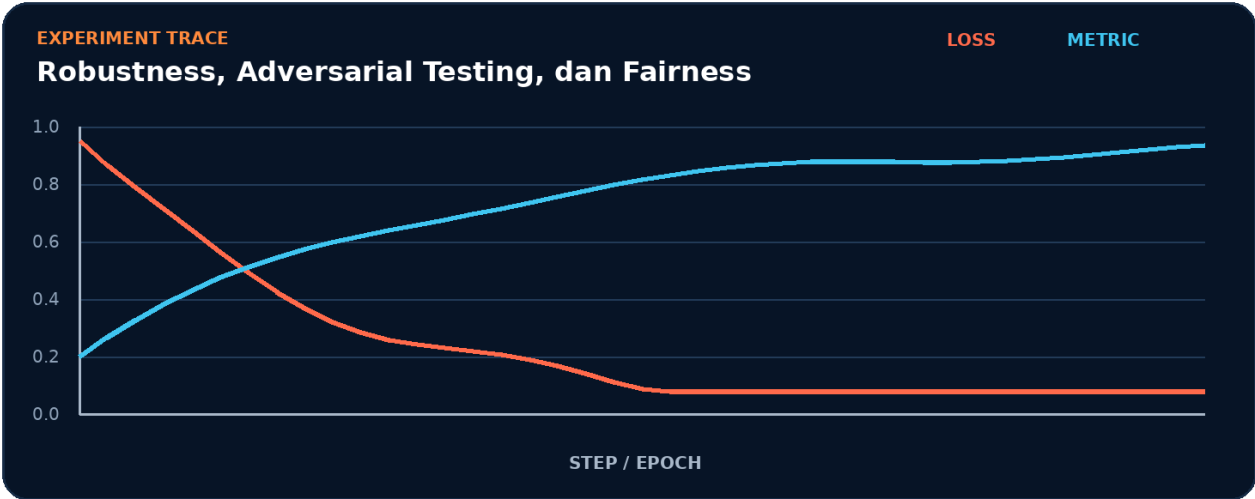


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk robustness.
2. Terapkan transformasi utama: perturbation.
3. Kelola state atau kontrol melalui subgroup.
4. Ukur hasil akhir memakai calibration.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur robustness -> perturbation -> subgroup -> calibration tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk calibration.
- 2. Ubah satu nilai yang memengaruhi perturbation.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada perturbation akan memengaruhi calibration.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	calibration
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah robustness menjadi bottleneck.
- Pisahkan biaya setup dari steady-state perturbation.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Metrik agregat menyembunyikan kegagalan subkelompok.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait robustness.
3. Isolasi	Nonaktifkan sementara perturbation dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Kartu risiko model

Bangun artefak kecil yang membuktikan Anda dapat menguji perilaku model di luar data ideal. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk calibration.
2. Implementasikan baseline memakai robustness dan perturbation.
3. Tambahkan logging untuk subgroup.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan robustness tanpa menggunakan istilah perturbation.
2. Apa shape, dtype, dan device yang diharapkan pada batas perturbation?
3. Kapan subgroup berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas calibration?
5. Bagaimana Anda mereproduksi kegagalan: metrik agregat menyembunyikan kegagalan subkelompok?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Kartu risiko model?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk perturbation, lalu buat tabel yang membandingkan calibration, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
robustness	Peran inti dalam menguji perilaku model di luar data ideal.
perturbation	Peran inti dalam menguji perilaku model di luar data ideal.
subgroup	Peran inti dalam menguji perilaku model di luar data ideal.
calibration	Peran inti dalam menguji perilaku model di luar data ideal.

Checklist siap pakai

- Verifikasi robustness sebelum eksekusi utama.
- Catat perubahan pada perturbation dan subgroup.
- Ukur calibration dengan baseline yang adil.
- Tambahkan test untuk mencegah: metrik agregat menyembunyikan kegagalan subkelompok.
- Simpan artefak proyek: Kartu risiko model.

API yang muncul

torch.nn.functional.cross_entropy, torch.autograd.grad, torch.inference_mode

SATU KALIMAT Bab 46 mengajarkan cara menguji perilaku model di luar data ideal dengan kontrak eksplisit dan bukti terukur.

BAGIAN 12 - RISET, KEAMANAN, DAN CAPSTONE

Bab 47. Custom Operator dan Ekstensi

Memperluas pytorch ketika operator standar tidak cukup.

Hasil belajar

- Menjelaskan peran custom_op dan hubungannya dengan compile.
- Menulis notebook Colab untuk memperluas PyTorch ketika operator standar tidak cukup.
- Mendiagnosis risiko: operator cepat tetapi gradien atau fallback CPU salah.
- Menghasilkan proyek mini: Operator kustom tervalidasi.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: custom_op, compile, autograd, benchmark.

Parameter	Target
Level	Advanced
Waktu	60-120 menit
Artefak	Operator kustom tervalidasi
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 47

Custom Operator dan Ekstensi



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah memperluas PyTorch ketika operator standar tidak cukup. Alur di atas memperlakukan `custom_op` sebagai input keputusan, `compile` sebagai transformasi utama, `autograd` sebagai state atau mekanisme kontrol, dan `benchmark` sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika `compile` berubah, bagaimana dampaknya terhadap `benchmark`? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk memperluas PyTorch ketika operator standar tidak cukup. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 import torch
2 from torch.library import custom_op
3 @custom_op('handbook::scaled_relu', mutates_args=())
4 def scaled_relu(x: torch.Tensor, scale: float) -> torch.Tensor:
5     return torch.relu(x) * scale
6 @scaled_relu.register_fake
7 def _(x, scale): return torch.empty_like(x)
8 x = torch.randn(8, requires_grad=False)
9 print(scaled_relu(x, 0.5))
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi benchmark dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
torch.library	Mewakili custom_op; catat input, output, dtype, device, serta state yang berubah.
torch.Tensor	Mewakili compile; catat input, output, dtype, device, serta state yang berubah.
torch.relu	Mewakili autograd; catat input, output, dtype, device, serta state yang berubah.
torch.empty_like	Mewakili benchmark; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Operator cepat tetapi gradien atau fallback cpu salah.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 47

Custom Operator dan Ekstensi

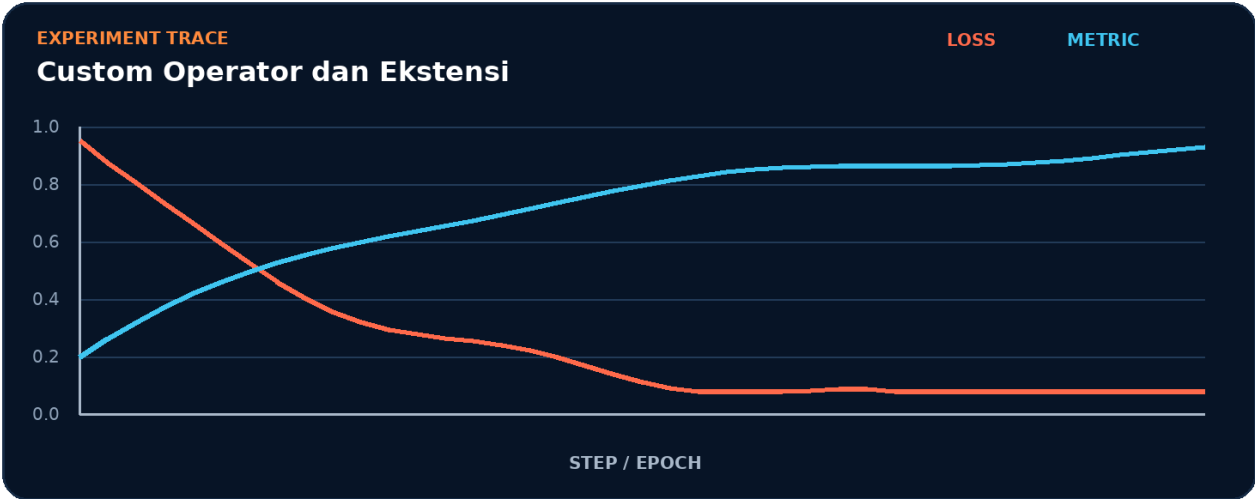


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk custom_op.
2. Terapkan transformasi utama: compile.
3. Kelola state atau kontrol melalui autograd.
4. Ukur hasil akhir memakai benchmark.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur custom_op -> compile -> autograd -> benchmark tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk benchmark.
- 2. Ubah satu nilai yang memengaruhi compile.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada compile akan memengaruhi benchmark.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	benchmark
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah custom_op menjadi bottleneck.
- Pisahkan biaya setup dari steady-state compile.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Operator cepat tetapi gradien atau fallback cpu salah.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait custom_op.
3. Isolasi	Nonaktifkan sementara compile dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Operator kustom tervalidasi

Bangun artefak kecil yang membuktikan Anda dapat memperluas PyTorch ketika operator standar tidak cukup. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk benchmark.
2. Implementasikan baseline memakai `custom_op` dan `compile`.
3. Tambahkan logging untuk autograd.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan `custom_op` tanpa menggunakan istilah `compile`.
2. Apa `shape`, `dtype`, dan `device` yang diharapkan pada batas `compile`?
3. Kapan `autograd` berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas benchmark?
5. Bagaimana Anda mereproduksi kegagalan: operator cepat tetapi gradien atau fallback CPU salah?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum profiling?
7. Bagaimana desain checkpoint atau test untuk proyek Operator kustom tervalidasi?
8. Sebutkan satu trade-off antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi baseline agar menerima dua konfigurasi berbeda untuk `compile`, lalu buat tabel yang membandingkan benchmark, waktu eksekusi, dan peak memory. Pertahankan seed serta jumlah step.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak tensor, state, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
custom_op	Peran inti dalam memperluas PyTorch ketika operator standar tidak cukup.
compile	Peran inti dalam memperluas PyTorch ketika operator standar tidak cukup.
autograd	Peran inti dalam memperluas PyTorch ketika operator standar tidak cukup.
benchmark	Peran inti dalam memperluas PyTorch ketika operator standar tidak cukup.

Checklist siap pakai

- Verifikasi custom_op sebelum eksekusi utama.
- Catat perubahan pada compile dan autograd.
- Ukur benchmark dengan baseline yang adil.
- Tambahkan test untuk mencegah: operator cepat tetapi gradien atau fallback CPU salah.
- Simpan artefak proyek: Operator kustom tervalidasi.

API yang muncul

torch.library, torch.Tensor, torch.relu, torch.empty_like, torch.randn, False

SATU KALIMAT Bab 47 mengajarkan cara memperluas PyTorch ketika operator standar tidak cukup dengan kontrak eksplisit dan bukti terukur.

BAGIAN 12 - RISET, KEAMANAN, DAN CAPSTONE

Bab 48. Capstone: Riset ke Produksi

Mengintegrasikan data, training, evaluasi, ekspor, dan monitoring.

Hasil belajar

- Menjelaskan peran problem_framing dan hubungannya dengan baseline.
- Menulis notebook Colab untuk mengintegrasikan data, training, evaluasi, ekspor, dan monitoring.
- Mendiagnosis risiko: optimasi teknis dilakukan sebelum kriteria sukses disepakati.
- Menghasilkan proyek mini: Sistem ML end-to-end.

PRASYARAT Pahami bab sebelumnya, jalankan sel diagnostik, dan siapkan runtime yang sesuai. Kata kunci: problem_framing, baseline, deployment, governance.

Parameter	Target
Level	Advanced
Waktu	60-120 menit
Artefak	Sistem ML end-to-end
Validasi	Shape + metric + reproducibility

Mental Model

MENTAL MODEL 48

Capstone: Riset ke Produksi



Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

Inti bab ini adalah mengintegrasikan data, training, evaluasi, ekspor, dan monitoring. Alur di atas memperlakukan `problem_framing` sebagai input keputusan, `baseline` sebagai transformasi utama, `deployment` sebagai state atau mekanisme kontrol, dan `governance` sebagai hasil yang harus diverifikasi.

PERTANYAAN KUNCI Jika `baseline` berubah, bagaimana dampaknya terhadap `governance`? Jawaban harus didukung bentuk tensor, log, atau metrik.

Setup Google Colab - Step by Step

1. Buka colab.research.google.com dan buat notebook baru.
2. Pilih Runtime > Change runtime type; gunakan GPU hanya bila bab memerlukan CUDA.
3. Jalankan sel diagnostik versi dan device.
4. Salin sel inti bab, jalankan berurutan, lalu simpan baseline.
5. Ubah satu parameter eksperimen dan bandingkan hasil.

COLAB - SETUP CELL

```
1 !pip -q install --upgrade 'torch>=2.11' torchvision
2 import torch
3 print('torch:', torch.__version__)
4 DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
5 print('device:', DEVICE)
```

OUTPUT torch: 2.11+ | device: cuda atau cpu

VERSI Jika API pada bab belum tersedia di runtime bawaan, pin versi eksplisit dan restart session. Jangan mencampur binary torch/vision yang tidak kompatibel.

Notebook Inti

Sel berikut adalah baseline minimal untuk mengintegrasikan data, training, evaluasi, ekspor, dan monitoring. Jalankan tanpa modifikasi terlebih dahulu dan simpan outputnya sebagai pembandingan.

COLAB - BASELINE

```
1 config = {'seed':42, 'lr':3e-4, 'batch_size':128, 'epochs':20}
2 seed_everything(config['seed'])
3 train_loader, valid_loader, test_loader = build_data(config)
4 model = build_model(config).to(device)
5 state = train(model, train_loader, valid_loader, config)
6 metrics = evaluate(state['best_model'], test_loader)
7 artifact = export_model(state['best_model'])
8 run_quality_gates(metrics, artifact)
9 print({'metrics': metrics, 'artifact': artifact})
```

OUTPUT Periksa bahwa sel selesai tanpa exception; validasi governance dan bentuk keluaran.

RUN ALL Notebook yang baik tidak bergantung pada state tersembunyi dari urutan eksekusi acak. Restart session lalu jalankan Run all.

Bedah Kode dan Kontrak

Simbol / konsep	Kontrak yang harus diperiksa
problem_framing	Mewakili problem_framing; catat input, output, dtype, device, serta state yang berubah.
baseline	Mewakili baseline; catat input, output, dtype, device, serta state yang berubah.
deployment	Mewakili deployment; catat input, output, dtype, device, serta state yang berubah.
governance	Mewakili governance; catat input, output, dtype, device, serta state yang berubah.
shape contract	Tuliskan bentuk tensor pada batas data, model, loss, dan metric.
state contract	Identifikasi parameter, buffer, optimizer state, RNG, dan checkpoint.
failure signal	Optimasi teknis dilakukan sebelum kriteria sukses disepakati.

Cara membaca

- Mulai dari sumber data dan ikuti tensor ke output.
- Tandai setiap perpindahan dtype atau device.
- Bedakan operasi yang membuat gradien dari operasi evaluasi.
- Pastikan metrik dihitung pada unit dan subset data yang tepat.

Ilustrasi Step by Step

MENTAL MODEL 48

Capstone: Riset ke Produksi

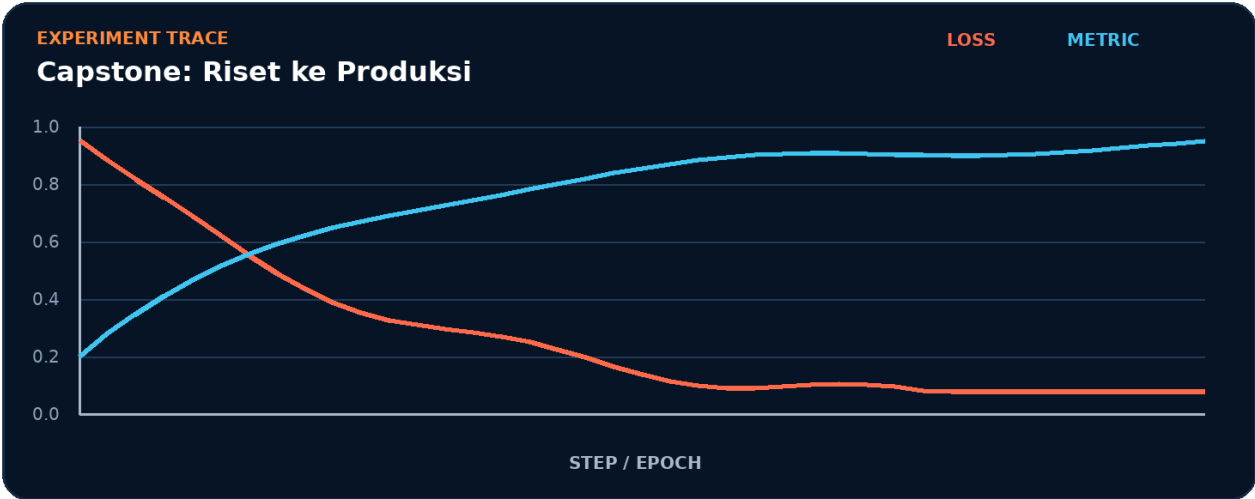


Alur dibaca dari kiri ke kanan. Validasi bentuk, state, dan metrik pada setiap batas.

1. Bentuk representasi awal untuk `problem_framing`.
2. Terapkan transformasi utama: `baseline`.
3. Kelola state atau kontrol melalui `deployment`.
4. Ukur hasil akhir memakai `governance`.

CHECKPOINT PEMAHAMAN Anda harus dapat menggambar ulang alur `problem_framing` -> `baseline` -> `deployment` -> `governance` tanpa melihat halaman ini.

Eksperimen Terpandu



- 1. Simpan baseline untuk governance.
- 2. Ubah satu nilai yang memengaruhi baseline.
- 3. Jalankan jumlah step yang sama dan catat loss, metric, waktu, serta memori.
- 4. Kembalikan konfigurasi awal sebelum mencoba variabel kedua.
- 5. Tulis kesimpulan yang membedakan korelasi dari sebab-akibat.

Komponen	Isi
Hipotesis	Perubahan pada baseline akan memengaruhi governance.
Variabel kontrol	Seed, split data, batch, jumlah step, device.
Metrik utama	governance
Metrik guardrail	Waktu, peak memory, stabilitas numerik.

Kinerja dan Profiling

Klaim performa harus berasal dari pengukuran yang adil. Gunakan warm-up, jumlah pengulangan cukup, dan sinkronisasi CUDA sebelum serta sesudah timer.

COLAB - MICROBENCHMARK

```
1 import time, torch
2 def timed(fn, repeats=30):
3     for _ in range(5): fn()
4     if torch.cuda.is_available(): torch.cuda.synchronize()
5     start = time.perf_counter()
6     for _ in range(repeats): fn()
7     if torch.cuda.is_available(): torch.cuda.synchronize()
8     return 1000*(time.perf_counter()-start)/repeats
9 print('ms/step:', timed(step_fn))
```

- Ukur apakah problem_framing menjadi bottleneck.
- Pisahkan biaya setup dari steady-state baseline.
- Catat batch size, dtype, device, dan versi PyTorch.
- Laporkan median atau distribusi, bukan satu angka tunggal.

Failure Mode dan Debugging

GEJALA UTAMA Optimasi teknis dilakukan sebelum kriteria sukses disepakati.

Langkah	Tindakan
1. Reproduksi	Kecilkan data dan model sampai kegagalan muncul cepat.
2. Observasi	Cetak shape, dtype, device, dan nilai yang terkait problem_framing.
3. Isolasi	Nonaktifkan sementara baseline dan bandingkan perilaku.
4. Assertion	Tambahkan pemeriksaan finite, rentang, dan bentuk.
5. Perbaiki	Ubah penyebab paling dekat, bukan menutupi exception.
6. Regression test	Simpan input minimal agar bug tidak kembali.

COLAB - DEBUG CELL

```
1 def inspect(name, x):
2     print(name, 'shape=', tuple(x.shape), 'dtype=', x.dtype, 'device=', x.device)
3     assert torch.isfinite(x).all(), f'{name} contains NaN/Inf'
4 inspect('input', x)
5 with torch.autograd.detect_anomaly(check_nan=True):
6     loss = step_fn(); loss.backward()
```

Mini Project: Sistem ML end-to-end

Bangun artefak kecil yang membuktikan Anda dapat mengintegrasikan data, training, evaluasi, ekspor, dan monitoring. Proyek harus dapat dijalankan dari runtime bersih dan menghasilkan ringkasan hasil yang dapat diperiksa.

1. Definisikan input, target, dan batas keberhasilan untuk governance.
2. Implementasikan baseline memakai `problem_framing` dan `baseline`.
3. Tambahkan logging untuk deployment.
4. Lakukan satu eksperimen terkontrol dan bandingkan dengan baseline.
5. Simpan config, metric, diagram, dan checkpoint bila relevan.
6. Tulis README singkat: tujuan, cara menjalankan, hasil, keterbatasan.

Rubrik	Kriteria
Correctness	Tidak ada error; shape dan dtype tervalidasi.
Reproducibility	Run all dari runtime bersih menghasilkan pola hasil sebanding.
Evidence	Ada metric, log, atau profil yang mendukung kesimpulan.
Communication	Notebook memiliki heading, komentar, dan ringkasan keputusan.

Latihan dan Knowledge Check

1. Jelaskan `problem_framing` tanpa menggunakan istilah `baseline`.
2. Apa `shape`, `dtype`, dan `device` yang diharapkan pada batas `baseline`?
3. Kapan `deployment` berubah dan siapa yang menyimpannya?
4. Metrik apa yang membuktikan kualitas `governance`?
5. Bagaimana Anda mereproduksi kegagalan: optimasi teknis dilakukan sebelum kriteria sukses disepakati?
6. Optimasi apa yang sebaiknya tidak dilakukan sebelum `profiling`?
7. Bagaimana desain `checkpoint` atau `test` untuk proyek Sistem ML `end-to-end`?
8. Sebutkan satu `trade-off` antara kecepatan, memori, dan akurasi pada bab ini.

Tantangan kode

Modifikasi `baseline` agar menerima dua konfigurasi berbeda untuk `baseline`, lalu buat tabel yang membandingkan `governance`, waktu eksekusi, dan `peak memory`. Pertahankan `seed` serta jumlah `step`.

KRITERIA JAWABAN Jawaban kuat menyebut kontrak `tensor`, `state`, pengukuran, dan risiko implementasi - bukan hanya nama API.

Ringkasan dan Cheat Sheet

Istilah	Makna operasional
problem_framing	Peran inti dalam mengintegrasikan data, training, evaluasi, ekspor, dan monitoring.
baseline	Peran inti dalam mengintegrasikan data, training, evaluasi, ekspor, dan monitoring.
deployment	Peran inti dalam mengintegrasikan data, training, evaluasi, ekspor, dan monitoring.
governance	Peran inti dalam mengintegrasikan data, training, evaluasi, ekspor, dan monitoring.

Checklist siap pakai

- Verifikasi `problem_framing` sebelum eksekusi utama.
- Catat perubahan pada `baseline` dan `deployment`.
- Ukur `governance` dengan `baseline` yang adil.
- Tambahkan test untuk mencegah: optimasi teknis dilakukan sebelum kriteria sukses disepakati.
- Simpan artefak proyek: Sistem ML end-to-end.

API yang muncul

problem_framing, baseline, deployment, governance

SATU KALIMAT Bab 48 mengajarkan cara mengintegrasikan data, training, evaluasi, ekspor, dan monitoring dengan kontrak eksplisit dan bukti terukur.

APPENDIX 01 / 12

Appendix A - Template Notebook Colab

Gunakan urutan: metadata, install, imports, seed, config, data, model, training, evaluation, export, conclusion.

- Pisahkan sel konfigurasi dari implementasi.
- Gunakan fungsi untuk tahap yang diulang.
- Hindari variabel global yang tidak terdokumentasi.
- Tambahkan assert pada shape dan nilai finite.

COLAB - TEMPLATE

```
1 # 0. Metadata
2 CONFIG = {'seed': 42, 'device': 'cuda', 'run_name': 'baseline-v1'}
3 # 1. Seed dan data
4 seed_everything(CONFIG['seed'])
5 train_loader, valid_loader = build_data(CONFIG)
6 # 2. Model dan training
7 model = build_model(CONFIG).to(device)
8 result = train_and_evaluate(model, train_loader, valid_loader, CONFIG)
9 # 3. Simpan
10 torch.save(result, '/content/result.pt')
```

GUNAKAN SEBAGAI REFERENSI Kembali ke appendix ini saat membangun notebook baru atau melakukan review kode.

APPENDIX 02 / 12

Appendix B - Cheat Sheet Tensor

Operasi inti tensor harus dipilih berdasarkan semantik bentuk, bukan sekadar agar kode berjalan.

- reshape/view: ubah bentuk dengan memperhatikan kontiguitas.
- permute/transpose: ubah urutan dimensi.
- stack/cat: tambah dimensi baru atau gabungkan dimensi yang ada.
- einsum: nyatakan kontraksi dengan indeks eksplisit.

No.	Pemeriksaan
1	reshape/view: ubah bentuk dengan memperhatikan kontiguitas.
2	permute/transpose: ubah urutan dimensi.
3	stack/cat: tambah dimensi baru atau gabungkan dimensi yang ada.
4	einsum: nyatakan kontraksi dengan indeks eksplisit.
GUNAKAN SEBAGAI REFERENSI Kembali ke appendix ini saat membangun notebook baru atau melakukan review kode.	

APPENDIX 03 / 12

Appendix C - Cheat Sheet Autograd

Gradien mengalir hanya melalui operasi yang dilacak dan tensor yang terhubung ke loss.

- `requires_grad_`: aktifkan tracking.
- `backward`: akumulasi gradien leaf tensor.
- `autograd.grad`: ambil gradien secara fungsional.
- `detach/no_grad/inference_mode`: putus atau nonaktifkan tracking.

No.	Pemeriksaan
1	<code>requires_grad_</code> : aktifkan tracking.
2	<code>backward</code> : akumulasi gradien leaf tensor.
3	<code>autograd.grad</code> : ambil gradien secara fungsional.
4	<code>detach/no_grad/inference_mode</code> : putus atau nonaktifkan tracking.
GUNAKAN SEBAGAI REFERENSI Kembali ke appendix ini saat membangun notebook baru atau melakukan review kode.	

APPENDIX 04 / 12

Appendix D - Rumus Deep Learning

Gunakan rumus untuk memeriksa implementasi dan skala numerik.

- Linear: $y = xW^T + b$.
- Cross entropy: $-\log$ probabilitas kelas benar.
- AdamW: adaptive moments dengan decoupled weight decay.
- Attention: $\text{softmax}(QK^T / \sqrt{d})V$.

GUNAKAN SEBAGAI REFERENSI Kembali ke appendix ini saat membangun notebook baru atau melakukan review kode.

APPENDIX 05 / 12

Appendix E - Debugging Decision Tree

Mulai dari error paling lokal: data -> shape -> dtype/device -> forward -> loss -> backward -> optimizer -> metric.

- Exception shape: cetak dimensi pada batas modul.
- NaN/Inf: cek input, loss scale, learning rate, dan operasi tidak stabil.
- GPU idle: profil input pipeline dan transfer.
- Akurasi stagnan: overfit satu batch sebelum memperbesar data.

No.	Pemeriksaan
1	Exception shape: cetak dimensi pada batas modul.
2	NaN/Inf: cek input, loss scale, learning rate, dan operasi tidak stabil.
3	GPU idle: profil input pipeline dan transfer.
4	Akurasi stagnan: overfit satu batch sebelum memperbesar data.

GUNAKAN SEBAGAI REFERENSI Kembali ke appendix ini saat membangun notebook baru atau melakukan review kode.

APPENDIX 06 / 12

Appendix F - GPU dan Memori

Memori GPU terdiri dari parameter, gradien, optimizer state, activation, workspace, dan cache allocator.

- Gunakan mixed precision bila sesuai.
- Ukur peak memory setelah warm-up.
- Kurangi activation lewat checkpointing.
- Jangan menyamakan reserved dengan allocated memory.

No.	Pemeriksaan
1	Gunakan mixed precision bila sesuai.
2	Ukur peak memory setelah warm-up.
3	Kurangi activation lewat checkpointing.
4	Jangan menyamakan reserved dengan allocated memory.

GUNAKAN SEBAGAI REFERENSI Kembali ke appendix ini saat membangun notebook baru atau melakukan review kode.

APPENDIX 07 / 12

Appendix G - Reproducibility Checklist

Reproducibility adalah properti pipeline lengkap, bukan hanya satu seed.

- Kunci seed Python, NumPy, CPU, dan CUDA.
- Simpan versi paket dan konfigurasi.
- Catat split data dan preprocessing.
- Dokumentasikan perangkat serta mode deterministik.

No.	Pemeriksaan
1	Kunci seed Python, NumPy, CPU, dan CUDA.
2	Simpan versi paket dan konfigurasi.
3	Catat split data dan preprocessing.
4	Dokumentasikan perangkat serta mode deterministik.

GUNAKAN SEBAGAI REFERENSI Kembali ke appendix ini saat membangun notebook baru atau melakukan review kode.

APPENDIX 08 / 12

Appendix H - Testing Strategy

Model membutuhkan test unit, integration, numerical, regression, dan serving contract.

- Uji output shape dan finite.
- Uji state_dict round-trip.
- Uji satu step training menurunkan loss pada batch kecil.
- Uji endpoint dengan input valid dan invalid.

No.	Pemeriksaan
1	Uji output shape dan finite.
2	Uji state_dict round-trip.
3	Uji satu step training menurunkan loss pada batch kecil.
4	Uji endpoint dengan input valid dan invalid.

GUNAKAN SEBAGAI REFERENSI Kembali ke appendix ini saat membangun notebook baru atau melakukan review kode.

APPENDIX 09 / 12

Appendix I - Production Quality Gates

Rilis hanya jika kualitas model dan kualitas sistem memenuhi ambang yang disepakati.

- Metric utama melewati baseline.
- Latency dan memory memenuhi SLO.
- Robustness dan subgroup test lulus.
- Rollback dan monitoring tersedia.

No.	Pemeriksaan
1	Metric utama melewati baseline.
2	Latency dan memory memenuhi SLO.
3	Robustness dan subgroup test lulus.
4	Rollback dan monitoring tersedia.

GUNAKAN SEBAGAI REFERENSI Kembali ke appendix ini saat membangun notebook baru atau melakukan review kode.

APPENDIX 10 / 12

Appendix J - Glosarium I

Istilah A-M yang sering muncul dalam rekayasa PyTorch.

- Autograd: mesin diferensiasi otomatis.
- Batch: kumpulan sampel per update.
- Checkpoint: snapshot state untuk resume atau deployment.
- Gradient: turunan loss terhadap parameter.
- Module: unit komposisi model.

Istilah	Definisi
Autograd	mesin diferensiasi otomatis.
Batch	kumpulan sampel per update.
Checkpoint	snapshot state untuk resume atau deployment.
Gradient	turunan loss terhadap parameter.
Module	unit komposisi model.

GUNAKAN SEBAGAI REFERENSI Kembali ke appendix ini saat membangun notebook baru atau melakukan review kode.

APPENDIX 11 / 12

Appendix K - Glosarium II

Istilah N-Z yang sering muncul dalam rekayasa PyTorch.

- Optimizer: aturan update parameter.
- Rank: identitas proses distributed.
- Tensor: array multidimensi bertipe dan ber-device.
- Throughput: jumlah sampel per satuan waktu.
- Warm-up: iterasi awal sebelum pengukuran.

Istilah	Definisi
Optimizer	aturan update parameter.
Rank	identitas proses distributed.
Tensor	array multidimensi bertipe dan ber-device.
Throughput	jumlah sampel per satuan waktu.
Warm-up	iterasi awal sebelum pengukuran.

GUNAKAN SEBAGAI REFERENSI Kembali ke appendix ini saat membangun notebook baru atau melakukan review kode.

APPENDIX 12 / 12

Appendix L - Referensi Resmi

Dokumentasi primer untuk memperbarui contoh saat versi berubah.

- <https://docs.pytorch.org/docs/stable/>
- <https://docs.pytorch.org/tutorials/>
- <https://research.google.com/colaboratory/faq.html>
- <https://docs.pytorch.org/executorch/>
- <https://captum.ai/>
- <https://onnx.ai/>

VERSI NASKAH Referensi diperiksa pada 29 Agustus 2026. API dan runtime cloud dapat berubah; utamakan dokumentasi stabil terbaru.

PENUTUP Kembali ke appendix ini saat membangun notebook baru atau melakukan review kode.